03

Enterprise Architecture

Book 3 · Strategy, governance and transformation

Frameworks, decisions and evidence for enterprise change

17 chapters · 97 exercises · 18 figures

A ten-week route from enterprise analysis to a three-year roadmap

Revised edition
October 2026

IT Architecture Series. Revised edition, October 2026.
All organisations, people, workloads, budgets and datasets in this book are fictional teaching examples. Any resemblance to real organisations is coincidental.
Product and company names are trademarks of their respective owners and are used only to identify their products. This book is a study text, not professional, legal or financial advice.
Set in Nimbus Sans and DejaVu Sans Mono.

Contents

How to study enterprise architecture 3
01 The enterprise architecture problem 5
02 Enterprise architecture frameworks, methods and modelling 11
03 Business capabilities, value streams and requirements 18
04 Operating models and strategic alignment 22
05 Application portfolios and rationalisation 28
06 Enterprise information and data governance 32
07 Technology strategy and shared platforms 37
08 Governance, decision rights and exceptions 42
09 Enterprise risk, security and assurance 47
10 Investment economics, benefits and uncertainty 51
11 Roadmaps, dependencies and transition architectures 57
12 Organisational change, adoption and architecture leadership 62
13 Sourcing, procurement and supplier ecosystems 67
14 Measuring enterprise outcomes and architecture value 71
15 Innovation, AI and sustainable technology decisions 76
16 Research methods and academic synthesis 79
17 Capstone transformation programme and board defence 82
Exercise answers and assessment guidance 85
Glossary 96
Templates and further study 100
References 104
Index 108

How to study enterprise architecture

This book teaches the organisational side of IT architecture. It asks which business capabilities an organisation should improve, how applications and data should support them, who may make each consequential decision, and how a transformation programme can create value that people can see and measure. It suits advanced undergraduate and postgraduate study, and practitioners who already design solutions and now need to work across a whole organisation.

Enterprise architecture draws on several disciplines. You will use ideas from strategy, information systems research, finance, governance, change management and evaluation methods. Each chapter introduces the ideas it needs, so you can study the book on its own. A working knowledge of how applications, databases and integrations behave will help, but the book explains the technical points it relies on.

How the chapters work

Every chapter follows the same pattern:

- **Learning objectives** state what you should be able to do when you finish.
- **Worked examples** apply each idea to one continuing case, usually with numbers you can check.
- **Exercises** mix recall, calculation, design and critique. Answers and marking guidance are in the answers chapter at the end of the book.
- **Key points** summarise the chapter in a few lines for revision.

Chapters 1 and 2 set out the problem and the main frameworks. Chapters 3 to 9 build an enterprise model: capabilities, operating model, applications, data, technology platforms, governance and risk. Chapters 10 to 15 turn that model into decisions about investment, sequencing, change, sourcing, measurement and innovation. Chapter 16 shows how to read research critically, and Chapter 17 is a capstone project that brings everything together.

The continuing case

Helios Services Group has three business units: Industrial Maintenance, Retail Distribution and Digital Services. Acquisitions have left it with forty-three applications, several customer identifiers for the same organisations and service reports that disagree with each other. Annual IT operating expenditure is €12 million. The board has approved a separate three-year transformation envelope of €9 million, released in stages as results come in.

Helios, its people, its budgets and its datasets are fictional teaching examples, as are the other organisations in the book. The numbers are realistic in scale, but they are assumptions for practice rather than benchmarks or supplier prices.

The board wants four things:

- more reliable handoffs between units when a customer needs service;
- a coherent view of each customer across the group;
- less duplication of applications and effort where duplication brings no benefit;
- a platform on which teams can launch new digital services.

Two kinds of identity run through the case, and the book keeps them apart. **Customer identity resolution** decides which records in different systems describe the same customer organisation, site or contact. **Workforce identity and access** controls how Helios's own staff sign in to applications and lose that access when they change role or leave. They have different owners, data, risks and budgets.

Business units remain accountable for their own results, and some of their processes differ for good reasons. "One system for everything" is therefore a hypothesis to test rather than an agreed requirement. Your job, built up chapter by chapter, is to propose a three-year roadmap that is affordable, keeps the business running at every intermediate step and can change when new facts arrive. Two change cards test that last property. The first cuts the envelope by 20%. The second adds an acquired business with a different customer data model.

A ten-week study route

Plan on five days a week at four to five hours a day, or about 200 to 250 hours in total, including reading, modelling, calculation and revision. Keep a dated copy of your enterprise model before each change card so that you can explain how and why your decisions changed.

Week	Chapters	Main output
1	1 and 2	Problem statement and a tailored method for Helios
2	3 and 4	Capability map, heat map and operating-model choice
3	5 and 6	Application dispositions and a fact-authority table
4	7 and 8	Platform policy and decision-rights matrix
5	9 and 10	Service risk register and investment model
6	11 and 12	Dependency roadmap and adoption plan
7	13 and 14	Supplier evaluation and measurement plan
8	15 and 16	Bounded experiment and critical research synthesis
9	17	Three-year transformation dossier
10	17, change cards	Revised programme and board defence

Sources, numbers and judgement

References are numbered in IEEE style in the order of first citation and listed at the end of the book. Different kinds of source support different kinds of claim. For example, a research paper can show what was observed in particular organisations. A standard defines terms and methods. A practitioner book or official guide describes how a method is meant to be used. None of them can show that a particular Helios investment will succeed. That argument has to be built from Helios's own data, and the book shows you how.

Treat every worked number as something to recompute. Several chapters print short Python programs that reproduce their calculations; they use only the standard library and run as printed. Discount rates, costs and benefit estimates are teaching inputs for practice. Where a design depends on law or contract, such as data retention or exit rights, the book frames it as a question to settle with the organisation's specialists.

CHAPTER 1

The enterprise architecture problem

Learning objectives

After this chapter you should be able to:

- Explain how locally sound solutions can create cost and risk for the enterprise as a whole.
- Define enterprise architecture and set the boundary of the enterprise for a given decision.
- Construct a benefit mechanism from an intervention to an outcome, with observable intermediate steps.
- Evaluate a benefit claim by identifying rival explanations and the data needed to test them.
- Describe how an enterprise architecture function is organised: its roles, placement, operating rhythm and measures of effectiveness.

Good solutions can make a poor enterprise

A well-designed solution can still create an enterprise problem. Three examples from Helios show the pattern:

- Retail Distribution introduces a new customer database that fits its workflow well, but uses identifiers that group reporting cannot match to the other units.
- A new cloud service is secure in isolation, but depends on an account-removal process that nobody owns.
- A low-cost order application saves licence fees, but keeps an expensive manual reconciliation between two divisions in place.

Each decision was reasonable for the team that made it. The cost appears only when the decisions are viewed together, across units, systems and years. Enterprise architecture exists to make those relationships visible before they become expensive.

What enterprise architecture is

The international standard for architecture description distinguishes an **architecture**, the abstract set of fundamental concepts and properties of a system or enterprise, from the descriptions that express it [1]. Applied to a whole organisation, **enterprise architecture** describes how capabilities, processes, information, applications, technology and decision rights fit together, and uses that description to choose and sequence change. Ross, Weill and Robertson give a more strategic definition: the organising logic for business processes and IT infrastructure, reflecting the integration and standardisation requirements of the organisation’s operating model [2]. Both definitions point to the same job: deciding what the organisation must share, what it can leave local and how to get from here to there.

The **enterprise** is whatever boundary the decision needs. It may be a group of companies, a public agency, a single division or an ecosystem of partners. State which entities the architect can influence and which are external authorities, such as regulators or suppliers. For Helios, the enterprise is the group and its three units; the customers, suppliers and regulators are outside it but shape its choices. Figure 1 shows the basic structure: local capabilities in each unit, a few shared decisions beneath them, and enterprise outcomes above.

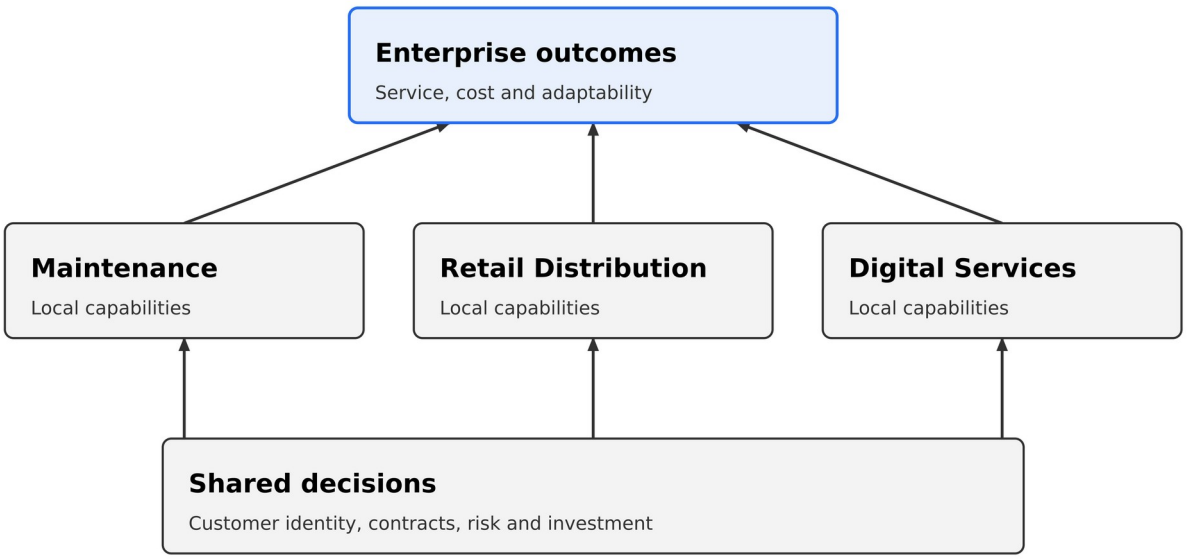


Figure 1 Enterprise architecture coordinates a few shared decisions so that local capabilities add up to enterprise outcomes.

Architecture informs decisions

An enterprise model has value when it changes a decision: whether to consolidate two applications, fund a shared capability, accept an exception or delay a migration. A small, accurate model used in an important investment review is worth more than a complete repository that nobody consults.

Different audiences need different content:

Audience	What they need	Helios example
Board	Choices, costs, consequences and uncertainty	Options for the customer platform, with costs and risks
Capability owner	Process and data implications	How a shared identifier changes the entitlement check
Delivery team	Boundaries, standards and transition duties	Which system is authoritative for each customer fact
Operations	Service dependencies and recovery ownership	What fails if the workforce identity service is down

One giant diagram cannot serve all four.

Drawing the model gives the architect influence, and authority only where it has been delegated. Be explicit about where the role advises, where it sets an agreed standard and where an accountable executive decides. A technically sound recommendation can fail because nobody addressed the decision rights and incentives around it.

Organising the enterprise architecture function

Enterprise architecture is also a function inside the organisation, with people, meetings and a place in the reporting lines. Van der Raadt and van Vliet describe the **enterprise architecture function** as the organisational functions, roles and bodies involved in creating, maintaining, ratifying, enforcing and observing architecture decisions. These work together at enterprise, domain, project and operational levels [3]. Designing the function is itself an architecture decision.

Roles

Most organisations distinguish three architect roles, although the titles vary:

- **Enterprise architects** work across the whole organisation. They maintain the principles, the capability map, the target state and the roadmap, and advise on investment.
- **Domain architects** cover one area, such as data, integration, security or technology, or one business unit. They turn enterprise direction into domain standards and reference designs.

- **Solution architects** design a specific change, such as replacing an order system, within those standards, and record the project's decisions.

Helios starts small: a head of architecture acting as enterprise architect, two domain architects (one for data and integration, one for security and platforms), and solution architects who sit in the delivery teams of each business unit.

Placement

Where the function reports shapes what it sees and what it can influence. Reporting to the CIO keeps it close to technology but can make it look like an IT specialism. Reporting to a strategy or transformation office keeps it close to investment decisions but risks losing touch with delivery. Many groups choose a **federated** arrangement: a small central team for enterprise-wide work, with domain and solution architects embedded in the business units. Helios chooses this arrangement, with the central team reporting to the CIO and attending the transformation board, because its main decisions cut across units while its delivery stays local.

Operating rhythm and the architecture board

An **architecture board** is a cross-unit body that reviews significant designs, approves standards and exceptions within the authority delegated to it, and sends investment and risk decisions to their accountable owners (Chapter 8). The function also needs a predictable rhythm, tied to the decisions it serves. For Helios:

Cycle	Activity	Decision it feeds
Fortnightly	Architecture board reviews designs that meet the review triggers	Approval of designs and time-limited exceptions
Monthly	Portfolio and risk update: inventory changes, overdue exceptions	Platform and security priorities
Quarterly	Roadmap review against delivery and benefits	Release of the next funding stage
Annually	Review of principles, standards and the capability map	Strategy and budget round

Measuring the function

Judge the function by its effect on decisions rather than by its output of documents. Useful measures include the share of significant investment decisions that used current architecture information, the time taken to review a design, the number of exceptions past their expiry date and the reuse of shared services. Survey research on enterprise architecture management supports this emphasis: intention to use the architecture and the organisational and project benefits it produced worked as measures of success, while satisfaction with it did not [4]. Chapter 14 shows how to measure such effects with care.

Define the baseline honestly

Helios's forty-three applications are an initial inventory count, which is far from a complete model of how the group works. For each application, record its business purpose, owner, users, cost, the facts it is authoritative for, its interfaces, its criticality, its lifecycle status and how confident the team is in each fact. An application with no known owner is a question to investigate, not a retirement candidate.

Show the current state and the desired state separately. For example, if the group wants a shared customer identity but each division has its own identifiers today, draw both, together with the uncertain mapping between them. A future-state label on a current-state diagram hides the work the transformation must do.

From slogan to mechanism

The board's phrase "reduce duplication" could mean at least four different things:

- fewer paid licences for applications that do the same job;
- fewer inconsistent copies of the same customer data;
- less repeated engineering effort across teams;
- fewer redundant steps in business processes.

Each has a different mechanism and a different measure. For example, retiring a CRM might cut subscription cost but add work for a division whose process the replacement supports poorly. A shared customer model might improve reporting without any CRM consolidation at all.

Three kinds of statement are easy to confuse. A **definition** says what a term covers. A **mechanism** explains how one change could lead to another. A **causal claim** says what would happen to an outcome if an intervention were made, compared with an alternative. Agreeing on the definition of a capability map does not show that producing one reduces operating cost. Chapter 16 develops these distinctions further.

A **benefit chain** makes the mechanism explicit, one observable step at a time:

1. A governed customer identifier makes previously ambiguous matches resolvable.
2. Resolvable matches reduce manual reconciliation.
3. Less reconciliation frees staff capacity.
4. A decision about staffing, contracts or new work turns that capacity into a financial benefit.

Each step needs an owner and something that can be observed. The existence of a master-data tool establishes none of them. Research on the value of enterprise architecture proposes similar intermediate mechanisms, such as better alignment and better information availability; Chapter 14 examines what that research can and cannot show [5].

Worked example: a mechanism and its rivals

Define the intervention narrowly: a verified customer-link service, plus a staffed correction process, for two divisions. Define the outcome as the proportion of eligible service requests that are reassigned because the customer or service site was misidentified. The unit of observation is the individual service request, even though the funding decision is made for the programme.

Start with the size of the problem. Last quarter, 410 of 5,200 eligible maintenance requests were reassigned, a rate of 7.9%. A review of 100 of those reassignments found that 46 were caused by a wrong customer or site. If that sample is representative, about 189 requests, or 3.6% of all eligible requests, were reassigned for identity reasons. Even perfect identity resolution could therefore remove at most about 3.6 percentage points of reassignment. The other 4.3 points have different causes, and the business case must not claim them.

Figure 2 shows the proposed mechanism and one competing influence.

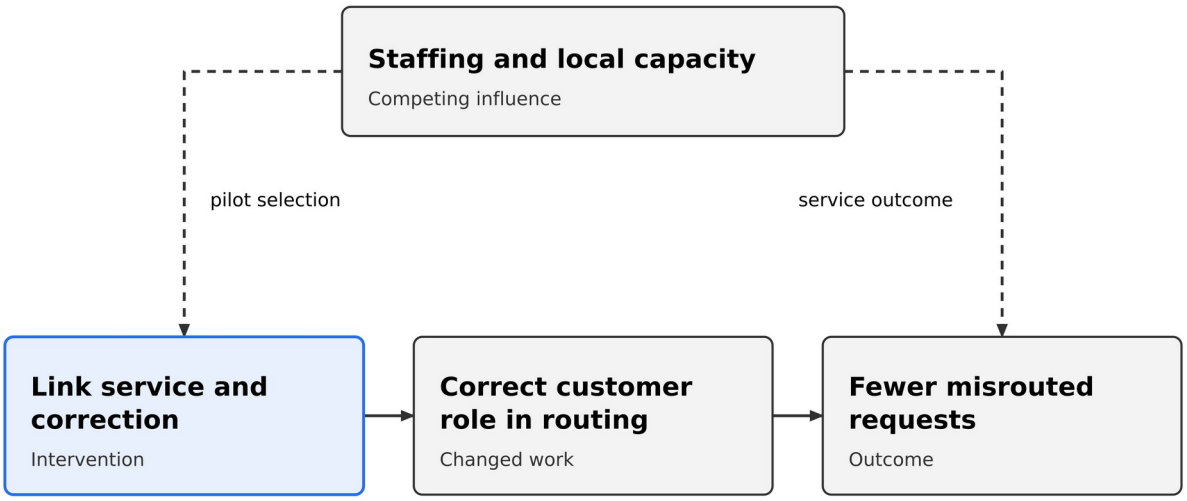


Figure 2 Proposed benefit mechanism (solid arrows) and a competing influence (dashed arrows) in the Helios pilot.

The solid arrows show the intended route: the link service changes how requests are routed, which reduces misrouting. The dashed arrows show that staffing could influence both which division is chosen for the pilot and how well that division performs. If the best-staffed division receives the pilot first, its later performance cannot simply be credited to the link service.

When the prediction fails

Work backwards from a failed prediction. Suppose verified links increase but misrouting does not fall:

- If routing does not actually consume the links, the fix is an integration change.
- If routing uses the links but selects the legal party when it needs the service site, the fix is a change of meaning in the data.
- If routing is correct but resolution is still slow, the binding constraint may be technician availability, which needs a capacity decision.

Buying a larger customer-matching platform would address none of these.

Compare a cheaper alternative too. Helios could keep its local identifiers and set up a small cross-division correction desk with an agreed response time. At low volumes it might give an adequate service and reveal how much ambiguity really exists, but its labour cost, coverage and handoff risk continue. The automated option needs implementation and stewardship, error monitoring and a fallback for unresolved matches. Choose between them on volume, consequence and observed correction effort.

Before collecting data, record the eligible workflows, the observation period, the treatment of unresolved cases and the quality checks. Keep every eligible request in the denominator, even those that cannot be matched, so that nobody can improve the figures by excluding difficult cases. Agree a decision rule in advance: expand only when the measured improvement and a sustainable support burden justify the next commitment.

A first decision for Helios

The board asks whether to mandate one CRM across the group immediately. A defensible first response proposes a short, time-boxed analysis before any group-wide procurement. In six weeks, the team will deliver:

- an assessment of how the divisions' service processes differ, and which differences are required by contracts or customers;
- a proposal for who is authoritative for each shared customer fact;
- one feasible consolidation option and one staged alternative, each with costs.

Meanwhile, the team fixes the handoffs that cause the most misrouted requests, because those fixes are useful whatever the CRM decision. If the divisions turn out to share essentially the same process and one product supports it, consolidation may be justified. If their differences are material, common data contracts and governance may create more value first.

Exercises

1.1 Give an example of a technically good local solution that increases enterprise cost or risk. Explain the mechanism.

1.2 Translate "reduce duplication" into three different measurable outcomes for Helios, each with a baseline and an owner.

1.3 Build a benefit chain for governed customer identity, and name one observation that would challenge each link.

1.4 In the next quarter, 5,000 eligible requests are received and 330 are reassigned. A review of 100 reassignments finds 40 caused by a wrong customer or site. Estimate the identity-related reassignment rate and compare it with last quarter. State two reasons why the comparison may mislead.

1.5 Design the brief for the six-week CRM analysis: its questions, outputs, owner, deadline and the decision rule the board will apply to its results.

1.6 A vendor tells the board that "organisations with mature enterprise architecture are more profitable, so investing in our modelling tool will raise your profits". Evaluate the claim using the distinction between definitions, mechanisms and causal claims.

1.7 The Helios board asks whether its three-person central architecture team should report to the CIO or to the transformation programme director. Compare the two placements, propose a meeting rhythm for the architecture board, and choose three measures of the function's effectiveness.

Key points

- Locally sound decisions can create enterprise-level cost and risk. Enterprise architecture makes the relationships between decisions visible.
- Enterprise architecture describes how capabilities, processes, information, applications, technology and decision rights fit together, and uses that description to choose and sequence change.
- A model is valuable when it changes a decision; different audiences need different views.

- Benefits must be argued through explicit mechanisms with observable steps, and bounded by the size of the problem.
- Every benefit claim needs rival explanations and a decision rule agreed before the data arrives.
- An architecture function needs clear roles, a deliberate placement, a rhythm tied to decisions and measures of its effect on those decisions.

CHAPTER 2

Enterprise architecture frameworks, methods and modelling

Learning objectives

After this chapter you should be able to:

- Distinguish a taxonomy, a method, a modelling language and a practice, and explain what each contributes.
- Classify enterprise artefacts with the Zachman Framework 3.0 and use the result to find gaps.
- Tailor the TOGAF Architecture Development Method to a specific decision and write architecture principles in a standard format.
- Model a service view in ArchiMate 4 using its domains, aspects and relationships, and explain what changed from ArchiMate 3.2.
- Evaluate framework claims against empirical studies of how enterprise architecture is practised.

Four kinds of help

In one week the Helios architect receives three requests. The board wants options for the customer platform. The data steward asks what “customer” should mean across the group. A delivery team wants to know which systems depend on the workforce identity service. Each request needs different content, a different level of detail and a different audience. Frameworks help the architect decide what to describe, in what order to work and how to draw it.

The word “framework” covers several different kinds of help, and confusing them causes trouble:

- A **taxonomy** classifies the things an enterprise description could contain. The Zachman Framework is the classic example.
- A **method** sets out the steps for producing and using architecture. The TOGAF Architecture Development Method is the most widely taught.
- A **modelling language** defines element types, relationships and notation so that models mean the same thing to every reader. ArchiMate is the open standard for enterprise models.
- A **practice** is what architects in a real organisation actually do: which artefacts they keep, which meetings they attend and which decisions they influence.

An international standard ties these together. ISO/IEC/IEEE 42010 separates an **architecture**, which is abstract, from an **architecture description**, the tangible work product that expresses it [1]. Stakeholders have **concerns**; a **viewpoint** sets the conventions for addressing particular concerns, and it governs one or more **views**, each of which applies those conventions to a particular system or enterprise. For example, a “board investment” viewpoint might specify a table of options, costs and risks; the Helios customer-platform options paper is a view built to that viewpoint.

Kind	Question it answers	Example	What it leaves to you
Taxonomy	What could we describe?	Zachman Framework	Order of work and depth
Method	In what order do we work?	TOGAF ADM	Which steps matter here
Modelling language	How do we express it precisely?	ArchiMate	Which views to draw
Practice	What do architects actually do?	Empirical studies	Fit to your organisation

The Zachman Framework: a classification scheme

John Zachman’s 1987 paper started from an analogy with building and manufacturing [6]. A building is described differently for the owner, the architect and the builder, and each description is complete for its purpose. Zachman argued that information systems need the same family of descriptions. His original grid had three columns, for data (what), function (how) and network (where), and he noted that who, when and why descriptions were also needed. Sowa and Zachman later extended the scheme to all six questions,

with rows for the planner, owner, designer, builder and subcontractor [7]. Older texts still use those row names.

The current version, 3.0, appeared in 2011 under the subtitle “The Enterprise Ontology” [8]. Zachman presents it as an **ontology**: a structured classification of the descriptions an enterprise can have, which deliberately says nothing about how architecture work should be done. Its grid has six columns and six rows, 36 cells in all:

- The **columns** are the six interrogatives, each named for the kind of description it holds: what (inventory), how (process), where (distribution), who (responsibility), when (timing) and why (motivation).
- The **rows** are audience perspectives, from the most abstract to the most concrete: the executive’s scope contexts, the business owner’s business concepts, the architect’s system logic, the engineer’s technology physics, the technician’s tool components and, in the bottom row, the operating enterprise itself.

Each cell is a distinct kind of description. For example, the cell where the why column meets the business-concepts row holds the business’s goals and rules, while the cell where the what column meets the system-logic row holds a logical data model.

Worked example: classifying Helios artefacts

The Helios architecture team lists six documents it already holds and classifies each one.

Helios artefact	Column	Row
List of business units and service regions	Where (distribution)	Scope contexts (executive)
Definition of “customer organisation”	What (inventory)	Business concepts (business owner)
Service-request handling process	How (process)	Business concepts (business owner)
Customer-link data model	What (inventory)	System logic (architect)
Integration schedule for nightly extracts	When (timing)	Technology physics (engineer)
Database table definitions for CRM North	What (inventory)	Tool components (technician)

The table shows a pattern. Helios has several what and how descriptions but nothing in the why (motivation) column. No document records the goals or rules that justify consolidating customer data, so every debate about the CRM restarts from opinion. That single gap is the useful output. Completing all 36 cells for an enterprise of Helios’s size would cost far more than it returns, and the bottom row is the running enterprise rather than a model of it.

Zachman is therefore best used as a checklist. Ask which cells a decision depends on, check that those cells are described at the right level, and stop there.

TOGAF and the Architecture Development Method

The TOGAF Standard, published by The Open Group, is a framework for developing and governing enterprise architecture [9]. Its 10th Edition separates a stable core, the **fundamental content**, from a library of **series guides** on particular topics. At its centre is the **Architecture Development Method (ADM)**, a cycle of phases for developing and changing an architecture [10]. Figure 3 shows the cycle.

The phases are:

1. **Preliminary.** Establish the architecture capability: scope, principles, governance and how the method will be tailored.

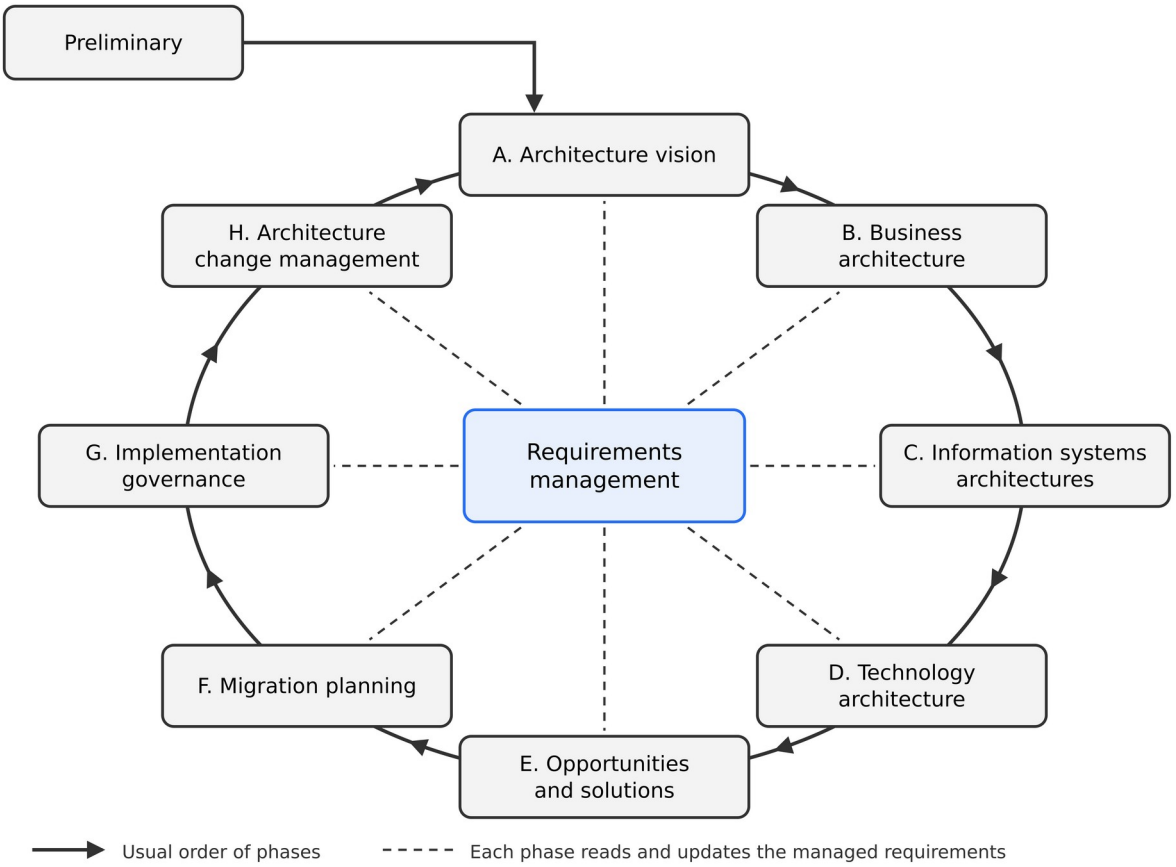


Figure 3 The TOGAF Architecture Development Method: a preliminary phase, eight phases in a cycle, and requirements management at the centre.

2. **A. Architecture vision.** Agree the scope of one cycle, the stakeholders, the business goals and a high-level target.
3. **B. Business architecture.** Describe the baseline and target business: capabilities, processes, organisation and information needs.
4. **C. Information systems architectures.** Describe the data and application architectures that support the business.
5. **D. Technology architecture.** Describe the technology services and platforms the applications need.
6. **E. Opportunities and solutions.** Group the changes into work packages and identify transition architectures.
7. **F. Migration planning.** Sequence the work into a costed, prioritised roadmap.
8. **G. Implementation governance.** Check that delivery projects conform to the architecture.
9. **H. Architecture change management.** Monitor changes in the business and technology, and decide when a new cycle is needed.

Requirements management sits at the centre because requirements change throughout the cycle. Each phase reads them and adds to them.

Tailoring the ADM

The ADM looks sequential on paper, but TOGAF itself expects organisations to adapt it: to iterate within and between phases, to vary the depth of each phase, and to fit the method to their own governance [10]. A literal pass through every phase for every decision would be slow and would produce many unused documents.

Tailoring starts from the decision the cycle must support. For Helios's first decision, whether and how to share customer identity across two divisions, a ten-week tailored cycle might look like this:

Phase and depth	Helios deliverable	Reason
Preliminary: light, 1 week	Principles, decision rights for customer data	Reuse existing governance where it works
A: full, 2 weeks	Scope: two divisions; named sponsor and owners	Scope disputes are the main risk
B and C: full, 4 weeks	Customer facts, owners, matching process, affected applications	This is where the uncertainty lies
D: light, 1 week	Confirm hosting on the existing platform	No new technology is proposed
E and F: full, 2 weeks	Options, transition states, pilot plan and costs	The board decides on this output
G and H: set up, then ongoing	Pilot governance and review triggers	Needed once the pilot starts

The tailored cycle spends most of its time where Helios is uncertain and moves quickly where existing decisions still hold. It also combines phases B and C, because the customer question is both a business question and a data question.

Architecture principles

The Preliminary phase usually produces a short set of **architecture principles**. TOGAF recommends a four-part format for each one [11]:

- **Name:** a short, memorable label.
- **Statement:** the rule, stated plainly.
- **Rationale:** the business benefit of following the rule.
- **Implications:** what the organisation must do or accept as a result, including costs.

Here are two Helios principles in that format.

Name	Statement	Rationale	Implications
One authority for each shared fact	Every business fact shared between units has one named authoritative source and owner.	Conflicting copies cause most of Helios's reconciliation work.	New integrations must name their source; owners need funded time; copies are read-only.
Share data before processes	Units share customer data and identifiers first; common processes follow only where analysis shows they fit.	Unit processes differ for contractual reasons that have not yet been analysed.	The CRM decision waits for process analysis; the identity work starts now.

A good principle can be broken, so it guides real choices. "We value high-quality data" cannot be broken and so guides nothing. The first principle, by contrast, rules out a proposal in which two CRMs both edit the customer's legal name.

TOGAF warns that too many principles reduce the flexibility of the architecture, and notes that many organisations prefer a small set of high-level principles, between 10 and 20 [11]. It describes a good set as understandable, robust, complete, consistent and stable. Consistency deserves attention: strict adherence to one principle may require a looser reading of another, so the set should say which wins. Give each principle an owner, and review the set when strategy changes.

ArchiMate: a modelling language

ArchiMate is The Open Group's modelling language for enterprise architecture. The current version, the ArchiMate 4 Specification, was published in April 2026 [12]. It gives each kind of element a defined meaning and a symbol, and it restricts which relationships are allowed between them. Two people reading the same ArchiMate model should draw the same conclusions, which is rarely true of informal box-and-line diagrams.

Domains and aspects

ArchiMate 4 organises its elements into **domains**:

- The **common domain** holds generic elements used everywhere: the behaviour elements service, process, function and event; the active structure elements role, collaboration and path; and the grouping and location elements.
- The **business**, **application** and **technology** domains, together called the core domains, hold the elements specific to each. Examples are business actors and business objects; application components and data objects; and nodes, devices, system software, artifacts and physical elements such as equipment and facilities.
- The **strategy** domain models capabilities, value streams, resources and courses of action.
- The **motivation** domain models stakeholders, drivers, assessments, goals, outcomes, principles, requirements, meaning and value. It explains why the architecture is the way it is.
- The **implementation and migration** domain models work packages, deliverables and **plateaus**, which are relatively stable states of the architecture, such as a baseline, a transition state and a target.

Elements also belong to one of three **aspects**. **Active structure** elements perform behaviour, such as a role or an application component. **Behaviour** elements are the work done, such as a process or a service. **Passive structure** elements are acted upon, such as a business object or a data object.

The most visible design choice in version 4 is that behaviour is no longer split by domain. There is one process, one function, one service and one event, and the same process can be performed by people, by software or by both together [12], [13]. A modeller no longer has to decide early whether a step is manual or automated, which suits work in which people and machines collaborate. Roles are generic in the same way: any internal active structure element, including an application component, can be assigned to a role.

Relationships come in groups. **Structural** relationships (composition, aggregation, assignment and realisation) say what an element is made of or what it makes real. **Dependency** relationships (serving, access, influence and association) say how elements use each other. **Dynamic** relationships (triggering and flow) show order and transfer between behaviours. Specialisation completes the set. Relationships can now also carry a **multiplicity**, which limits how many instances may sit at each end; for example, a rule that every service is realised by at least one process.

Worked example: a Helios service view

Figure 4 models how Helios handles a maintenance service request today.

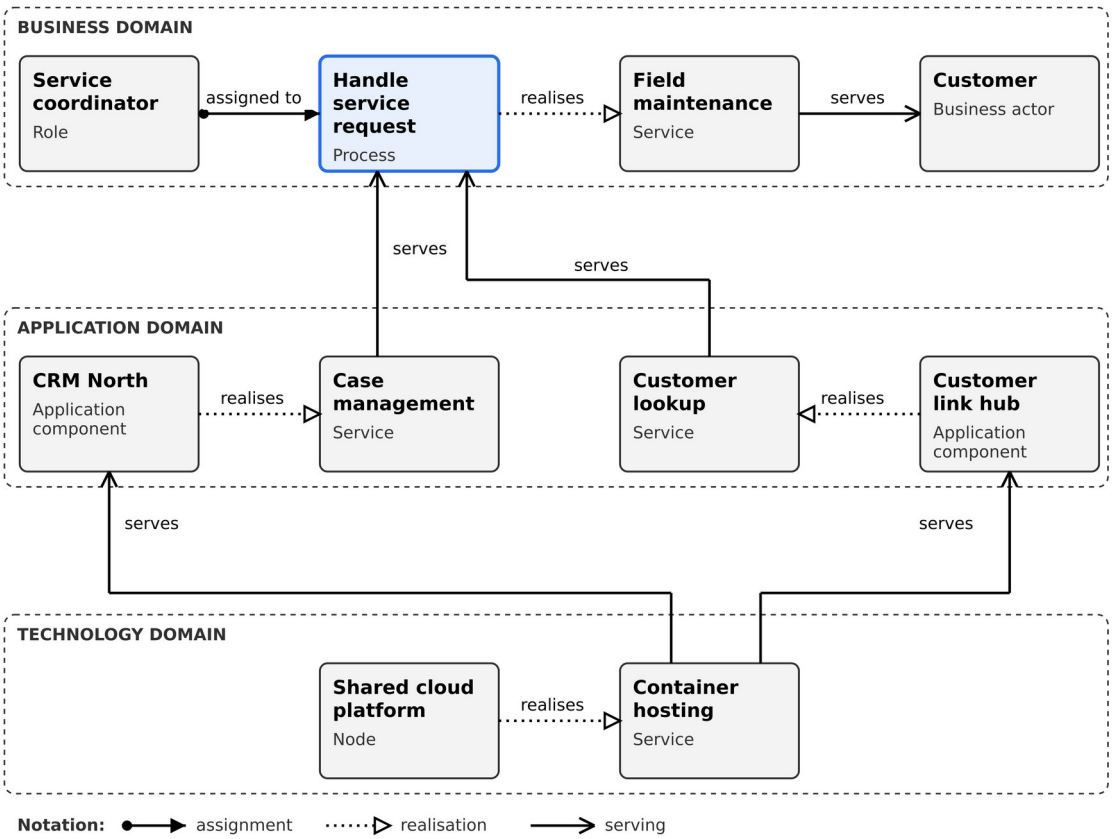


Figure 4 A Helios service-fulfilment view drawn in ArchiMate 4, arranged by business, application and technology domain. Every relationship is labelled and drawn in ArchiMate line style; element symbols are simplified to boxes.

Read the view from the top. A service coordinator role is **assigned to** the process “Handle service request”, which **realises** the service “Field maintenance” that **serves** the customer, a business actor. Two services from the application domain serve that process: case management, realised by CRM North, and customer lookup, realised by the customer link hub. Both components are served by a container hosting service, realised by the shared cloud platform. In ArchiMate 4 all four services are the same kind of element; the view places each in the domain of the elements that realise it.

The view answers a precise question: what does the maintenance process depend on? If the customer link hub fails, the coordinator loses customer lookup but keeps case management. If the container hosting service fails, both application services are lost at once, a shared dependency that Chapter 9 returns to. In the terms of ISO/IEC/IEEE 42010, this is a view built to a layered viewpoint suited to an operations audience; the specification’s appendix offers example viewpoints for other purposes, including capability maps, value streams and migration. The board would need a different view of the same model, for example one showing capabilities and work packages.

What changed from ArchiMate 3.2

Many organisations hold repositories drawn in ArchiMate 3.2 [14], so architects will meet both versions for some years. The main differences are these:

- **Layers became domains.** The framework is now drawn as concentric hexagons rather than a matrix of layers and aspects. Lankhorst explains that “layer” was often misread as a level of abstraction or a strict hierarchy [13].
- **Behaviour is merged.** The separate business, application and technology processes, functions, services and events of 3.2 are now one generic set in the common domain, and the implementation event is replaced by the generic event.
- **Roles and collaborations are generic.** The business role is replaced by a single role element, and the business, application and technology collaborations by a single collaboration.
- **Some elements were removed.** The three interaction elements, constraint, contract, gap and representation no longer exist. Suggested replacements include a process with several participants for an interaction, a requirement for a constraint, a business object for a contract, and an assessment between two plateaus for a gap [13].
- **Relationships can have multiplicity.**
- **The language is smaller.** Lankhorst counts 42 concepts in version 4 against 60 in version 3.2, a reduction of about 30%.

The aspects, the relationship groups (including composition, which version 4 keeps as a structural relationship) and most structure elements, such as business actors, application components, nodes and data objects, carry over unchanged. Converting a 3.2 model is therefore mainly a matter of retyping behaviour elements and replacing the removed ones; for example, a 3.2 “business process” and an “application process” both become a process in version 4.

Capability-based planning

Many organisations plan change by project or by application. **Capability-based planning** plans and funds change by business capability instead, and it treats projects as the means of delivering capability increments [15]. For example, Helios’s capability “Customer identity resolution” could improve in three increments:

1. Verified links between customer records in two divisions.
2. Verified links across all three divisions, with a correction service.
3. Customer self-service correction of contact and site details.

Each increment needs work from several projects: data cleansing, an integration change, training and new support roles. Funding the increment rather than the projects keeps attention on the business ability the board is paying for. It also connects the strategy domain to the implementation and migration domain: each increment corresponds to a plateau in the roadmap. Chapter 3 builds the capability map on which such planning depends.

How enterprise architecture is practised

Frameworks describe how enterprise architecture could be done. Empirical research asks how it is done. Kotusev has studied this question through case studies and interviews with practitioners. In an in-depth case study of one organisation whose practice was described as TOGAF-based, he concluded that none of the TOGAF-specific recommendations proved useful, and that the practice hardly resembled any established EA methodology [16]. In an analysis of the artefacts used in 27 diverse organisations, he

identified 24 common artefacts that proved useful in practice and described their usage and purpose. He argued that the familiar picture of enterprise architecture as layered current-state and future-state descriptions linked by a roadmap does not match how practitioners work [17].

These studies are qualitative and cover a limited number of organisations, so they show what happened in those cases rather than what must happen everywhere. They still support three practical conclusions for Helios:

- Treat frameworks as libraries of ideas to select from.
- Judge architecture work by the decisions it changes rather than by the number of artefacts produced.
- Keep the artefacts that people actually consult, and keep them current.

> **Common mistake.** Announcing that an organisation “uses TOGAF” and measuring progress by deliverables completed. Progress is better measured by whether investment, design and governance decisions now use architecture information.

Choosing and combining frameworks

Most organisations combine elements from several sources. Helios might choose:

- Zachman as a periodic checklist for gaps in its descriptions;
- a tailored ADM cycle for each major decision, using the phase names as a shared vocabulary;
- ArchiMate for the repository, with a few agreed viewpoints for the board, data owners and operations;
- the C4 model for software views inside delivery teams, where a lighter notation suits developers [18].

The combination works if each element has a stated purpose and an owner. It fails if the organisation adopts all of them in full and then has no time left to make decisions.

Exercises

2.1 Explain the difference between a taxonomy, a method, a modelling language and a practice. Give one Helios example of each.

2.2 Classify the following Helios artefacts in the Zachman Framework: the board’s three-year goals; the technician on-call rota; the network diagram of the Retail warehouses; the entitlement rules for maintenance contracts; the job schedule that loads the data warehouse each night; a list of data owners. Identify the most important remaining gap for the customer-identity decision.

2.3 Change card B adds an acquired business whose customer data is organised by service site and contract. Design an eight-week tailored ADM cycle for deciding how to integrate its customer data. For each phase state the deliverable, the depth and the reason.

2.4 Write one further architecture principle for Helios in the four-part format. Then describe one real proposal the principle would rule out or require.

2.5 Using Figure 4, state the domain and aspect of every element. Then add Retail Distribution’s CRM Central to the view: which elements and relationships would you add, and what new shared dependency does the view reveal? Finally, name the ArchiMate 3.2 element type that each behaviour element in the view would have had.

2.6 A consultant tells the Helios board: “Adopt TOGAF in full and your architecture work will be done properly.” Critique this claim using the empirical studies discussed in this chapter, and propose a better measure of progress.

Key points

- Frameworks provide four different kinds of help: a taxonomy of what to describe, a method for the order of work, a modelling language for precise expression and, through research, evidence of what practice looks like.
- Zachman’s grid of six perspectives and six interrogatives is an ontology of descriptions. Use it to find the gaps that matter for a decision.
- The TOGAF ADM is a cycle of phases around requirements management. Tailor its depth to the decision, and write principles with a name, statement, rationale and implications.
- ArchiMate 4 models the enterprise in domains and aspects with defined relationships, and uses one set of behaviour elements across the business, application and technology domains. Viewpoints select what each audience needs to see.
- Empirical studies show that practitioners adapt frameworks heavily. Architecture is judged by the decisions it improves.

CHAPTER 3

Business capabilities, value streams and requirements

Learning objectives

After this chapter you should be able to:

- Distinguish a capability from a process, an application and an organisational unit.
- Build a two-level capability map and a value stream for a customer outcome.
- Construct a capability heat map that rates importance and effectiveness separately and records confidence.
- Translate a strategic objective into requirements that can be tested.

What a capability is

A **business capability** is something the organisation is able to do to achieve a purpose, such as managing customer identity, scheduling technicians or reconciling financial records [19]. A capability describes what the organisation can do. It says nothing about how the work is done, which system supports it or who reports to whom.

Four related ideas are easy to confuse. The table uses Helios's service work to separate them.

Concept	Question it answers	Helios example
Capability	What must we be able to do?	Entitlement checking
Process	How is the work done, step by step?	Look up contract, check asset, record decision
Application	Which software supports the work?	CRM North, ERP Core
Organisational unit	Who is accountable and where do people report?	Maintenance service desk

The distinction matters because these things change at different rates. Helios could move entitlement checking from the service desk to a shared team, replace CRM North, and redesign the steps of the process, and the capability "entitlement checking" would still exist. That stability is what makes a capability map useful: it gives the board a structure for comparing investments that survives reorganisations and system changes.

A capability map is still a model. If Helios launched a new line of business, for example equipment leasing, it would need capabilities it does not have today, and the map would change.

Building a capability map

A **capability map** arranges capabilities in a hierarchy. Level 1 groups them into broad domains; level 2 breaks each domain into capabilities that a single owner can be accountable for. Deeper levels are added only where a decision needs them.

Good maps follow a few rules:

- Name capabilities with nouns, such as "Order allocation", rather than verbs or project names.
- Keep organisation names out. "Retail order desk" is a unit; the capability it performs is order allocation.
- Avoid overlaps. Each piece of work should belong to exactly one capability at a given level.
- Give each level-2 capability one accountable owner, even when several units perform it.
- Choose a level of detail that suits the decision. "Run the business" is too broad to guide investment; "Call the case API" is too technical.

Helios's first map has five level-1 domains and fourteen level-2 capabilities. Figure 5 shows it with the heat-map ratings that are explained below.

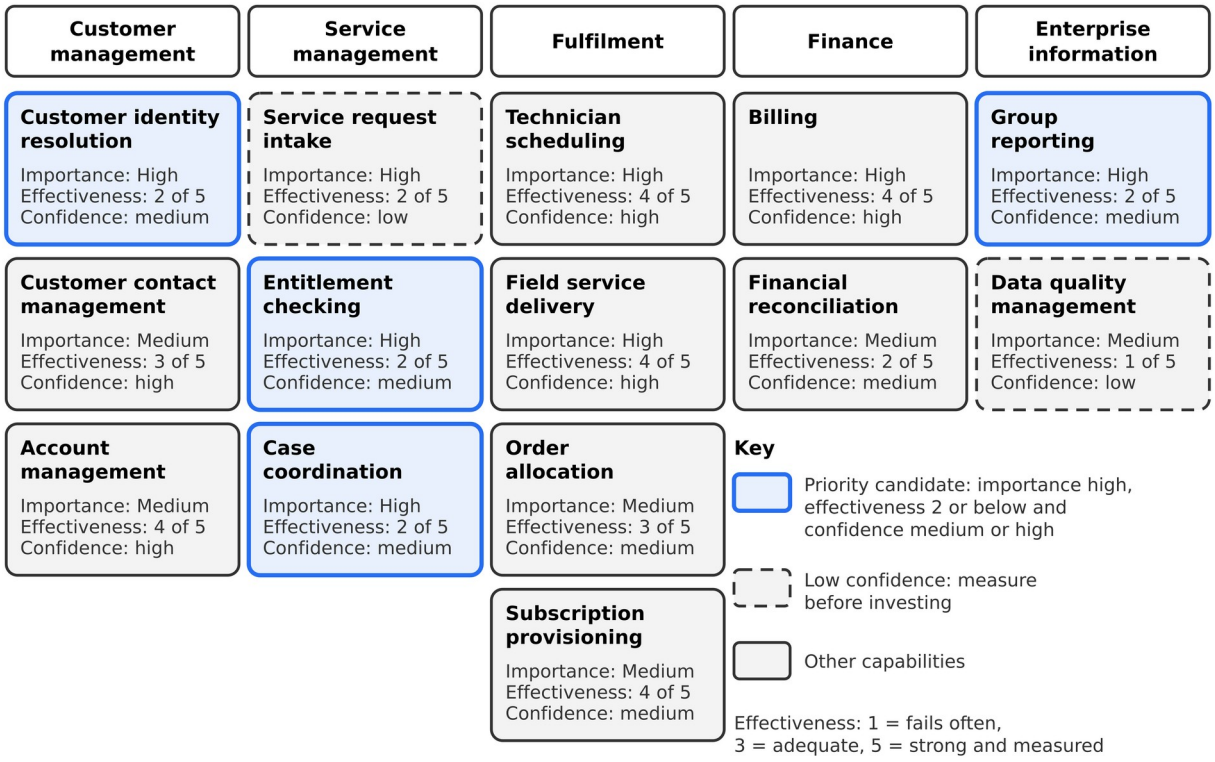


Figure 5 Helios capability map with heat-map ratings. Importance and effectiveness are rated separately, and confidence is shown by the border.

The map is common to all three business units. It shows what Helios does, and many capabilities are performed in different ways by different units. Industrial Maintenance needs technician scheduling, Retail Distribution needs order allocation and Digital Services needs subscription provisioning. All three need customer identity resolution, case coordination and financial reconciliation. The overlap tells the architect where shared information and controls could help, and the differences show where local processes may be justified.

Value streams

A capability map is static. A **value stream** shows how value reaches a stakeholder as a sequence of stages, from the trigger to the result the stakeholder wants [20]. Each stage draws on several capabilities, and one capability can support many value streams.

Helios's maintenance customers experience the value stream "Restore customer equipment". Its five stages, with times measured over one month of urgent requests, are shown below. Active time is when someone is working on the request; waiting time is when it sits in a queue.

Stage	Main capabilities used	Active (days)	Waiting (days)
Request service	Service request intake, customer identity resolution	0.1	0.4
Confirm entitlement	Entitlement checking, account management	0.1	2.4
Schedule technician	Technician scheduling	0.2	1.3
Complete work	Field service delivery	0.3	0.0
Confirm and bill	Case coordination, billing	0.1	1.6
Total		**0.8**	**5.7**

The request takes 6.5 days from start to finish, but people work on it for only 0.8 days. The **flow efficiency**, active time divided by elapsed time, is 0.8 / 6.5 = 12.3%. The single largest delay is waiting for entitlement confirmation: 2.4 days, or 36.9% of the elapsed time.

A departmental view would miss this. The service desk processes its own queue quickly, and so does the contracts team; the delay sits in the handoff between them. If Helios cut the entitlement wait from 2.4 to 0.6 days, elapsed time would fall by 1.8 days to 4.7 days, a 27.7% reduction, without anyone working faster. The value stream therefore points to entitlement checking and case coordination as capabilities to investigate, which is what the heat map below also suggests.

Shared stages need a coordination mechanism. Where two units both touch a stage, name one person accountable for the whole value stream's outcome as well as an owner for each unit's part.

Heat maps with evidence and confidence

A **capability heat map** adds ratings to the capability map. The most common mistake is to combine several dimensions into one colour. If a capability is shaded red, readers cannot tell whether it is important, failing, expensive or simply poorly understood.

Rate each dimension separately, with a defined scale:

- **Importance** to the current strategy: high if a board objective depends directly on it, medium if it supports one, low if it is a commodity.
- **Effectiveness** today, from 1 to 5: 1 means it fails often and staff rely on workarounds; 3 means adequate with known gaps; 5 means strong and measured.
- **Confidence** in the effectiveness rating: high when it rests on measured data, medium when it rests on consistent reports from several sources, low when it rests on opinion or a single source.

Record the basis for each rating. "Effectiveness 2: 14% of maintenance requests needed manual customer matching last quarter, from the service desk log" is useful. "Integration maturity 2" is not, because nobody can check it.

Worked example: rating five Helios capabilities

Capability	Importance, effectiveness and confidence	Basis
Customer identity resolution	High; 2 of 5; medium confidence	Duplicate customers found in samples from three units
Entitlement checking	High; 2 of 5; medium confidence	2.4-day entitlement wait measured in the value stream over one month; accuracy of decisions not yet measured
Case coordination	High; 2 of 5; medium confidence	1.6-day wait between completed work and billing in the value stream; failed handoffs in the service-credit records
Service request intake	High; 2 of 5; low confidence	Service desk reports duplicate and incomplete requests; no measurements
Technician scheduling	High; 4 of 5; high confidence	Scheduling system reports 96% of urgent visits booked within target

Apply a simple rule. A capability is a **priority candidate** when its importance is high, its effectiveness is 2 or below, and confidence is at least medium. When confidence is low, the first action is to measure before investing.

Under this rule, customer identity resolution, entitlement checking and case coordination are priority candidates, which matches what the value stream showed. Entitlement checking has medium rather than high confidence because one month of waiting times shows the delay but not its cause, or whether the decisions themselves are correct. Service request intake is high in importance and probably weak, but the rating rests on reports alone, so the first step is a two-week sample of incoming requests. Technician scheduling is important and works well, so it needs protection during change rather than investment. The full map in Figure 5 shows one more priority candidate: group reporting, rated high in importance because consolidated reporting is a legal and board requirement (Chapter 4), with effectiveness 2 because units define the same measures differently.

The rule is a starting point for discussion. A low-importance capability with a legal obligation attached, such as retaining billing records, may still need urgent work. The value of the heat map is that it makes each judgement visible and open to challenge.

> **Common mistake.** Sending every weak capability into the largest transformation programme. A weak capability may need a quick control, a measurement exercise or simply better training. Match the action to the cause.

From strategy to requirements

The board's objective "a coherent customer view" needs a user and a purpose before anyone can design for it. For example:

- A service agent needs to recognise the same customer across divisions, but should see only the cases they are authorised to see.
- Finance needs consolidated revenue per customer group, based on an agreed mapping of legal entities.
- Marketing may want cross-selling lists, which raises different questions about permitted use of data.

Each purpose leads to different requirements. A good enterprise requirement for the first purpose reads:

> **Requirement CV-1.** An authorised service agent can find a customer organisation by any division's identifier, see its governed group identifier and see the cross-division service history they are permitted to view, within five seconds. Links that have not been verified are shown as unverified.

This requirement is testable. It also implies several pieces of architecture: identifier mapping, an access policy, a named authority for each fact, a freshness target and a process for resolving ambiguous matches. It does not require one CRM.

Requirements need owners and acceptance tests. Some questions are policy decisions rather than design questions. Should a Retail agent see a Maintenance case for the same customer? The architect should raise that question with the accountable owners before anyone builds a cross-division data join. Being able to combine records technically does not grant permission to use them for every purpose.

Exercises

3.1 Using Helios's service work, give one example each of a capability, a process, an application and an organisational unit. Explain why the capability is the most stable of the four.

3.2 Recalculate the flow efficiency of the "Restore customer equipment" value stream. Then calculate the elapsed time and flow efficiency if the billing wait also fell from 1.6 to 0.4 days, in addition to the entitlement improvement.

3.3 The procurement team asks to add "Supplier invoice matching" to the Finance domain. The evidence is: 12% of supplier invoices needed manual matching last quarter, from the accounts-payable log; two audit comments on late payments; and no board objective that depends directly on it. Design a heat-map entry with importance, effectiveness and confidence ratings and the basis for each, and state the action the priority rule implies.

3.4 Write a testable requirement for Finance's purpose in the "coherent customer view" objective. Include scope, authority, freshness and treatment of uncertain mappings.

3.5 A manager proposes colouring the capability map red, amber and green, with red meaning "needs attention". Critique the proposal and suggest a better design.

Key points

- A capability is what the organisation can do. It is more stable than the processes, applications and units that deliver it, which makes it a good structure for investment decisions.
- Value streams show how stages and capabilities combine to deliver an outcome. Measuring waiting time as well as active time reveals the handoffs that departmental views miss.
- Heat maps should rate importance and effectiveness separately, define their scales, record the basis for each rating and show confidence.
- Strategic objectives become useful once they are rewritten as testable requirements with a named user, purpose, scope and owner.

CHAPTER 4

Operating models and strategic alignment

Learning objectives

After this chapter you should be able to:

- Explain the four operating models defined by process integration and process standardisation.
- Analyse which capabilities need integration, standardisation or both, and justify each choice.
- Compare central, federated and local operating designs for a specific decision.
- Evaluate the alignment between business strategy and IT using the strategic alignment model.
- Distinguish an operational backbone from a digital platform, and classify an organisation’s components accordingly.

Sharing data and sharing processes are different choices

Two of Helios’s divisions could share customer identity while running quite different fulfilment processes. Equally, they could follow the same process template while keeping their customer data apart. These are two separate choices:

- **Process integration** is the extent to which units share data, so that work in one unit depends on accurate, timely data from another.
- **Process standardisation** is the extent to which units perform the same work in the same way, with common definitions and procedures.

Integration buys coordination: a customer who calls Retail about a maintenance contract can be recognised and helped. Standardisation buys efficiency and predictability: one process is cheaper to train, support and improve than three. Each has costs, and each can be chosen capability by capability.

Four operating models

Ross, Weill and Robertson define an **operating model** as the level of business process integration and standardisation an organisation needs to deliver its goods and services to customers [2]. Crossing the two dimensions gives four models, shown in Figure 6:

- **Diversification** (low integration, low standardisation). Independent units with few shared customers or data. Each unit chooses its own processes and systems.
- **Coordination** (high integration, low standardisation). Units share customer, product or supplier data, but each keeps its own processes.
- **Replication** (low integration, high standardisation). Similar units run the same standard processes, but have few reasons to share data with each other.
- **Unification** (high integration, high standardisation). One business with standard processes and shared data across the organisation.

The same authors define enterprise architecture as the organising logic for business processes and IT infrastructure that reflects the integration and standardisation requirements of the operating model [2]. In their account, an organisation builds a **foundation for execution**, a set of digitised core processes and shared data, in stages. Organisations move from business silos, through standardised technology and an optimised core, towards business modularity. A related research briefing argues that IT should be focused on the operating model rather than on a strategy that may change every year [21].

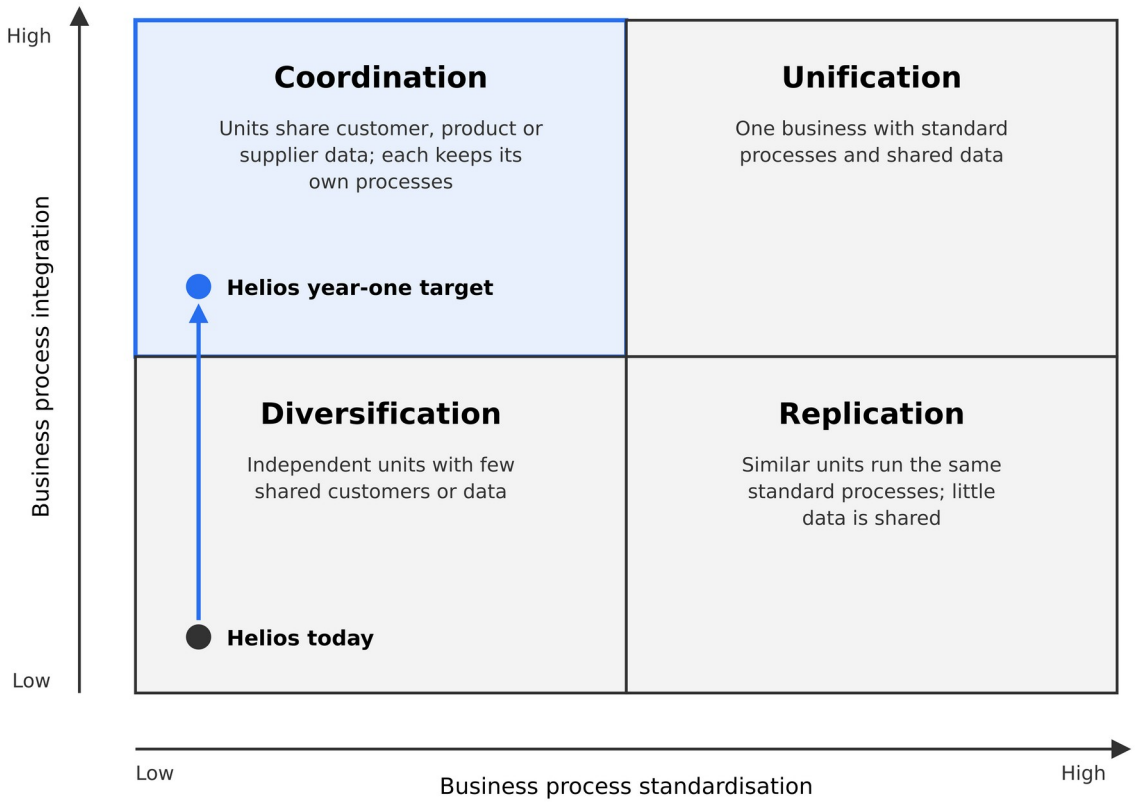


Figure 6 The four operating models defined by process integration and process standardisation, with Helios's intended move from Diversification towards Coordination.

Applying the grid to Helios

Helios today looks like a diversified group. Acquisitions brought their own systems, customer identifiers and processes, and little data crosses unit boundaries. The board's objectives point upwards on the grid: reliable cross-unit handoffs and a coherent customer view both need integration. The case for standardisation is weaker. The units' fulfilment processes differ, and nobody has yet shown whether those differences are accidental or legitimate.

That reasoning suggests Coordination as the year-one target. It also shows a limit of an enterprise-wide label: different capabilities need different positions on the grid.

The grid describes how units relate to each other, so it applies only to capabilities that more than one unit performs or depends on. A capability performed by one unit alone, such as technician scheduling in Maintenance, has no position on it; its design is a local decision. For the cross-unit capabilities, the question is how much the units need to share data and how much they need to work the same way:

Cross-unit capability	Integration and standardisation needed	Reason
Customer identity resolution	High integration, low standardisation	Units must recognise shared customers; local processes can stay
Group financial reporting	High integration, high standardisation	One set of definitions is a legal and board requirement
Workforce identity and access	Low integration, high standardisation	The same minimum controls apply in every unit; little business data is shared
Service fulfilment (visits, deliveries, subscriptions)	Low integration, low standardisation	Each unit fulfils a different product; the handoffs between them rely on shared customer identity rather than shared fulfilment

Read the table as a set of hypotheses for owners to confirm. Its value is that each row can be argued on its own evidence, instead of one debate about "centralisation".

Strategic alignment

Henderson and Venkatraman's **strategic alignment model**, first published in 1993, explains why alignment is more than IT taking orders from the business [22]. It has four domains, and each is described by three components:

Domain	Components	Helios example
Business strategy	Business scope, distinctive competencies, business governance	Which customers and services the group offers; reliable cross-unit service as a competence
IT strategy	Technology scope, systemic competencies, IT governance	Shared customer identifiers and integration as competencies; who decides on IT
Organisational infrastructure and processes	Administrative structure, processes, skills	Three business units, their service processes and their staff
IT infrastructure and processes	IT architecture, processes, skills	Forty-three applications, platform operations, architects and engineers

The model links the domains in two directions. **Strategic fit** links the external choices, where to compete and with what distinctive capabilities, to the internal arrangements that carry them out. **Functional integration** links the business side to the IT side. Alignment requires both: a sound IT strategy that the IT infrastructure cannot deliver fails on fit, and an excellent IT department pursuing goals the business does not share fails on integration.

The authors describe four **alignment perspectives**. Each starts from one strategy domain, works through a second domain and changes a third:

Perspective	Starts from	Works through	Changes
Strategy execution	Business strategy	Organisational infrastructure	IT infrastructure
Technology transformation	Business strategy	IT strategy	IT infrastructure
Competitive potential	IT strategy	Business strategy	Organisational infrastructure
Service level	IT strategy	IT infrastructure	Organisational infrastructure

In strategy execution, the business designs its organisation and IT follows. In technology transformation, the business strategy calls for a new IT strategy, which reshapes the IT infrastructure. In competitive potential, new IT capabilities open new business strategies. In service level, IT strategy shapes the IT infrastructure so that IT serves the organisation well.

For example, Helios's board is currently following a strategy-execution path: improve cross-unit service, then design the processes and systems to support it. The competitive-potential path matters too. Once customer identity is resolved across units, the Digital Services unit could offer a single maintenance-plus-monitoring contract to industrial customers, a product the group cannot sell today. An architect who presents the customer identity work only as a cost-saving measure hides that option from the board.

Alignment is therefore a recurring test. At each major investment, ask whether the capability, process, information model and responsibilities still support the current strategy. If Helios later decides to sell a division, separability becomes more valuable than integration, and the architecture should show the cost of each direction rather than pretending one target suits both.

Operational backbone and digital platform

Ross, Beath and Mocker extend the operating-model argument to digital offerings [23]. From field research in large companies, they describe five building blocks of digital business: shared customer insights, an operational backbone, a digital platform, an accountability framework and an external developer platform. Two of them matter most for enterprise architecture:

- An **operational backbone** is the integrated set of core systems, data and processes that runs the business reliably and efficiently, such as transaction processing, back-office services and authoritative core data.

- A **digital platform** is a set of reusable business, data and technology components from which teams can quickly assemble, test and change digital offerings.

The two need different things. The backbone values stability, integration and standard processes, and changes deliberately. The platform values speed, reuse and experimentation, and changes continually. The backbone is necessary for a digital business, because offerings built on unreliable core data inherit its errors, but it does not by itself let a company innovate quickly.

At Helios, ERP Core, the party register, the CRMs and the billing process form the backbone, and the Coordination choice above is mainly a decision about it. The platform is still small: a customer lookup service offered by the link hub, events announcing verified customer links, the workforce sign-in service and shared monitoring. Digital Services could build the maintenance-plus-monitoring contract from those components without changing the backbone's processes. Two design rules follow. Platform components read and change backbone data only through published contracts, and the platform's faster release cycle must never become a route for unreviewed changes to the backbone.

Who decides, funds and operates

An operating model has to be expressed as responsibilities. For Helios's year-one design:

- **Group** sets customer-identity standards, minimum security controls and consolidated reporting definitions.
- **Business units** own their service processes, local product features and approved exceptions.
- **Shared platform teams** provide workforce identity, customer matching, integration and monitoring services under published contracts.

Two questions often go unanswered. The first is funding. If every division expects another to pay for the customer-mapping service, a sound shared capability may never be maintained. The second is priority. If the central team sets all priorities without understanding local needs, divisions build unofficial alternatives.

Match the decision level to the consequence. A change to a local screen can stay with a product team. A change to the meaning of the enterprise customer identifier affects reporting and integrations across units, so it needs broader agreement. Centralising both decisions slows the small one; decentralising both lets incompatible facts spread.

The cost of standardisation

A common process reduces variation, simplifies training and makes shared data comparable. It can also remove useful local differences or push exceptions into manual work. Examine the variation before removing it. Some is accidental legacy. Some reflects a distinct product, customer promise or contract.

For example, Maintenance closes a service case when the technician finishes the work, while Digital Services closes a case only after the customer confirms the fix. Mapping both events to a single "closed" status makes the reports look standard while hiding two different meanings. There are two honest options: agree a common business definition of closure and change one process, or keep two distinct events and map them openly for each measure that needs them.

Standardisation also has transition costs: training, data conversion, a period of running two processes, and changes in accountability. These can exceed the licence savings in the first years. Count them in the business case and measure whether the common process is actually used. An approved process diagram is not proof that daily work follows it.

Worked comparison: three operating designs

Consider three designs for Helios's service operations:

- **Central consolidation.** All divisions move to one CRM and one closure process.
- **Federated coordination.** Divisions keep their CRMs and fulfilment processes, and standardise customer identity, selected event contracts and group measures.
- **Local autonomy.** Operational decisions stay local; group reporting is reconciled at month end.

Dimension	Central consolidation	Federated coordination	Local autonomy
Process ownership	One common process owner	Local owners within shared contracts	Local owners
Customer meaning	One governed operational definition	Shared matching, explicit local meanings	Local definitions, reconciled for reports
Main dependency	Central product and process capacity	Contract and platform coordination	Local delivery capacity
Main risk	The common process fits no unit well	Shared contracts lack funded owners	Group promises exceed what reconciliation can deliver

Central consolidation makes one process easier to enforce and can remove duplicate configuration. It concentrates change decisions, and it requires a genuinely common process, which means an authorised business change as well as a field mapping.

Federated coordination corresponds to the Coordination operating model. It allows different processes but needs durable ownership of shared contracts. A shared customer-matching service without funded stewardship becomes an unreliable dependency for everyone. Each division must agree how disputed matches are resolved and how quickly it publishes the events others depend on.

Local autonomy suits a strategy in which divisions have separate customers and little cross-selling. It costs delayed group visibility and manual reconciliation, which may be acceptable for monthly reporting but not for a promised cross-division service journey.

Helios chooses federated coordination for year one. The group has a clear need for shared visibility, but no one has shown that the divisions' fulfilment processes are equivalent. The decision states what would change it: if analysis shows that process differences are accidental and one product supports all obligations, central consolidation becomes the better option; if a division is to be sold, a narrower integration boundary is better.

Keep funding, process ownership, data authority and deployment as separate questions. A centrally funded customer identity service can support locally owned applications. A shared cloud account says nothing about common process meanings. Show each relationship explicitly so that reviewers can see exactly where people agree and disagree.

Exercises

4.1 Name one Helios capability that needs high integration with limited standardisation, and justify the choice.

4.2 Explain how a single "closed" status code can hide different business meanings across divisions, and describe two ways to resolve the problem.

4.3 Place each of the following cross-unit capabilities in the operating-model grid and justify your choice: Helios's payroll; customer billing across the three units; group cash management. Then explain why Retail's warehouse picking process does not belong in the grid at all.

4.4 The board plans to sell Retail Distribution in two years. Compare the three operating designs for this scenario. Identify which shared dependencies can remain contractual after the sale and which need physical separation.

4.5 A manager says: "Alignment means IT does what the business strategy says." Evaluate this claim using the strategic alignment model, and give one Helios example of a different alignment perspective.

4.6 Classify each of the following as part of Helios's operational backbone or its digital platform, and justify your choice: ERP Core; the customer lookup service offered by the link hub; the nightly billing run; the stream of verified-link events; the party register. Then explain one risk of letting Digital Services build its new monitoring contract directly on CRM North's database.

Key points

- Process integration and process standardisation are separate choices. Together they define four operating models: Diversification, Coordination, Replication and Unification.
- Enterprise architecture expresses the operating model as shared data, processes and infrastructure. Different capabilities can sit in different parts of the grid.

- Strategic alignment needs both strategic fit and functional integration, and technology can open new strategies as well as support existing ones.
- An operational backbone runs the core business reliably; a digital platform lets teams assemble new offerings quickly. They need different change processes and meet only through published contracts.
- Operating choices must be turned into explicit responsibilities for deciding, funding and operating shared services.
- Helios’s year-one choice is federated coordination, with stated conditions under which it would change.

CHAPTER 5

Application portfolios and rationalisation

Learning objectives

After this chapter you should be able to:

- Design an application inventory record that supports investment and retirement decisions.
- Classify applications with the TIME model and explain what the classification leaves out.
- Compare cloud migration strategies for a given application and deadline.
- Evaluate a retirement plan for completeness.

Inventory for a decision

An **application portfolio** is the set of applications an organisation runs, managed as a group of investments rather than as separate systems. Managing it needs an inventory, and the inventory should hold the facts that decisions depend on. For each application, record:

- business purpose and the capabilities it supports;
- accountable owner, users and non-interactive consumers such as batch jobs;
- the facts it is authoritative for, its interfaces and its dependencies;
- criticality and recovery needs;
- annual operating cost, split into licences, infrastructure, internal support and allocated shared services;
- lifecycle status, contract terms and planned changes;
- the date of each fact and how confident the team is in it.

Old inventories mislead. After acquisitions, supplier changes and departmental development, an inventory that is a year old may miss applications and list others that no longer run.

Helios's forty-three applications include six that represent the main issues. Their annual operating costs are already part of the €12 million group IT baseline; they are not additional spending.

Application	Main responsibility	Annual operating cost (EUR)	Initial issue
CRM North	Industrial service cases	420,000	Weak handoff reconciliation
CRM Central	Distribution service cases	360,000	Similar customer data, different identifiers
ERP Core	Financial and asset records	900,000	Shared critical dependency
Legacy Orders	Order intake and state	600,000	Hosting contract ends in 14 months
Analytics Warehouse	Group reporting	300,000	Inconsistent metric definitions
Identity Platform	Workforce identity and access	240,000	Incomplete account lifecycle integration

The sample totals €2.82 million. It is a sample, so the average cost per application cannot be extrapolated to the other thirty-seven without data about them. Splitting cost by type matters for savings: retiring an application removes its licence, but shared infrastructure costs may remain unless the platform is resized.

The TIME model

The **TIME model** is a widely used practitioner heuristic for application portfolios, generally attributed to the analyst firm Gartner. It scores each application on two dimensions:

- **Business value:** how much the application contributes, for example the importance of the capabilities it supports, its usage and whether its functions are unique.
- **Technical fitness:** how sound the application is, for example its supportability, reliability, integration quality and cost to run.

Splitting each dimension at the midpoint of a 1-to-5 scale gives four suggested dispositions, from which the model takes its name. This book counts a score of 3.0 or more as high and below 3.0 as low:

- **Tolerate:** sound but low value. Keep running at minimum cost and avoid new investment.

- **Invest:** valuable and sound. Extend and improve.
- **Migrate:** valuable but technically weak. Move its functions to a better platform or product.
- **Eliminate:** low value and technically weak. Retire when it is safe to do so.

Worked example: scoring Helios's sample

The architecture team and each application owner score four criteria per dimension from 1 to 5 and take the mean. For CRM North, the business-value criteria are capability importance 4, number of users 3, uniqueness of functions 4 and contribution to revenue 3, giving (4 + 3 + 4 + 3) / 4 = 3.5. The technical-fitness criteria are supportability 2, reliability 2, integration quality 3 and running-cost efficiency 1, giving (2 + 2 + 3 + 1) / 4 = 2.0. CRM North therefore falls in the Migrate quadrant. Figure 7 plots all six applications.

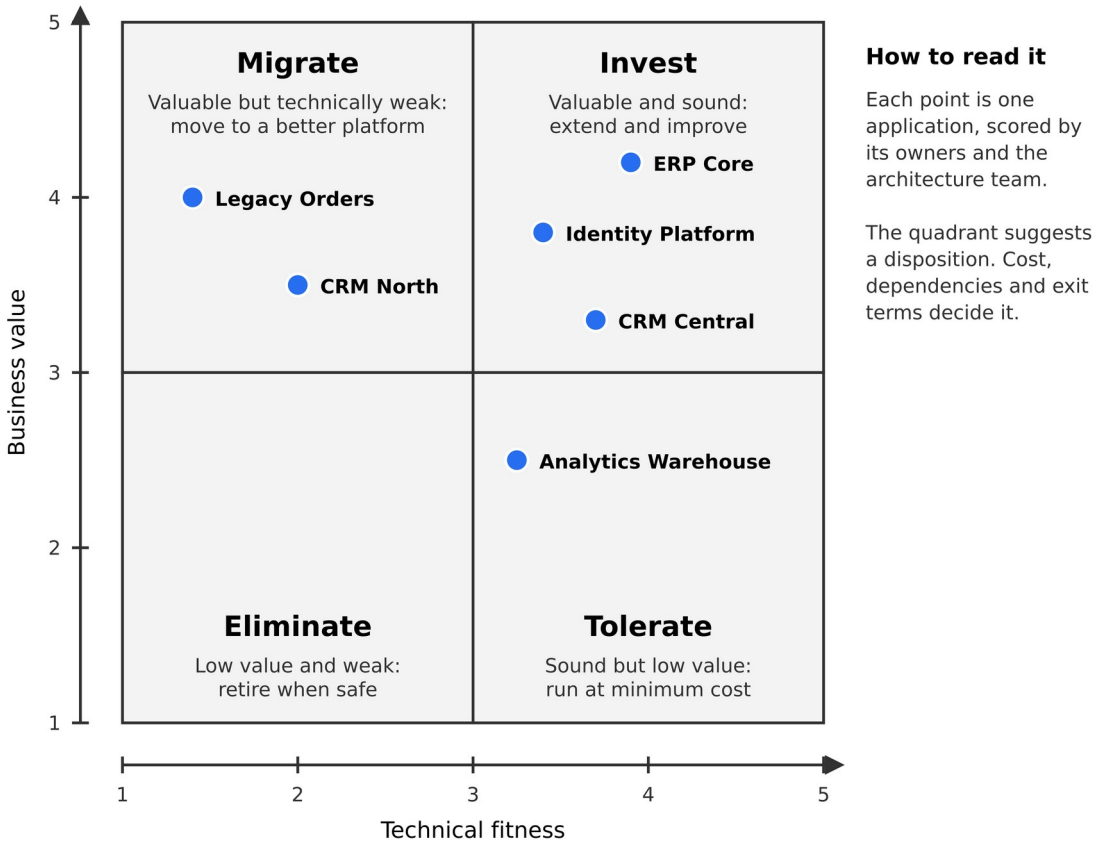


Figure 7 TIME classification of Helios's six sample applications by technical fitness and business value.

Application	Fitness and value scores	Quadrant	What could override the suggestion
ERP Core	3.9 and 4.2	Invest	Its role as a shared dependency limits the pace of change
Identity Platform	3.4 and 3.8	Invest	Lifecycle gaps are an urgent risk rather than an enhancement
CRM Central	3.7 and 3.3	Invest	Close to the value boundary; depends on the CRM decision
CRM North	2.0 and 3.5	Migrate	Process differences may make migration harder than it looks
Legacy Orders	1.4 and 4.0	Migrate	The hosting deadline fixes the timing
Analytics Warehouse	3.25 and 2.5	Tolerate	Its low value comes from poor definitions, which data governance can fix

TIME is useful because it is quick, visual and easy to explain to a board. Its limits are just as important. The scores are judgements, and an application near a boundary, such as CRM Central, can change quadrant with one criterion. The model also ignores dependencies, contracts, migration cost and timing. A quadrant suggests a disposition; the business case, the dependency analysis and the exit terms decide it.

Dispositions and migration strategies

Portfolio decisions use a wider vocabulary than TIME's four words. An application can be retained, improved, consolidated with another, replaced, moved to new hosting or retired. When the decision involves moving to the cloud, AWS describes seven common migration strategies, often called the **seven Rs** [24]:

- **Retire:** switch the application off.
- **Retain:** keep it where it is for now and revisit later.
- **Rehost:** move it to the cloud without changing it ("lift and shift").
- **Relocate:** move it to the cloud at the virtualisation level, without changing the application or its operating system.
- **Repurchase:** replace it with a different product, often software as a service.
- **Replatform:** move it with a few targeted changes, such as using a managed database.
- **Refactor:** redesign it to use cloud-native features.

Worked example: Legacy Orders

Legacy Orders falls in Migrate, and its hosting contract ends in 14 months. The team estimates four options.

Strategy	One-off cost and duration	Margin before the deadline	Main consequence
Rehost	€0.3m over 4 months	10 months	Keeps its technical weaknesses
Replatform	€0.6m over 8 months	6 months	Managed database cuts support effort
Repurchase	€0.9m over 12 months	2 months	New subscription cost; process change for staff
Refactor	€1.8m over 18 months	–4 months	Misses the deadline without a hosting extension

Refactoring would produce the best long-term design, but it needs four months more than the deadline allows, so it is only feasible with a hosting extension. Repurchase leaves only two months of margin for a project that also changes staff processes, which is little room for the usual slippage. Replatform meets the deadline with six months to spare and reduces running effort; rehost is cheaper but leaves the technical problems in place. A defensible recommendation is to replatform now and to reconsider repurchase when the subscription market and Helios's order processes are better understood. The €0.6 million comes from the legacy modernisation work package, so it must be shown there, not as new money.

Application counts are poor success measures

Reducing forty-three applications to thirty may still increase cost if the remaining products are more expensive and integration work grows. Conversely, a new shared service can increase the count while removing duplicated logic and risk. Measure the outcome the rationalisation is meant to create: lower run cost, fewer reconciliation hours, fewer failed handoffs or reduced exposure to unsupported software.

Worked comparison: two CRMs

CRM North and CRM Central both support service cases, but their fulfilment processes differ. There are three main options:

1. Keep both CRMs, with common customer identifiers and reliable integration between them.
2. Consolidate onto one product while preserving the process variations that are genuinely needed.
3. Replace both as part of a broader process redesign.

The first option delivers shared-data benefits sooner but keeps two systems to run. The second may reduce platform cost but needs migration, a good process fit and a tested way of separating each division's access. The third gives the most design freedom with the largest scope and adoption risk. Licence price alone cannot rank them.

Before selecting, test representative workflows and exports. Include attachments, case history, identifiers and audit records. A demonstration of a new case form shows only that the form works. It does not show that the product can carry over the existing business history or support the exception process.

Retirement is a project

A system marked “retired” in the inventory while it still runs, and is still billed, has delivered no saving. A retirement plan needs:

- migrated users and replacement functionality, confirmed by the business owner;
- a decision on records: migrate, archive or keep read-only access, under the retention policy;
- removal of integrations and scheduled jobs, including monthly and annual ones;
- revoked access and decommissioned infrastructure;
- cancelled or reduced contracts, with the notice period in the timeline;
- a watch period for missed dependencies, with a plan for responding if one appears.

Archives need care. A spreadsheet export of current rows may lose relationships, attachments and the meaning of historical codes. Check that the archive answers the questions people will actually ask of it.

Prioritising the first actions

For Helios, the Legacy Orders deadline is a fixed time constraint, the gaps in the workforce identity lifecycle are a risk across every application, and inconsistent reporting makes it hard to measure the programme at all. These may justify early work even though CRM consolidation promises larger savings later. Sequence investments by dependency and risk as well as by projected benefit.

Record each major disposition in a portfolio decision record: the reason, options considered, supporting data, transition cost, owner, timing and the condition that would trigger a review. This links the enterprise model to funded action.

Exercises

5.1 Verify the sample’s total operating cost and explain why it must not be added to the €12 million group baseline.

5.2 An application has few users and a high cost. Give three reasons why it might still be unsafe or uneconomic to retire it immediately.

5.3 Compare keeping two CRMs with shared data contracts against consolidating them. Identify the information most likely to change the decision.

5.4 A seventh application, Retail’s warehouse picking system, scores 3, 2, 2 and 3 on the four business-value criteria and 4, 2, 3 and 2 on the four technical-fitness criteria. Calculate its TIME position and quadrant. Then explain why the quadrant alone should not decide its disposition, naming two facts you would check first.

5.5 The Legacy Orders hosting provider offers a six-month extension for €150,000. Re-evaluate the refactor and repurchase options. Which would you now recommend, and what would you need to believe for refactoring to be the better choice?

Key points

- An application inventory is useful when it records the facts that decisions depend on, with dates and confidence.
- The TIME model classifies applications by business value and technical fitness. It suggests a disposition; cost, dependencies and timing decide it.
- The seven Rs give a vocabulary for cloud migration choices. Compare them against the real deadline, cost and long-term consequence.
- Application counts and licence prices are inputs. Measure the outcomes that rationalisation is meant to improve.
- Retirement delivers savings only when users, data, integrations, access and contracts have all been dealt with.

CHAPTER 6

Enterprise information and data governance

Learning objectives

After this chapter you should be able to:

- Assign authority for specific business facts rather than for whole entities.
- Distinguish the roles of data owner, data steward and technical custodian.
- Design a data contract and a correction workflow for shared data.
- Compare central and federated data governance for a given decision.
- Compare the four master data management styles and place a design among them.

One entity, several authorities

“Customer” can mean a legal organisation, a service site, a billing party, a contact person or a commercial relationship. Different systems can legitimately be authoritative for different facts about the same customer. An **authoritative source** is the system whose value wins when copies disagree, and a **data owner** is the person accountable for that fact’s meaning and quality.

Helios’s first fact-authority table looks like this:

Fact	Authoritative source	Owner	Who may change it
Legal organisation and registration number	Party register	Group data owner	Party stewards
Billing account and invoices	ERP Core	Finance director	Finance operations
Staff credentials and access lifecycle	Identity Platform	Security officer	Identity service only
Service-case progress	The CRM handling the case	Division service manager	Case handlers in that division
Link between records in different systems	Customer link hub	Group data owner	Party stewards

The table settles arguments before they start. When CRM North and ERP Core disagree about a customer’s legal name, the party register wins and CRM North’s copy is corrected. When they disagree about an invoice, ERP Core wins.

An enterprise data model should make these distinctions explicit. It does not need one enormous customer object stored everywhere. Define the shared identifiers and meanings needed for coordination, and leave local models alone where their meanings genuinely differ. Figure 8 shows the pattern.

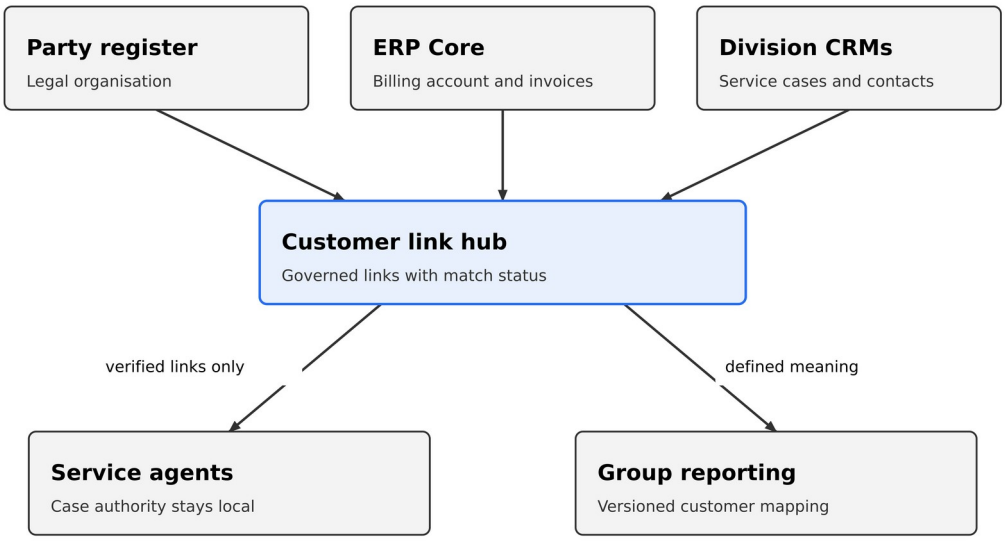


Figure 8 Authority is assigned to specific facts, while governed links connect the local models.

Governed identifiers and match status

Helios creates a **group customer identifier** for each customer organisation. It carries the identifiers used in each source system, the organisation's legal relationships, where each link came from and a **match status**:

- **Proposed:** an automated rule suggests two records describe the same organisation.
- **Verified:** a steward has confirmed the link using an agreed check.
- **Rejected:** a steward has confirmed that the records describe different organisations.

For example, CRM North holds "Nordvik Pumps AS" (record N-1042) and CRM Central holds "Nordvik Pumps" (record C-0077). The matching rule scores the pair at 0.86 on name and address similarity, so the link is proposed. A steward checks the company registration number in both records, finds that they agree and marks the link verified. A second pair, "Hallberg Engineering Ltd" and "Hallberg Engineering Services Ltd", scores 0.91 but has two different registration numbers. The steward rejects the link, because these are two legal entities that happen to share a name.

The example shows why a high score is not the same as a verified fact. A merge can expose one customer's information to another, so the resolution process needs both data-quality checks and access-policy controls. An ambiguous match stays proposed until the agreed process resolves it.

Master data management styles

Master data is the core data about the business's main entities, such as customers, sites, products and suppliers, that many processes and systems share. DAMA's body of knowledge for data management treats reference and master data as one of its knowledge areas [25]. Practitioners commonly distinguish four implementation styles for master data, a vocabulary usually attributed to the analyst firm Gartner. The styles differ in where the authoritative values live and where changes are made:

Style	Where the master values live	Where changes are made	Typical use
Registry	In the source systems; the hub holds identifiers, links and match status	In the source systems	Linking records across systems with little disruption
Consolidation	Copied into a hub that builds a single best record	In the source systems; the hub is read-only	Reporting and analysis
Coexistence	In the hub and the source systems, kept in step	In either, synchronised both ways	A gradual move towards shared master data
Centralised	In the hub, which is the system of record	Only in the hub; sources read from it	Strict, uniform control of core data

Helios's customer link hub is a **registry-style** hub. Each CRM stays authoritative for its own records, and the hub holds the group customer identifier, the source identifiers and the match status of each link. The party register adds a centralised element for one narrow set of facts, the legal organisation and its registration number. The registry style is the least disruptive and fits the Coordination operating model of Chapter 4, but it cannot by itself fix a wrong address in a CRM; that still needs a correction in the source. Moving towards coexistence or a centralised hub becomes worth considering only if the divisions later agree common meanings for more customer facts.

Owners, stewards and custodians

Three roles are easy to blur:

- The **data owner** is accountable for the meaning of the data, its approved uses and the decisions that affect its quality.
- The **data steward** maintains definitions, resolves day-to-day issues such as proposed matches, and coordinates corrections.
- The **technical custodian** operates the storage, access mechanisms and backups.

In a small organisation one person may hold several roles, but the responsibilities remain distinct. A database administrator can restore a table; only the owner can decide whether the restored values are correct.

Data contracts

A dataset offered to other teams needs a **data contract**: a published agreement about what the data means and what consumers can rely on. Calling a dataset a “data product” does not by itself supply these properties. One element deserves a definition first. The **grain** of a dataset states what a single row represents, for example one customer organisation, one pair of linked records or one service request per day. Two datasets with different grains cannot be joined or compared safely until one is converted to the other’s grain. Helios’s contract for verified customer links states:

Element	Verified customer links
Meaning	Two source records describe the same legal organisation
Grain	One row per pair of linked source records
Identifiers	Group customer identifier plus source system and source record identifier
Freshness	Available within 15 minutes of verification
Quality	Fewer than 5 false links per 1,000 in the monthly audit sample
Access	Service agents within their permitted scope; analysts in aggregate
Change policy	Additions with 30 days’ notice; changes of meaning need data council approval
Support	Party stewardship team, business hours, named escalation

The contract also says what the data cannot be used to establish. A proposed match with a score of 0.86 may be useful to an analyst estimating the number of shared customers, but it is unsuitable for granting a service agent access to another division’s cases. Reuse depends on fitness for the consumer’s purpose as well as on technical availability.

Data management as a discipline

The work in this chapter belongs to a wider discipline. The DAMA Data Management Body of Knowledge (DAMA-DMBOK) describes eleven knowledge areas, with **data governance** at the centre guiding the other ten [25]:

- data architecture, and data modelling and design;
- data storage and operations, and data security;
- data integration and interoperability;
- document and content management;
- reference and master data;
- data warehousing and business intelligence;
- metadata management and data quality.

An enterprise architect seldom runs these functions, but needs to know which of them a decision touches. Helios’s fact-authority table draws on governance, master data and metadata. The correction workflow later in this chapter draws on data quality, integration and security. Naming the knowledge areas helps the architect find the right specialists and avoid designing a governance process that the data team already runs.

Central and federated governance

A central team can define shared identifiers, interoperability rules and minimum controls. Domain teams can remain responsible for the meaning and quality of their own data. This **federated** arrangement combines common coordination with local expertise, but it needs a way to resolve disagreements between domains and a way to fund shared work.

Too much centralisation makes the central team a bottleneck for every local schema change. Too little lets incompatible definitions and duplicated sensitive data spread. Choose decision boundaries by who bears the consequence. For example, adding an optional field to a local maintenance form is a domain decision; changing the definition of group revenue requires broader governance.

A data catalogue connects these decisions to actual systems and owners. Its value depends on being current and used. A catalogue entry labelled “trusted” without a contract or an enforcement route tells a consumer very little.

Correction and lifecycle

Suppose a steward wrongly merges two customers, and the merged identifier has already reached CRM North, CRM Central and the analytics warehouse. A shared identifier that cannot be corrected safely spreads errors faster than separate systems ever did. A correction workflow needs these steps:

1. Pause automated propagation of the affected identifier.
2. Identify every downstream copy and report that used the merged record, using lineage records.
3. Decide the authoritative correction: a split into two identifiers, with the correct source records attached to each.
4. Publish the correction as an event that consumers process, preserving the history of the mistaken link.
5. Reconcile both CRMs and rebuild or annotate affected reports.
6. Review whether the merge disclosed information to people who should not have seen it, and involve the privacy officer if it did.
7. Record the cause and close the case only when every consumer confirms completion.

Lifecycle rules apply across the same copies. The EU General Data Protection Regulation requires organisations to process personal data for specified purposes, to limit it to what is necessary and to build data protection into the design of processing [26]. The exact obligations depend on the processing and on each organisation's role, so Helios's retention, access and transfer policies must come from its privacy and legal specialists. The architecture's job is to make those policies enforceable: a deletion or restriction must reach operational systems, analytical stores, exports and backups, and a restore from backup must not silently undo it.

Worked first-year data programme

Begin with the small set of customer-organisation facts needed for service coordination and group reporting. Assign authority, define matching and correction workflows, and pilot with two divisions. Measure:

- the number and age of proposed matches awaiting a decision;
- false merges found in audit samples;
- the time taken to complete a correction;
- whether consumers find the links useful for their purpose.

Do not ingest every available personal field simply because the platform can store it. Publish one governed service metric with a declared grain and correction policy before attempting a group-wide dashboard. The enterprise contribution is to settle who owns each definition, who may use it, which divisions must adopt it and how disagreements are resolved.

Exercises

6.1 Extend Helios's fact-authority table with rows for service site, customer contact person and contract entitlement. Name the authoritative source, owner and permitted writers for each, and explain one choice.

6.2 Explain why a probabilistic customer match that is acceptable for analysis may be unacceptable for authorisation.

6.3 In the monthly audit, stewards check a random sample of 400 verified links and find 3 false links. Calculate the false-link rate per 1,000 and compare it with the contract. If Helios holds 20,000 verified links, estimate how many could be false, and state one limitation of the estimate.

6.4 Design the correction workflow for a mistaken merge that has reached two CRMs and the analytics warehouse. Name the owner of each step and the evidence that the step is complete.

6.5 A sponsor proposes "one customer master record that every system must use for every customer fact". Critique the proposal and suggest a better design.

6.6 Helios's acquired business (change card B) proposes replacing the registry-style link hub with a centralised customer hub in which all customer facts are maintained. Compare the two styles for Helios, and state the conditions under which the change would be justified.

Key points

- Assign authority to specific facts rather than whole entities. Different systems can be authoritative for different facts about the same customer.
- Governed identifiers carry a match status. Proposed links are useful for some purposes and dangerous for others, such as granting access. Helios's link hub is a registry-style master data hub.

- Owners decide meaning and use, stewards maintain and correct, custodians operate.
- Data contracts state meaning, grain, freshness, quality, access and change rules, so that consumers know what they can rely on.
- Correction and lifecycle processes must reach every copy, including reports and backups.

CHAPTER 7

Technology strategy and shared platforms

Learning objectives

After this chapter you should be able to:

- Write a technology standard with a scope, rationale, owner and exception route.
- Evaluate a shared platform as a service, including the failures it can spread.
- Compare cloud strategies and the four meanings of portability.
- Explain Conway’s law and the mirroring hypothesis, and use them to diagnose a delivery bottleneck.

Four levels of technology guidance

Architecture guidance comes at four levels, and mixing them up causes needless conflict:

- A **principle** states a direction, such as “one authority for each shared fact”.
- A **standard** is an agreed requirement or preferred technology within a stated scope, such as “customer-facing services use the workforce identity service for staff sign-in”.
- A **reference design** is a reusable arrangement with stated conditions, such as a pattern for publishing customer events.
- A **project decision**, often recorded as an architecture decision record, applies these to one context.

If the levels are confused, a local experiment that uses a different library can look like a breach of an enterprise principle, when it is only a different implementation of the same design.

A standard should solve an identified problem

A technology standard can reduce support variety, improve interoperability or provide a control that has already been tested. It should state its scope, rationale, permitted use, owner and exception route. “Use the preferred cloud” is incomplete because it says neither which workloads it covers nor what would justify another choice.

For example, Helios writes its workforce identity standard like this:

> **Standard TS-04: workforce identity and access. Scope:** all customer-facing and internal applications in production. **Requirement:** staff sign in through the workforce identity service, and accounts are created and removed by its lifecycle events. **Rationale:** acquired applications with local accounts left former staff with access. **Owner:** identity service owner. **Evidence for release:** an automated test shows that a removed account is refused. **Exceptions:** time-limited, approved by the standard owner, with compensating controls.

Standardise the properties that need coordination. For Helios, a common sign-in contract, a logging structure and a deployment evidence policy create more value than forcing one programming language on fundamentally different products.

Treat a shared platform as a service

A **platform team** provides capabilities that other teams use: identity integration, deployment pipelines, monitoring, secrets management, network access or data processing. Its users need documentation, support, predictable change and an escape route for legitimate needs. A mandatory platform with long provisioning delays pushes teams towards unsupported alternatives. Figure 9 shows the contracts between product teams and platform services.

Measure the platform by what its users experience:

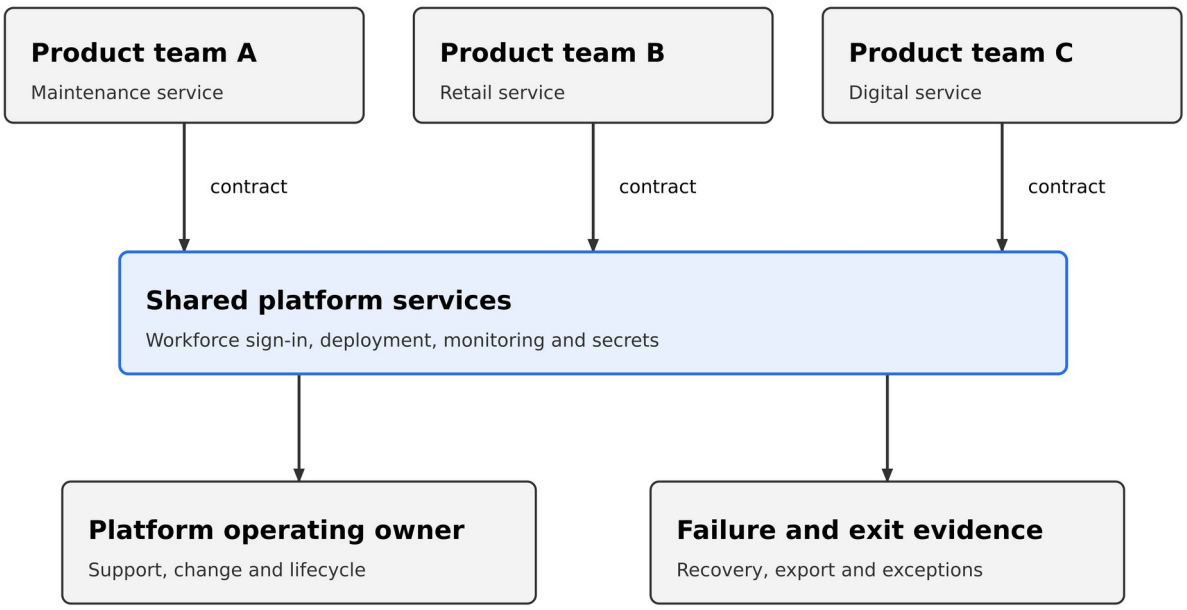


Figure 9 Shared platform services support product teams through contracts and named operating responsibility.

- time to obtain a working environment;
- the share of standard deployments that succeed first time;
- recovery time after platform incidents;
- the effort a team needs to meet the baseline controls.

Counting platform features or registered teams does not show that delivery improved.

A platform also creates shared dependencies. If every application depends on one deployment control plane, one identity configuration or one shared network component, a single failure can affect them all. This is a **common-mode failure**. Standardisation reduces variety but can increase common-mode risk, so the architecture should evaluate both effects. For each platform service, identify its failure domain, the applications inside it and the recovery path.

Cloud strategy is a set of workload decisions

A vendor-neutral architecture begins with responsibilities and guarantees rather than identical implementations on every provider. Azure and AWS both provide managed compute, storage, databases, identity and monitoring, but the semantics, configuration and operating responsibilities of their services differ. Both providers publish well-architected frameworks that organise workload reviews around pillars such as reliability, security, cost optimisation, operational excellence and performance efficiency [27], [28]. They are useful checklists of questions; the answers still come from the workload.

Helios compares three strategies:

- **Preferred provider.** New workloads use one provider unless an exception is approved.
- **Selective multi-provider.** A second provider is used for specific workloads with a clear reason.
- **Dual-provider operation.** Critical workloads are built to run on either provider.

A second provider removes some dependencies but adds skills, contracts, identity integration, monitoring and data transfer costs. It does not create disaster recovery by itself; recovery comes from tested procedures. Helios chooses a preferred provider for ordinary new workloads, allows exceptions for material constraints, and requires tested data export and a documented exit plan for strategic dependencies.

Portability has four meanings

“Portable” can mean four different things:

- **Code portability:** the application can run elsewhere.
- **Data portability:** the data can be extracted with its meaning and relationships intact.
- **Operational portability:** deployment, identity, monitoring, skills and recovery work in the new environment.

- **Commercial portability:** licences and contracts allow the move at an acceptable cost.

A container image addresses only the first. For example, for a critical customer service, a useful exit rehearsal exports representative data, rebuilds a small workload in a separate environment and runs a real business journey through it. A policy demanding theoretical portability without funding any test gives weak assurance. Avoiding every managed service in the name of portability can also impose large operating costs with no realistic exit benefit.

Lifecycle and technical debt

Standards need review dates and a process for retiring obsolete technologies. Record each technology's support status, upgrade dependencies and the business consequence of delay. **Technical debt** is the future cost of change created by past design shortcuts; connect it to a specific cost or risk, rather than using it as a label for any technology an architect dislikes.

Fund platform maintenance alongside product change. A mandated shared component with no capacity for patching, upgrades or support creates an unmanaged responsibility. The roadmap must include the work needed to keep the foundation usable.

Organisation structure and system structure

In 1968 Melvin Conway stated the thesis that "organizations which design systems (in the broad sense used here) are constrained to produce designs which are copies of the communication structures of these organizations" [29]. The observation, now called **Conway's law**, has a simple mechanism: two components can only fit together well if the people designing them talk to each other, so the interfaces in a system tend to follow the lines of communication between teams.

MacCormack, Baldwin and Rusnak tested a related idea, the **mirroring hypothesis**, by comparing five matched pairs of software products [30]. Each pair contained an open-source product, developed by a loosely coupled, distributed community, and a commercial product of similar size and function, developed by a tightly coupled firm. In every pair the open-source product was significantly more modular, and the differences were large: up to a factor of eight in how far a change to one component could spread to others. The sample is small and the products' functional equivalence was not tested directly, so the study supports a relationship in software products rather than a rule that changing an organisation chart will improve any system. Skelton and Pais turn the same idea into a design approach: they define team types, including platform teams that provide internal services to teams aligned with business streams, and shape team boundaries to match the system boundaries the organisation wants [31].

Worked diagnosis: a shared database

Helios observes a problem: three application teams wait for a central database team to approve their changes, because their applications write directly to shared tables. Figure 10 shows the technical dependencies and the approval dependencies side by side.

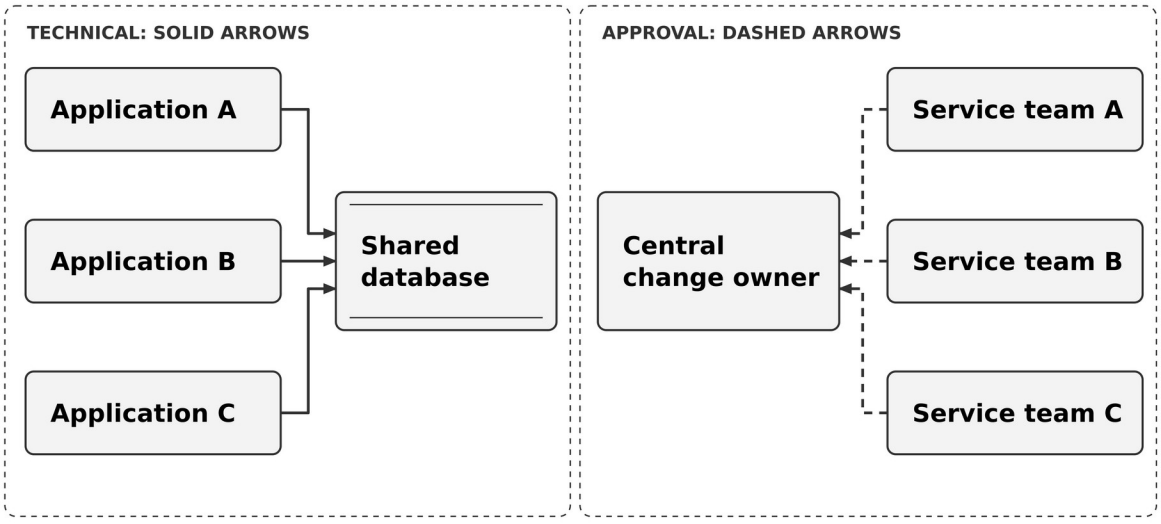


Figure 10 Technical dependencies (solid) and approval dependencies (dashed) for three Helios applications and a shared database.

Before redesigning anything, the architect examines the ten most recent changes:

- 8 of the 10 needed approval from the central team.
- Waiting times for the ten changes were 0, 0, 2, 3, 6, 9, 9, 11, 12 and 15 working days, a median of 7.5 days; the two zeros needed no approval.
- 5 of the changes wrote directly to a shared table.
- 6 of the 8 approvals were for columns used by a single application, with no shared meaning.

A cross-tabulation shows how these counts fit together:

Change touched	Needed central approval	No approval	Total
Shared table, column with shared meaning	2	0	2
Shared table, column used by one application	3	0	3
The application's own table	3	2	5
Total	**8**	**2**	**10**

The five shared-table changes are the first two rows. The six approvals for single-application data are the 3 + 3 in the second and third rows.

The pattern suggests that most of the delay comes from an approval rule rather than from genuine coupling. Six approvals concerned data that only one team uses, so delegating those decisions to the owning team would remove most of the waiting without any new technology. The three single-application columns in shared tables could move to their owning applications' tables, and the two changes to data with shared meaning deserve a technical response.

Two technical alternatives are available for those:

- **A shared data service.** Give the shared data a published contract, a funded backlog and a service owner. Teams use supported operations without individual requests. This reduces repeated work but makes the service's capacity an enterprise dependency.
- **Ownership by capability.** Each team owns its authoritative data and publishes events or APIs for others. This removes shared writes but adds integration, duplication and reconciliation work. If a rule genuinely needs one immediate transaction across the data, splitting it across teams can make things worse.

Conway's law explains why either option needs an organisational change as well. Product teams that still write to shared tables still depend on each other; a new API owned by an overloaded central team keeps the old bottleneck. Ten changes are a small sample, so treat the diagnosis as a hypothesis and keep the counterexamples, the changes where central review caught a real problem.

Exercises

7.1 Write a technology standard for Helios's application logging, with scope, requirement, rationale, owner, release evidence and exception route.

7.2 Compare code, data, operational and commercial portability for an application that uses a provider's managed database.

7.3 Give one common-mode failure introduced by a shared platform and a recovery requirement that addresses it.

7.4 Using the ten-change sample, calculate the mean wait across all ten changes and across the eight that needed approval, and the share of approvals that concerned single-team data. Recommend a first action and explain what result after three months would show it worked.

7.5 Explain Conway's law. Then use it and the mirroring study to justify an investigation at Helios, and explain why neither can prove that reorganising Helios will improve delivery performance.

Key points

- Separate principles, standards, reference designs and project decisions. A good standard states its scope, rationale, owner and exception route.
- A shared platform is a service with users. Measure it by their experience, and identify the common-mode failures it introduces.
- Cloud strategy is a set of workload decisions, and portability has code, data, operational and commercial meanings that need separate tests.

- Conway’s law and the mirroring hypothesis link system structure to communication structure. Diagnose technical and organisational dependencies together before choosing a fix.

CHAPTER 8

Governance, decision rights and exceptions

Learning objectives

After this chapter you should be able to:

- Explain IT governance as the allocation of decision rights, using five decision domains and six governance archetypes.
- Explain how ISO/IEC 38500 and COBIT 2019 separate governance from management, and apply them to a board's oversight of a programme.
- Design a decision-rights matrix that separates the right to give input from the right to decide.
- Design proportionate review triggers and a governed exception process.
- Evaluate governance mechanisms against the tensions they must manage, using research on bimodal IT.

Governance decides who may decide

Helios's architecture work stalls whenever nobody knows who may approve a change of customer meaning or accept a risk. This chapter gives the group a way to allocate those decisions and to review changes in proportion to their consequences.

Weill and Ross define **IT governance** as specifying the decision rights and accountability framework that encourages desirable behaviour in the use of IT [32]. Governance is not the same as management. Management makes and carries out decisions; governance determines who makes which decisions, who must be consulted and who is held accountable.

Their book, which draws on a study of about 250 enterprises, organises IT decisions into five related **decision domains**. Each has a Helios example:

- **IT principles:** high-level statements about how IT is used, such as "share data before processes".
- **IT architecture:** the organising logic for data, applications and infrastructure, such as the group customer-identifier standard.
- **IT infrastructure:** shared technical services, such as the cloud platform and the workforce identity service.
- **Business application needs:** the applications that units need, such as Maintenance's request for site-level scheduling.
- **IT investment and prioritisation:** how much to spend and on what, such as the allocation of the €9 million envelope.

They also describe six **archetypes**, typical patterns of who holds the rights for a domain:

Archetype	Who decides	Where it might fit at Helios
Business monarchy	Senior business executives, sometimes with the CIO	Approving the transformation envelope
IT monarchy	IT executives	Technical standards for the cloud platform
Feudal	Each business-unit leader, for their own unit	Local features in a unit's own applications
Federal	Group executives and business-unit representatives together, with IT	The meaning of the group customer identifier
IT duopoly	IT executives with one group of business leaders	One division's application priorities
Anarchy	Individual users or small groups	Personal spreadsheets and tools

The archetypes are descriptive. None is right everywhere, but each suits some domains better than others. An IT monarchy for infrastructure standards can work well; an IT monarchy for business application needs usually produces systems nobody asked for. Anarchy is rarely chosen deliberately, yet it is what an organisation gets when nobody holds a decision right explicitly.

Input rights and decision rights

Weill and Ross also separate **input rights**, the right to be consulted, from **decision rights**, the right to make the call. They summarise an organisation's arrangements on a single page: a matrix of domains against archetypes, showing who provides input and who decides in each [33]. For Helios's year-one design:

Domain	Input from	Decision by
IT principles	Business units, IT	Business monarchy: group executive committee with CIO
IT architecture	Business units, data owners	Federal: architecture council with unit representatives
IT infrastructure	Product teams	IT monarchy: CIO and platform owners
Business application needs	Users, architects	Feudal within shared standards: each unit
Investment and prioritisation	All units, architects	Business monarchy: board and executive committee

Standards for IT governance: ISO/IEC 38500 and COBIT 2019

Weill and Ross describe how decision rights are allocated. Two widely used standards describe what the governing body and management should do with them.

ISO/IEC 38500

ISO/IEC 38500 gives principles for the governance of IT to the governing body of any organisation, such as a board of directors [34]. Its model separates governance from management. The governing body **evaluates** proposals and plans for the current and future use of IT, **directs** management through strategies and policies, and **monitors** performance and conformance against them. Management prepares the proposals and runs the IT.

The 2015 edition defined six principles: responsibility, strategy, acquisition, performance, conformance and human behaviour. Many textbooks still teach these. The current edition, ISO/IEC 38500:2024, replaces them with the eleven governance principles of ISO 37000 [35]:

- **Primary:** purpose.
- **Foundational:** value generation, strategy, oversight and accountability.
- **Enabling:** stakeholder engagement, leadership, data and decisions, risk governance, social responsibility, and viability and performance over time.

According to the working group's white paper, the 2024 edition's model also adds **engage stakeholders** as an element alongside evaluate, direct and monitor, and stresses that governance is exercised throughout the organisation, with management working inside the framework the governing body sets. For Helios, the board evaluates the options paper and the €9 million envelope, directs through the architecture principles and the funding gates, and monitors through gate results, the benefits register and overdue exceptions. Engaging stakeholders means hearing the business units and key customers before each gate rather than after it.

COBIT 2019

COBIT 2019, published by ISACA, is a framework for the governance and management of enterprise information and technology [36]. Its core model holds 40 objectives and keeps governance and management apart:

- **Governance objectives** sit in one domain, Evaluate, Direct and Monitor (EDM). Its five objectives include ensured benefits delivery and ensured risk optimisation.
- **Management objectives** sit in four domains: Align, Plan and Organise (APO); Build, Acquire and Implement (BAI); Deliver, Service and Support (DSS); and Monitor, Evaluate and Assess (MEA). Their 35 objectives include managed enterprise architecture (APO03) and managed data (APO14).

COBIT does not prescribe one governance system for every organisation. It tailors the system through eleven **design factors**: enterprise strategy, enterprise goals, risk profile, I&T-related issues, threat landscape, compliance requirements, role of IT, sourcing model for IT, IT implementation methods, technology adoption strategy and enterprise size. Each factor raises or lowers the priority of particular objectives and the capability level to aim for. At Helios, a risk profile shaped by acquired applications with local accounts, and an I&T-related issue of inconsistent customer data, would raise the priority of managed security services and managed data.

The three sources answer different questions. Weill and Ross ask who holds each decision right. ISO/IEC 38500 asks what the governing body must do. COBIT supplies a detailed catalogue of governance and management objectives and a method for tailoring it. An architect can use COBIT's managed enterprise architecture objective to check their own practice, and the evaluate–direct–monitor model to frame what the board needs from architecture: options to evaluate, principles through which to direct and evidence to monitor.

A decision-rights matrix

The governance matrix sets the pattern. Day-to-day work needs a more specific matrix for recurring decisions. The following proposal for Helios names the decision, who provides input, who is accountable and what information the decision needs:

Decision	Input from	Accountable decision-maker	Information expected
Local implementation within standards	Product engineering, operations	Product owner, within delegated scope	Tests and a decision record
Meaning of the shared customer identifier	Domain owners, data stewards	Group data owner	Meaning and consumer-impact analysis
Strategic application consolidation	Business units, finance, architects	Investment sponsor or board	Options, costs, transition and benefits
Exception to a platform standard	Delivery, platform and risk specialists	Owner of the standard	Reason for the exception and compensating controls
Acceptance of material residual risk	Security, operations, legal, business	Authorised business risk owner	Consequences and treatment options

The matrix makes the decision path visible before a dispute blocks delivery. It also prevents two common errors. A committee can review information and recommend, but someone must still be accountable. An architect can advise on a risk, but does not acquire the authority to accept a financial or legal risk merely because it appears on an architecture diagram.

Review in proportion to consequence

A small, reversible implementation choice should not wait for the same forum as a change to the meaning of customer data. Define **review triggers**, the conditions that send a change for wider review:

- impact across business units;
- a new use of sensitive data;
- spending above a set threshold;
- a new strategic supplier dependency;
- a changed recovery promise;
- an exception to a mandatory control.

Teams should know when they may proceed on their own. Consult early on uncertain choices, and use a release gate to confirm conditions that can be verified. If the only architecture review happens just before deployment, it finds expensive problems too late. If every experiment needs detailed approval, teams cannot gather the information a good decision needs.

Record the scope of each approval. Approving a target architecture does not approve every later configuration. Tie release evidence to the version being deployed.

Exceptions are governed decisions

An **exception record** states the affected standard, the reason, the scope, the duration, the owner, the risk, the compensating measures and the review trigger. It also states the path to compliance, or the condition under which the standard itself should change. An exception without an owner or an end date becomes invisible permanent policy.

For example, an acquired application cannot use the workforce identity service before its first migration deadline. A time-limited exception might allow restricted interim access with extra monitoring and a funded plan to integrate within six months. Whether that is acceptable depends on the actual risk and obligations.

The record must describe controls that are actually operating; a planned control is a condition of approval until it works.

Repeated similar exceptions tell you something about the standard. The platform may not meet a common need, the migration schedule may be unrealistic, or the policy may be out of date. Governance should learn from the pattern rather than keep issuing waivers.

Funding and incentives

Suppose the group mandates a common integration platform while each division pays only for its own delivery team. The platform team asks divisions to fund shared reliability work, and each division prefers another to pay, because all will benefit. A platform funded centrally without visible demand has the opposite problem: it may chase adoption counts rather than useful service.

Compare two funding models:

- **Central allocation** can fund capabilities whose benefits are shared and hard to attribute, but it needs an accountable sponsor and explicit outcome measures.
- **Consumption charges** reveal demand and cost, but a badly chosen unit can discourage the behaviour the architecture needs. Charging per published event, for example, might encourage teams to suppress events that reconciliation depends on.

A charging design must not contradict the architecture's correctness requirements.

Governance tensions: research on bimodal IT

Some organisations split IT into a traditional mode, optimised for stability, and an agile mode, optimised for speed. Jöhnk and colleagues studied the governance of two such **bimodal IT** organisations, a public-sector IT function and the IT function of a multinational automotive company, through fourteen semi-structured interviews and internal documents [37]. They classified governance mechanisms as **structural** (roles and units), **procedural** (processes and rules) or **relational** (collaboration and trust), and as either **impeding** mechanisms that create or worsen challenges or **coping** mechanisms that help resolve them. For example, a diluted and distributed strategic vision impeded coordination, while synchronising agile release planning with the capacity of traditional IT helped.

They interpret the challenges as five paradoxes, pairs of goals that pull against each other and must be balanced rather than resolved once:

- **Strategic vision:** flexibility versus predictability.
- **Alignment:** aligning business with IT versus aligning the two IT modes with each other.
- **Organisation:** simplicity versus complexity of structures.
- **Distinction:** comparability between the modes versus differentiation.
- **Collaboration:** integration versus autonomy.

Helios is not a bimodal organisation, but it faces the same kinds of tension. The collaboration paradox appears whenever a division's autonomy meets a shared contract. The distinction paradox appears in group reporting, which needs comparable measures while divisions legitimately differ. The study is exploratory, based on two contrasting cases, so it provides sensitising questions rather than a recipe.

Worked example: a change of meaning

Helios needs to add a service-site role to its customer event contract. Retail can use its existing buyer identifier; Maintenance needs a separate site identifier because entitlement depends on location. The implementation adds one field, but the request changes the meaning of the contract, so it is not eligible for automatic approval.

Arrangement	How the decision works	What could defeat it
Central committee	One forum resolves meaning, priority and transition	Meeting delay; local specialists absent; approval without funded delivery
Delegated contract owner	The owner approves compatible additions under published rules	Delegation beyond expertise; affected consumers left out; no escalation route
Independent local change	Each division publishes its own interpretation	The same field acquires incompatible meanings; consumers absorb the cost

A defensible choice delegates genuinely compatible changes to the contract owner but escalates this change of meaning to the group data owner, with both business roles represented. It uses all three kinds of mechanism:

- **Structural:** name the decision owner and the escalation authority.
- **Procedural:** require example records, consumer-impact tests, a migration window and a versioned decision.
- **Relational:** bring the service staff who resolve disputed cases into the design discussion.

The choice also needs a failure path. If the meaning cannot be agreed before the delivery deadline, keep the existing contract and publish a provisional interface to a bounded pilot, if its controls and funding are authorised. Do not silently repurpose a field in production. The sponsor then chooses whether to delay, narrow the scope or fund a separate bridge; the architect makes those consequences visible.

Measure governance as a service

Track decision lead time by category, rework caused by late findings, overdue exceptions and consumer incidents after approved changes. Read them together. A shorter approval queue is weak evidence of success if teams now spend longer repairing mismatches, and a longer deliberation can be justified when it prevents a costly error. Classify requests by consequence, so that a change in the mix of work is not mistaken for a change in governance quality.

Publish decisions and their rationale in a form that teams can use. Meeting minutes nobody can find invite repeated debate; a concise standard, an example and a decision record reduce coordination effort while keeping accountability clear.

> **Common mistake.** Giving “the architecture board” every decision. Boards are good at reviewing and recommending. Accountability for accepting cost and risk belongs to named business owners.

Exercises

8.1 Distinguish the right to advise from the right to accept risk in a platform exception. Who holds each at Helios?

8.2 Draft a time-limited exception for an acquired application’s identity integration. Describe only controls that are actually operating, and state the conditions for approval.

8.3 Three teams request the same exception within a quarter. Give two possible explanations and the information you need before changing the standard.

8.4 Classify Helios’s current, informal arrangements for each of the five decision domains by archetype, assuming that units buy their own applications, the CIO sets infrastructure standards and nobody owns the customer identifier. Identify the most serious gap.

8.5 For the service-site change, choose one decision path. Specify its structural, procedural and relational mechanisms, and name which of the five paradoxes it must balance.

8.6 A division proposes paying for the integration platform per event published. Evaluate the proposal and suggest a better charging unit.

8.7 Map the Helios board’s oversight of the transformation programme to ISO/IEC 38500’s tasks of evaluating, directing, monitoring and engaging stakeholders, giving one concrete activity for each. Then name one COBIT 2019 design factor that should raise the priority of managed data (APO14) at Helios, and explain your choice.

Key points

- IT governance allocates decision rights and accountability. Weill and Ross’s five domains and six archetypes give a vocabulary for describing who decides what.
- ISO/IEC 38500 frames the governing body’s work as evaluate, direct and monitor, with stakeholder engagement and eleven principles in its 2024 edition. COBIT 2019 separates five governance objectives from 35 management objectives and tailors them with design factors.
- Separate input rights from decision rights, and record both in a matrix before disputes arise.
- Match review to consequence with explicit triggers, and keep exceptions owned, bounded and visible.
- Funding models create incentives; a charging unit must not discourage behaviour the architecture depends on.
- Research on bimodal IT shows that governance balances persistent tensions with structural, procedural and relational mechanisms, some of which help and some of which hinder.

CHAPTER 9

Enterprise risk, security and assurance

Learning objectives

After this chapter you should be able to:

- Map a critical business service to its technical, supplier and human dependencies.
- Write a risk statement with a condition, an event and a consequence, and derive controls from it.
- Explain the principles and logical components of a zero trust architecture and apply them to an access risk.
- Calculate the effect of serial and shared dependencies on service availability.
- Design an assurance plan that links each claim to current evidence.

Begin with the service the organisation must preserve

Helios must keep urgent maintenance running while it changes the systems underneath. This chapter shows how to find what that service depends on, how to state its risks and how to prove that the controls work.

A critical business service may depend on several applications, identity systems, networks, data stores, suppliers and people. Assess the combined service rather than treating each application as a separate entry in a risk register. Helios's urgent maintenance service, for example, cannot schedule work if entitlement checking, technician sign-in through the workforce identity service or access to ERP Core fails, even when the customer portal is running normally. Figure 11 shows the dependencies.

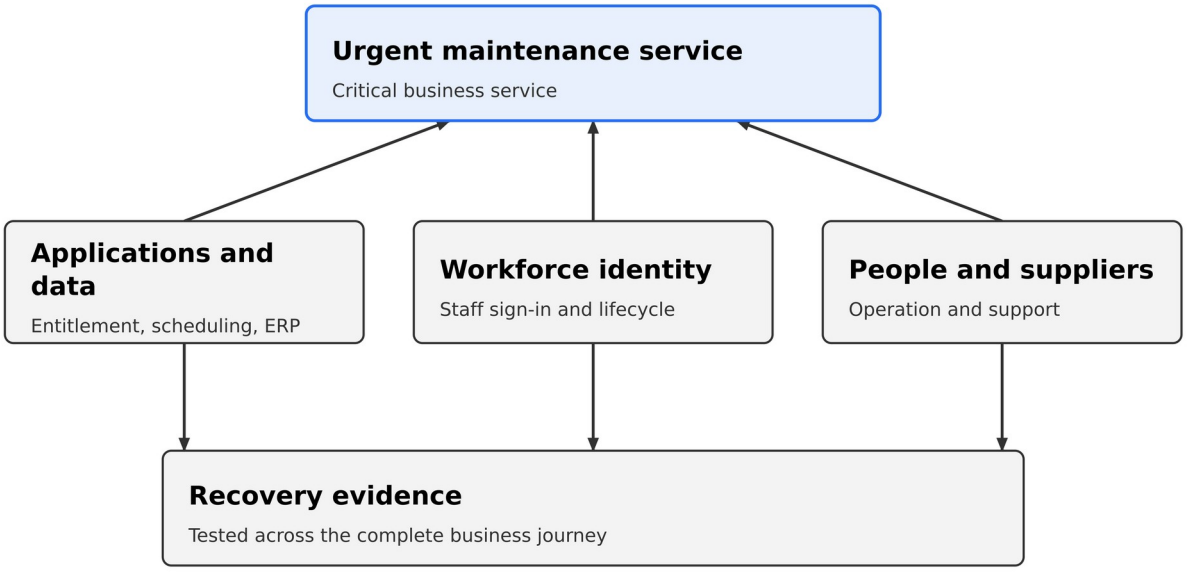


Figure 11 A critical business service depends on applications, workforce identity, people and suppliers, and recovery must cover the whole journey.

Map each dependency and its consequence. Ask three questions:

- Which functions can continue in a degraded mode, and for how long?
- Which information must remain trustworthy during the disruption?
- Who can authorise a switch to manual operation?

A fallback spreadsheet may carry a short interruption, but it creates reconciliation, access and recovery work of its own. Plan that work in advance.

Frameworks organise the work

The NIST Cybersecurity Framework 2.0 groups cybersecurity outcomes under six functions: Govern, Identify, Protect, Detect, Respond and Recover [38]. The Govern function, new in version 2.0, connects cybersecurity strategy, roles, policy and oversight to the organisation's enterprise risk management. The framework describes outcomes; each organisation chooses the controls and evidence that achieve them

in its context. More generally, ISO 31000 defines **risk** as the effect of uncertainty on objectives, which keeps attention on what the organisation is trying to achieve rather than on a list of threats [39].

Write risk statements, then derive controls

A useful **risk statement** names a condition, an event and a consequence. For example: because account removal does not reach the acquired applications (the condition), a former employee may sign in after leaving (the event), causing unauthorised disclosure of customer service data (the consequence).

The controls follow from the mechanism:

- an authoritative lifecycle event that reaches every application;
- monitoring of which applications receive those events;
- an automated test that a removed account is refused;
- an exception process for applications that cannot yet integrate.

“Use encryption” would leave this risk untouched, because an authorised application account can still decrypt the data. Control selection needs a threat and an access path.

Zero trust as a design response

The same risk shows why many organisations are moving towards **zero trust**. NIST describes zero trust as a set of principles that move defences from static, network-based perimeters to users, assets and resources [40]. A zero trust architecture grants no implicit trust because of where a device sits on the network or who owns it. Its tenets include treating every data source and service as a resource, securing all communication wherever it travels, granting access to each resource per session with the least privilege needed, and deciding access by dynamic policy that considers the identity of the user or service, the state of the requesting device and other attributes. NIST separates the decision from its enforcement. A **policy engine** decides whether a subject may use a resource, a **policy administrator** opens or closes the session accordingly, and a **policy enforcement point** in front of the resource carries out the decision.

The link to Helios’s risk is direct. The acquired applications keep local accounts, so each one trusts whoever holds a valid local password, whether or not that person still works for Helios. Under zero trust, every sign-in to those applications would pass through an enforcement point that consults the policy engine, and the engine would check the workforce identity service, where the leaver’s account is already disabled. Access would fail even if the application’s own account list were out of date. Zero trust is a direction for the target architecture rather than a product to buy. Helios can move towards it application by application, starting with those that hold customer service data, while the lifecycle events and denial tests above close the gap in the meantime.

Be careful with numerical risk scores. A score of “likelihood 3 × impact 4 = 12” looks precise but is the product of two judgements. Where likelihood is uncertain, show the supporting data, a range or a qualitative judgement. A severe but poorly quantified risk may still deserve action.

Shared dependencies and availability

Availability figures combine in ways that surprise people. Suppose the urgent maintenance service needs three applications, each available 99.9% of the time, and that their failures are independent. The service is available only when all three are, so its availability is 0.999 × 0.999 × 0.999 = 0.997, about 26 hours of downtime a year.

Now add the workforce identity platform, which all three applications need for staff sign-in and which is available 99.5% of the time. The service’s availability falls to 0.997 × 0.995 = 0.992, about 70 hours a year. The workforce identity platform alone contributes about 44 hours. A single shared component can matter more than all the applications together.

The calculation also rests on an assumption that is often false: independence. Applications hosted in different environments can still share an identity provider, a DNS configuration, an administrator account, a software release or a supplier’s control plane. When failures are correlated, multiplying separate availability estimates understates the risk. Identify those shared elements and test their recovery directly.

Assurance follows claims through evidence

Assurance is the justified confidence that controls work as intended. For each important claim, identify the requirement, the control, who implements it and the evidence that it works.

Claim	Control	Evidence	Scope and date
Removed staff cannot sign in	Lifecycle events to all applications	Automated denial test results	Production, each release
Privileged access is reviewed	Quarterly access review	Signed review records and removals made	All admin accounts, last quarter
Customer data can be recovered	Daily backups with restore tests	Restore test report with record counts	Production database, last month

Evidence that privileged access is reviewed is a signed review record and a list of removals; a policy stating that reviews happen is only an intention. A test run in a different environment supports a claim about that environment. Good evidence states its scope, date, configuration and exceptions.

Different kinds of evidence answer different questions: design reviews, configuration inspections, automated tests, restore exercises, supplier reports and operating observations. A certification or audit report is useful within its scope, but it rarely covers the customer's own configuration or every service used.

Supplier dependencies

Supplier assessment should cover the service contract, data use, subcontractors, incident communication, recovery evidence, export and exit. A supplier's promised recovery time usually refers to its infrastructure rather than to Helios's complete business service.

For example, a supplier advertises a 15-minute recovery time for its hosting platform. The end-to-end recovery of Helios's service also includes its own steps:

Step	Minutes
Helios detects the outage and declares an incident	20
Supplier restores the infrastructure	15
Helios decides to resume and restores user access	45
Integrations catch up on queued messages	60
Staff reconcile records changed during the outage	120
Total	**260**

The business service takes about 4 hours 20 minutes to recover, and the supplier's 15 minutes is the smallest step. Most of the time belongs to Helios. Design the response to supplier failure before signing a long-term dependency: tested export, alternative operating procedures, recoverable copies held under policy, and a realistic transition plan. A second supplier helps only if Helios can actually switch the business responsibility safely.

A first-year assurance plan

Helios's first-year plan focuses on four areas: workforce identity lifecycle coverage, recovery of critical service data, reliable handoffs and incident ownership. It begins with a baseline of known gaps and links each remediation to a roadmap milestone. Working controls with current evidence make the programme compliant; adopting a framework organises the work.

For each release, keep evidence that critical access and recovery controls work in the deployed configuration. Reassess when an acquisition, a new data use or a shared-platform change alters the risk.

> **In practice.** Ask for the most recent restore test rather than the backup policy. A dated report with record counts answers the question the board actually cares about: could we get the data back?

Exercises

9.1 Write a risk statement for incomplete removal of staff access in acquired systems. Derive two controls and state the evidence that each one works.

9.2 Explain why three applications hosted in different environments can still share a common failure mode. Give two examples.

9.3 A service needs four applications, each 99.8% available, and a shared network component that is 99.9% available. Assuming independent failures, calculate the service's availability and annual downtime in hours. Then explain how correlated failures would change your answer.

9.4 A supplier advertises a fifteen-minute recovery objective. List the questions you would ask before using that number in Helios's business-continuity plan.

9.5 A project team presents an ISO certificate held by its cloud supplier as proof that the new customer service is secure. Evaluate this evidence and say what further evidence you would request.

9.6 Explain how a zero trust architecture would change the risk created by acquired applications that keep local staff accounts. Name the three logical components NIST describes, and say where each would sit for one Helios application.

Key points

- Analyse risk at the level of the business service, following its dependencies across applications, shared platforms, people and suppliers.
- A risk statement names a condition, an event and a consequence; controls follow from the mechanism. Zero trust removes implicit trust and checks every access against current identity and policy.
- Serial dependencies multiply, and shared components can dominate a service's downtime. Correlated failures make simple multiplication optimistic.
- Assurance links each claim to a control and to current evidence with a stated scope and date.
- Supplier recovery times cover the supplier's part. Plan and measure the organisation's own recovery steps as well.

CHAPTER 10

Investment economics, benefits and uncertainty

Learning objectives

After this chapter you should be able to:

- Calculate net present value, internal rate of return and simple and discounted payback for an architecture investment, stating the timing convention used.
- Apply real and nominal discount rates consistently, and compare options by total cost of ownership.
- Build a benefits register that separates cash, capacity and risk benefits without double counting.
- Evaluate a staged investment as a real option, using the value of the information a pilot provides.
- Test whether a recommendation survives changes to its assumptions, including a funding cut.

A business case is a model of consequences

The board will release the €9 million only against business cases it can check. This chapter builds the case for the customer-handoff investment and tests how far it bends before it breaks.

Architecture changes cash flows through delivery cost, operation, transition, retirement, revenue and losses. A credible business case says who experiences each effect and when. Three rules prevent the most common errors:

- **Compare with a realistic baseline.** “Do nothing” still includes running costs, necessary maintenance, supplier renewals and known end-of-life events. A baseline that assumes free continuation forever makes every change look expensive; one that assumes certain disaster makes every change look attractive.
- **Count only incremental future cash flows.** Money already spent cannot be recovered by any choice, so it stays out of the comparison. Existing assets can still have future disposal, contract or transition consequences, and those belong in it.
- **Count the group’s cash.** Moving a cost from one business unit to another saves the group nothing. Reducing staff effort creates capacity; it becomes a cash saving only when an actual payment stops.

The Helios customer-handoff investment

The worked example in this chapter is the investment that makes cross-unit handoffs reliable. It draws on three roadmap work packages from Chapter 11: customer identity resolution and data foundations (€1.0 million), integration capability (€1.2 million) and €0.3 million of the adoption and assurance package, a total of €2.5 million within the €9 million envelope.

The separate workforce identity and access package (€0.5 million) is deliberately left out. It makes sure that staff accounts are created and removed through the group identity service, which reduces the risk of former employees keeping access. That is a risk benefit with its own justification, and charging it to the handoff case would make the handoff case look worse and the access case invisible.

The expected net cash benefit is €1.2 million a year, made up of:

- €0.5 million less spent on contractors who reconcile customer records by hand;
- €0.3 million of licence and support fees for two matching tools that will be retired;
- €0.4 million less paid in service credits to customers after failed handoffs.

To keep the arithmetic clear, the model treats the €2.5 million as spent at the start and the benefits as arriving at the end of years 1, 2 and 3, with nothing after year 3. In the roadmap (Chapter 11) most of this spending falls in years 1 and 2 and a small part, mainly integration work, in year 3. Because later spending is discounted, booking it all at the start makes the cost side slightly conservative; the board should know that the model simplifies the timing. All values are in euros; the discount rate is 8%.

Net present value

Net present value (NPV) converts future cash flows to their value on a single date by discounting them, then subtracts the investment [41]:

$$NPV = -I + CF1/(1 + r) + CF2/(1 + r)^2 + CF3/(1 + r)^3$$

where I is the investment at time zero, CFt is the net cash flow at the end of year t and r is the annual discount rate. At 8%, the three benefits are worth €1,111,111, €1,028,807 and €952,599 today. Their sum is €3,092,516, so the NPV is €592,516. The investment creates value under these assumptions, and the sensitivity analysis below shows how firmly.

Internal rate of return and payback

The **internal rate of return (IRR)** is the discount rate at which the NPV is exactly zero [42]. For this investment it is 20.71%, well above the required rate of 8%, which agrees with the positive NPV. IRR is easy to communicate but has limits: it says nothing about the size of the gain, and cash flows that change sign more than once can have several IRRs. Use it alongside NPV.

Payback is the time until cumulative benefits repay the investment [43]. Its answer depends on a timing convention, which must be stated:

- **Year-end cash.** The NPV model places each year's benefit at the end of the year. Cumulative benefits are €1.2 million after year 1, €2.4 million after year 2 and €3.6 million after year 3, so the €2.5 million is repaid only when the year-3 cash arrives: a payback of 3 years. Discounted payback is also 3 years, because the discounted benefits reach €2,139,918 after two years and €3,092,516 after three.
- **Even accrual.** If benefits arrive evenly through each year, payback can be interpolated within year 3. Simple payback is 2 + 0.1 / 1.2 = 2.08 years. Discounted payback is 2 + 360,082 / 952,599 = 2.38 years.

The discounted figure is a hybrid: it interpolates within year 3 while still discounting that year's benefit as if it arrived at the year end, which is how most spreadsheets compute it. Interpolated payback is a common reporting convention, but it contradicts a model that books each year's benefit at the year end. Report the convention that matches the cash-flow model, here 3 years, and show the other only with its label. Payback is a useful liquidity check, but it ignores everything that happens after the cut-off.

A listing to reproduce the figures

The following program computes the NPV under seven scenarios, including the two funding-cut scenarios discussed at the end of the chapter, the IRR by bisection, and payback under both timing conventions. It uses only the Python standard library.

```
"""NPV, IRR and payback for the Helios customer-handoff investment (EUR)."""

def npv(rate: float, investment: float, flows: list[float]) -> float:
    """Investment at time 0; flows[t - 1] arrives at the end of year t."""
    return -investment + sum(cf / (1 + rate) ** t for t, cf in enumerate(flows, 1))

def irr(investment: float, flows: list[float]) -> float:
    lo, hi = 0.0, 1.0                       # bisection: NPV falls as the rate rises
    for _ in range(100):
        mid = (lo + hi) / 2
        lo, hi = (mid, hi) if npv(mid, investment, flows) > 0 else (lo, mid)
    return mid

def payback(investment: float, flows: list[float], rate: float = 0.0,
            even: bool = False) -> float:
    """Cash arrives at year ends, or accrues evenly through each year if even=True."""
    left = investment
    for t, cf in enumerate(flows, 1):
        pv = cf / (1 + rate) ** t
        if pv >= left:
            return t - 1 + left / pv if even else t
        left -= pv
    return float('inf')

cost, base = 2_500_000, [1_200_000] * 3
scenarios = {  # name: (rate, investment at time 0, flows at the end of years 1, 2, ...)
    'Base case, 8%': (0.08, cost, base),
    'Lower rate, 4%': (0.04, cost, base),
    'Higher rate, 12%': (0.12, cost, base),
    'Benefits 20% lower': (0.08, cost, [960_000] * 3),
    'Benefits one year later': (0.08, cost, [0] + base),
    'Cut: 0.6m deferred': (0.08, cost - 600_000, [-600_000] + base),
    'Cut: 0.6m removed': (0.08, cost - 600_000, [800_000] * 3),
}
for name, (rate, inv, flows) in scenarios.items():
    print(f'{name:<24} NPV EUR {npv(rate, inv, flows):>12,.0f}')
print(f'IRR {irr(cost, base):.2%}')
for even, timing in [(False, 'year-end cash:'), (True, 'even accrual:')]:
    print(f'Payback, {timing:<15} simple {payback(cost, base, 0, even):.2f}, '
          f'discounted {payback(cost, base, 0.08, even):.2f} years')
```

Running it prints:

```
Base case, 8%            NPV EUR      592,516
Lower rate, 4%           NPV EUR      830,109
Higher rate, 12%         NPV EUR      382,198
Benefits 20% lower       NPV EUR      -25,987
Benefits one year later  NPV EUR      363,441
Cut: 0.6m deferred       NPV EUR      407,886
Cut: 0.6m removed        NPV EUR      161,678
IRR 20.71%
Payback, year-end cash:  simple 3.00, discounted 3.00 years
Payback, even accrual:   simple 2.08, discounted 2.38 years
```

The bisection assumes that NPV falls as the rate rises, which holds for an investment followed only by positive cash flows. It is used here for the base case only.

Sensitivity: does the answer survive?

Figure 12 shows the first five scenarios. The case survives a higher discount rate and a one-year delay in benefits, but it is sensitive to the size of the benefits. The break-even annual benefit at 8% is €2,500,000 / 2.5771 = €970,084, which is 19.2% below the estimate. A 20% shortfall therefore turns the NPV slightly negative, to –€25,987. Delaying the benefits by one year, with the investment still at time zero, reduces the NPV to €363,441; a real delay would usually add a year of transition cost as well.

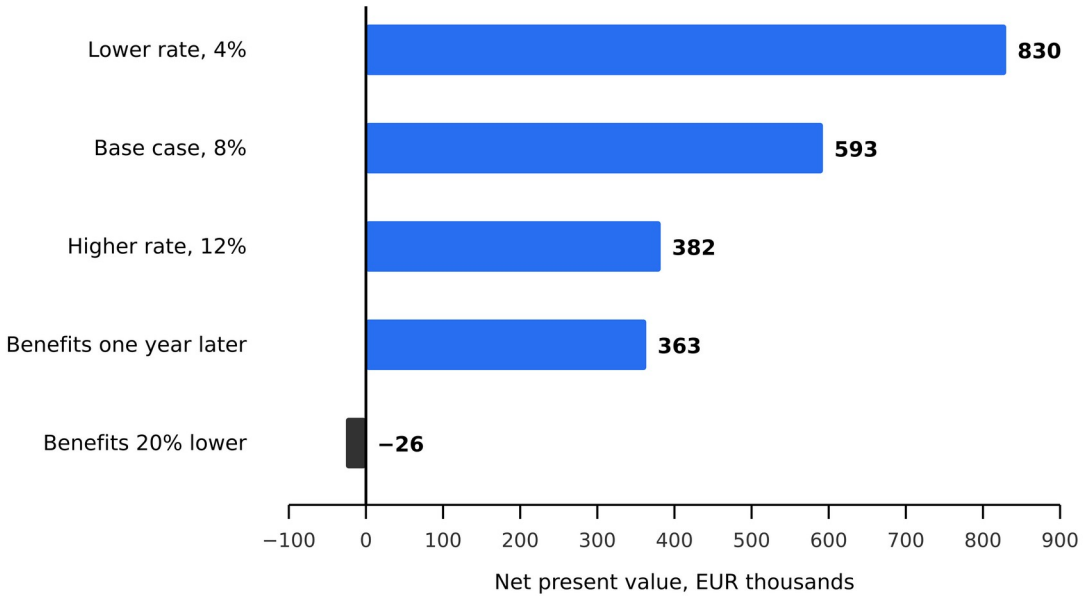


Figure 12 NPV of the €2.5 million customer-handoff investment under five scenarios, in thousands of euros.

Sensitivity analysis shows what the conclusion depends on; it does not say how likely each scenario is. The useful output here is a question for the board: how confident are we that benefits will reach at least €0.97 million a year? The three benefit lines deserve different answers. The licence fees are contractual and easy to verify, the contractor saving depends on a staffing decision, and the service-credit reduction depends on how often handoffs fail in future. Keep full precision in calculation and round only the reported result. A difference of a few thousand euros means little when migration costs are uncertain by hundreds of thousands.

Real and nominal rates

Cash flows and discount rates must use the same convention. **Nominal** amounts are in the euros of the year they occur; **real** amounts are in constant, today's prices. The Fisher relation links the two rates: (1 + nominal) = (1 + real) × (1 + inflation) [44].

The base case treats the €1.2 million benefits as nominal: fixed euro amounts, as with a contract fee that does not rise. Suppose instead that €1.2 million is in today's prices and the benefits rise with inflation of 2.5% a year. Two consistent calculations give the same answer:

- Inflate the benefits to nominal amounts (€1,230,000, €1,260,750 and €1,292,269) and discount at the nominal 8%.
- Keep the benefits in today's prices and discount at the real rate, 1.08 / 1.025 – 1 = 5.37%.

Both give an NPV of €745,623. Discounting today's-price benefits at the nominal 8% gives €592,516 and understates the NPV by about €153,000, roughly a quarter of the base-case value. Check every business case for this mistake.

Total cost of ownership

Total cost of ownership (TCO) counts all the costs of an option over its life: implementation, migration, licences, operation, support, training, upgrades and exit [45]. Chapter 5 compared options for Legacy Orders. Replatforming takes 8 months and repurchasing takes 12, and Legacy Orders keeps running, at €0.6 million a year, until the new service takes over. A fair comparison therefore covers the same five years from the decision for both options and includes the legacy running cost during each implementation.

Option	One-off cost	Year 1 running cost	Years 2 to 5, each	Five-year TCO, undiscounted	Five-year TCO, present value
Replatform	€0.60m	€0.55m	€0.45m	€2.95m	€2.49m
Repurchase	€0.90m	€0.60m	€0.40m	€3.10m	€2.68m

The year-1 figures combine both systems. Replatforming pays legacy costs for 8 months (€0.40 million) and the new platform for 4 months (€0.15 million). Repurchasing pays legacy costs for the whole year. One-off costs are at the start and running costs at each year end, discounted at 8%.

Repurchase has the lower running cost once it is live, but the higher one-off cost and a longer period of paying for the old system. Over five years replatforming is about €0.19 million cheaper in present value, roughly 7%. A naive comparison that ignores the transition, assuming each new service runs from day one at its own cost (€0.45 million and €0.40 million a year for all five years), gives present values of €2.40 million and €2.50 million, a gap of only €0.10 million. The omitted legacy cost therefore nearly doubles the difference. The difference is still within the uncertainty of the estimates, which tells the team that cost leans towards replatforming but will not decide the choice alone; deadline risk, process fit and exit terms will.

Benefits need owners and clean definitions

A **benefits register** records, for each benefit, its baseline, intended change, mechanism, measure, owner, timing and the data that will show it was realised. Benefits management research finds that organisations focus on implementing technology rather than on realising the benefits they expected, and recommends planning benefits explicitly from the start [46].

Classify each benefit:

- **Cash benefits** stop or reduce a payment, for example a licence fee after verified retirement.
- **Capacity benefits** free staff time, for example service agents who no longer reconcile records. They become cash only through a decision about staffing, contracts or new work.
- **Risk benefits** reduce exposure, for example fewer people with stale access after the workforce identity work. Describe the exposure and the mechanism; an invented probability multiplied by a dramatic loss is a weak estimate.

Avoid double counting. If reduced contractor hours are already in the lower operating cost, count them once. If customer retention improves, count the incremental contribution margin rather than gross revenue, and explain what else could have caused the improvement.

Staging and real options

A staged programme buys information before an expensive commitment. In finance, a **real option** is the right, without an obligation, to take a later action on a real investment, such as expanding, delaying or abandoning it. Benaroch and Kauffman showed how option-pricing ideas can value flexibility in IT investments, such as the choice of when to deploy [47]. For most architecture decisions, a simple decision tree captures the essential point: flexibility has value only when new information can change what you do next.

Worked example: a pilot before expansion

Helios is considering expanding the shared customer service to all three divisions. All values below are in euro millions at the same valuation date, so they are already discounted.

- If adoption is high, expansion is worth +1.0; if adoption is low, it is worth –0.6.

- Without further information, the probability of high adoption is 0.5.
- Keeping the current arrangement is worth 0.

Expanding now is worth 0.5 × 1.0 + 0.5 × (–0.6) = 0.20.

A pilot in one division costs 0.05, taken from the CRM and process pilot work package. The pilot gives a favourable result with probability 0.8 when adoption would be high and 0.2 when it would be low. Overall, a favourable result occurs with probability 0.5 × 0.8 + 0.5 × 0.2 = 0.5. By Bayes' rule, the probability of high adoption after a favourable result is (0.5 × 0.8) / 0.5 = 0.8; after an unfavourable result it is 0.2.

- After a favourable result, expansion is worth 0.8 × 1.0 + 0.2 × (–0.6) = 0.68, so Helios expands.
- After an unfavourable result, expansion is worth 0.2 × 1.0 + 0.8 × (–0.6) = –0.28, so Helios does not expand and gets 0.

The pilot policy is worth 0.5 × 0.68 + 0.5 × 0 – 0.05 = 0.29, against 0.20 for expanding now. The **expected value of the information** is 0.34 – 0.20 = 0.14, so any pilot costing more than 0.14 would not be worth running. Figure 13 shows the tree.

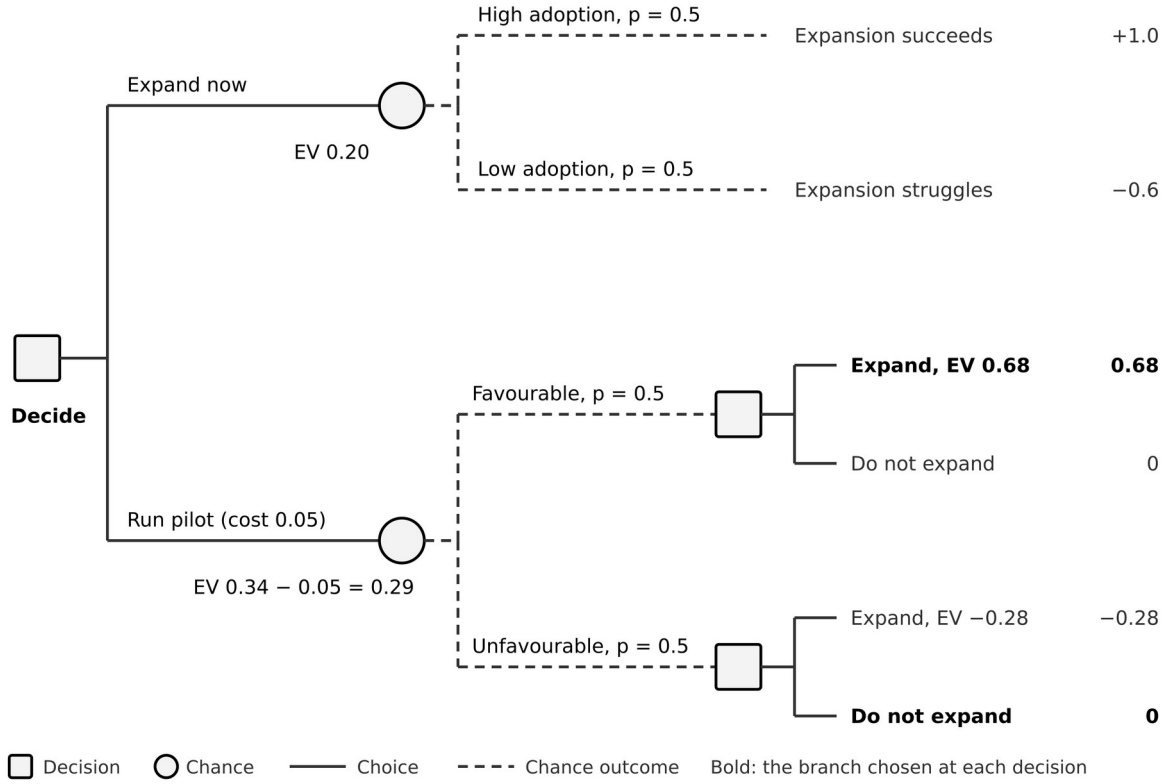


Figure 13 A pilot creates value only when its result can change the next decision. Squares are decisions and circles are chance nodes; solid lines are choices, dashed lines are chance outcomes and values are in euro millions.

Two variations show where the value comes from. If the pilot reported "favourable" with probability 0.5 whatever the true adoption, it would carry no information: expansion would still be worth 0.20 after either result, and paying 0.05 would leave 0.15. If Helios had already signed an irrevocable contract for full expansion, the pilot could not change the action and its information value would be zero. A polished demonstration can therefore be a poor experiment.

The model leaves things out. A pilot may delay benefits, teach the team how to implement, or fail to represent the other divisions. A low-adoption outcome may be a loss the board cannot accept even when the expected value is positive. State these concerns openly and rework the model where they matter, rather than hiding them in a higher discount rate.

Revisiting the case under the funding cut

Change card A cuts the programme envelope from €9.0 million to €7.2 million. The deferral scenario in Chapter 11 removes analytics improvement and part of legacy modernisation, and leaves the three packages behind the customer-handoff investment intact. Under that scenario the NPV is unchanged at €592,516.

Now suppose the cut fell instead on integration capability, taking €0.6 million from its €1.2 million. “Cut” can mean two different things, and the listing models both:

1. **Deferral.** The €0.6 million of work is postponed by a year and spent at the end of year 1. Because reliable handoffs need that work, the benefits also start a year later, in years 2 to 4. The cash flows are –€1.9 million at time zero, –€0.6 million at the end of year 1 and +€1.2 million at the end of years 2, 3 and 4. The NPV falls from €592,516 to €407,886.
2. **Removal.** The €0.6 million is never spent. It would have paid for the monitoring and recovery support that stops failed handoffs, so the €0.4 million a year of service-credit savings is lost. The cash flows are –€1.9 million at time zero and +€0.8 million at the end of years 1, 2 and 3. The NPV falls to €161,678.

Both versions save €0.6 million of programme budget, and both destroy value: €184,630 through delay, or €430,838 through lost benefits. Removal also leaves the case far more fragile, because a modest further shortfall in benefits would turn it negative. The architect’s job is to make that consequence visible, so that the board can choose where the cut falls with its eyes open.

Exercises

10.1 Recalculate the base-case NPV and the 20% benefit-shortfall case by hand. State the cash-flow timing and explain why the sign changes.

10.2 A proposal claims €200,000 from reduced contractor invoices and another €200,000 from the same released hours. Correct the benefits register and classify each entry.

10.3 Nominal rates are 8% and expected inflation is 3%. Calculate the real rate. If the €1.2 million benefits are in today’s prices and rise with inflation, calculate the NPV of the €2.5 million investment.

10.4 Recalculate the pilot policy if the pilot costs 0.16. Then explain what happens to its value if the pilot result cannot change the expansion decision.

10.5 The board must decide on the CRM migration although its cost is uncertain by ±40%. Design a bounded investigation and a decision rule that uses its findings.

10.6 A sponsor argues: “The customer-handoff investment has an IRR of 20.7%, far above our 8% hurdle, so it must be approved in full whatever happens to the budget.” Evaluate the argument using the sensitivity results and the two funding-cut scenarios.

10.7 A finance analyst reports a payback of 2.08 years for the customer-handoff investment, and the architect reports 3 years. Explain how both figures arise, and say which one the board should see alongside the NPV and why.

10.8 The finance team proposes charging the whole €1.5 million of identity work, workforce and customer, to the customer-handoff case. Calculate the resulting NPV at 8% and explain why the split used in this chapter gives the board better information.

Key points

- Compare options against a realistic baseline, count only incremental future cash flows, and count the group’s cash rather than a unit’s.
- NPV is the main measure; IRR and payback add useful perspectives but have known limits. Payback depends on a timing convention that must match the cash-flow model.
- Use nominal rates with nominal cash flows and real rates with real cash flows. Mixing them can move the NPV by a quarter or more.
- TCO counts every cost over the same period for each option, including the old system’s running cost during the transition. Benefits registers separate cash, capacity and risk benefits and avoid double counting.
- Staging is a real option: a pilot is worth paying for only when its result can change the next decision. Funding cuts must be traced through to the cash flows they change.

CHAPTER 11

Roadmaps, dependencies and transition architectures

Learning objectives

After this chapter you should be able to:

- Design intermediate states that keep the business operating during a transformation.
- Calculate earliest finish times, slack and the critical path, and explain how shared resources change them.
- Allocate a programme envelope to work packages that respect both package totals and annual limits.
- Revise a roadmap after a funding cut, stating clearly what changes and what is lost.

A roadmap is a sequence of operable states

A **target architecture** describes a desired arrangement. A **roadmap** explains how the organisation reaches it through intermediate states while continuing to operate. TOGAF calls these intermediate states **transition architectures** and develops them in phases E and F of the ADM [10]; ArchiMate models them as plateaus (Chapter 2). Every transition creates temporary ownership, data movement, support and retirement work.

“Replace the CRM in year two” is therefore incomplete. A usable roadmap entry says what migrates, which integrations change, how existing customers continue to be served during the change, and what has to be true before the old system can be switched off.

Begin with outcomes and constraints, then identify work packages and dependencies. Dependencies come in several kinds:

- **Technical:** the integration needs the shared identifier service.
- **Informational:** the migration needs an agreed meaning of “customer organisation”.
- **Contractual:** the export needs a right to the supplier’s data format.
- **Organisational:** the rollout needs trained stewards.
- **Regulatory:** the data transfer needs an approved legal basis.

Buying the target platform satisfies only one link in such a chain.

The Helios funding envelope

The following allocation is the starting scenario for the €9 million programme envelope. It is separate from the €12 million annual operating baseline. Some programme work will later change operating costs, but no saving is assumed merely because money has been allocated.

Work package	Three-year allocation	Intended outcome
Customer identity and data foundations	€1.0m	Governed cross-system customer identifiers, matching and stewardship
Workforce identity and access	€0.5m	Staff accounts created and removed through the group identity service
Integration capability	€1.2m	Reusable contracts, monitoring and recovery support
Legacy modernisation	€2.4m	Safe transition away from selected unsupported systems
Analytics improvement	€0.9m	Reproducible, governed reporting for agreed decisions
CRM and process pilot	€1.2m	Evidence about shared customer processes before wider consolidation
Adoption and assurance	€0.9m	Training, service ownership, testing and transition assurance
Contingency	€0.9m	Controlled response to identified uncertainty

The packages sum to €9.0 million, and the contingency sits inside that total. It is released only through a documented decision. The two identity packages are deliberately separate. Customer identity resolution links records about customer organisations across systems; workforce identity and access controls how staff sign in and lose access when they leave. They have different owners, benefits and risks, and Chapter 10 charges only the first to the customer-handoff business case.

Allocating the envelope by year

The board has also set annual spending limits of €3.0 million, €3.5 million and €2.5 million, which also sum to €9.0 million. A valid schedule must satisfy both sets of totals at once: each row must add up to its package total and each column must stay within its year's limit. The table shows one schedule that does, in € millions.

Work package	Year 1	Year 2	Year 3	Total
Customer identity and data foundations	0.6	0.4	0.0	1.0
Workforce identity and access	0.4	0.1	0.0	0.5
Integration capability	0.5	0.5	0.2	1.2
Legacy modernisation	0.6	0.9	0.9	2.4
Analytics improvement	0.0	0.4	0.5	0.9
CRM and process pilot	0.3	0.6	0.3	1.2
Adoption and assurance	0.3	0.3	0.3	0.9
Contingency, held in reserve	0.3	0.3	0.3	0.9
Annual total	**3.0**	**3.5**	**2.5**	**9.0**
Annual limit	3.0	3.5	2.5	9.0

The profile follows the dependencies rather than spreading money evenly. The customer identity foundations come first because integration, the pilot and analytics all need them, and the workforce identity work closes the former-staff access risk early. Legacy modernisation spends €0.6 million in year 1 because the Legacy Orders hosting contract ends in 14 months (Chapter 5). Analytics waits until year 2, when governed customer identifiers exist to report on. Year 1 uses its whole limit, so any year-1 overrun must be met from the year-1 contingency or by moving work later.

The critical path

The **critical path method** schedules a project as a network of activities and finds the longest chain of dependent activities, which determines the earliest completion date [48]. Activities on that chain have no **slack**: any delay to them delays the whole project.

For example, consider the customer identity pilot, with durations in weeks and finish-to-start dependencies:

- A: agree identity semantics, 4 weeks.
- B: implement shared identifiers, 6 weeks, after A.
- C: define integration contracts, 3 weeks, after A.
- D: pilot a customer handoff, 4 weeks, after both B and C.
- E: assess results and decide on rollout, 2 weeks, after D.

Figure 14 shows the network.

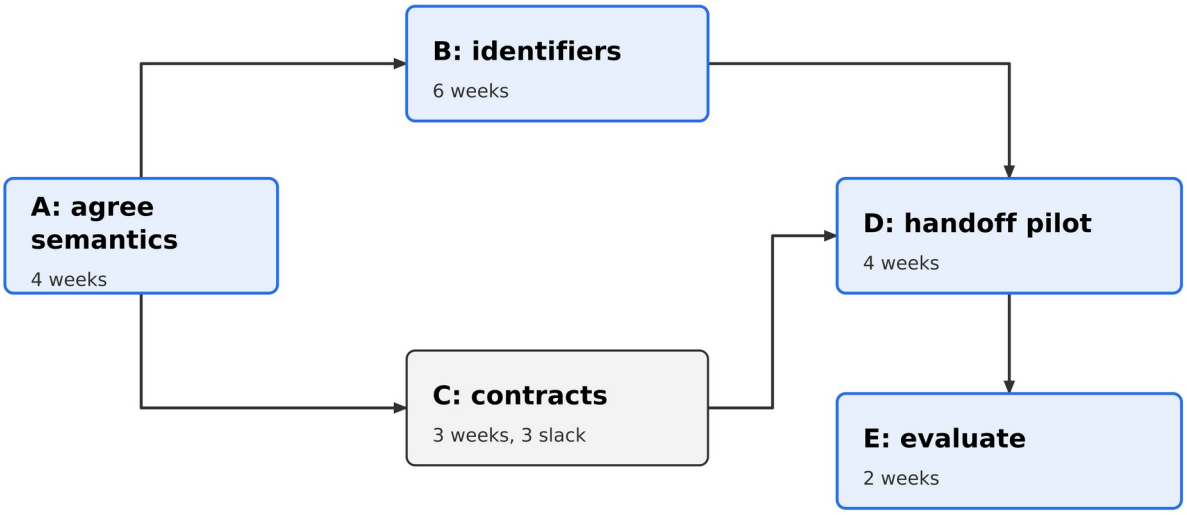


Figure 14 Dependency network for the customer identity pilot. The highlighted longest path, A–B–D–E, sets the earliest completion at 16 weeks.

A **forward pass** gives each activity's earliest finish: A at week 4, B at 10, C at 7, D at 14 (it must wait for B) and E at 16. A **backward pass** from week 16 gives the latest finish that does not delay completion: E at 16, D at 14, B at 10, C at 10 and A at 4. Slack is the difference. C can finish as late as week 10, so it has three weeks of slack; every other activity has none. The critical path is A–B–D–E.

A listing for the two passes

The following program performs both passes. It assumes that the tasks are listed so that every predecessor appears before the tasks that depend on it.

```
"""Critical-path schedule for the Helios customer identity pilot (weeks)."""

TASKS = {  # name: (duration, predecessors), listed in dependency order
    'A': (4, []),          # agree identity semantics
    'B': (6, ['A']),       # implement shared identifiers
    'C': (3, ['A']),       # define integration contracts
    'D': (4, ['B', 'C']),  # pilot a customer handoff
    'E': (2, ['D']),       # assess results and decide rollout
}

def schedule(tasks: dict[str, tuple[int, list[str]]]) -> tuple[dict, dict, int]:
    early: dict[str, int] = {}                 # earliest finish (forward pass)
    for name, (dur, preds) in tasks.items():
        early[name] = max((early[p] for p in preds), default=0) + dur
    end = max(early.values())
    late: dict[str, int] = {}                  # latest finish (backward pass)
    for name in reversed(list(tasks)):
        succs = [s for s, (_, preds) in tasks.items() if name in preds]
        late[name] = min((late[s] - tasks[s][0] for s in succs), default=end)
    return early, late, end

early, late, end = schedule(TASKS)
for name, (dur, _) in TASKS.items():
    start, slack = early[name] - dur, late[name] - early[name]
    print(f'{name}: start {start:>2}  finish {early[name]:>2}  slack {slack}')
critical = [n for n in TASKS if late[n] == early[n]]
print('Completion:', end, 'weeks; critical path:', '-'.join(critical))

shared = dict(TASKS, C=(3, ['B']))             # one team must do B, then C
print('With one shared team for B and C:', schedule(shared)[2], 'weeks')
```

Running it prints:

```
A: start  0  finish  4  slack 0
B: start  4  finish 10  slack 0
C: start  4  finish  7  slack 3
D: start 10  finish 14  slack 0
E: start 14  finish 16  slack 0
Completion: 16 weeks; critical path: A-B-D-E
With one shared team for B and C: 19 weeks
```

The last line shows why a precedence schedule is a lower bound rather than a promise. If one specialist team must build the identifiers and then define the contracts, C cannot start until week 10. D then starts at week 13 and the pilot completes at week 19. C's three weeks of slack existed only because the calculation assumed unlimited people. Real roadmaps must check resource constraints, and the people most often shared are the scarce experts in identity, data and integration.

Design each intermediate operating state

For the move from separate identifiers to governed group identifiers, Helios defines three states:

	State 1: today	State 2: pilot	State 3: adopted
Authority for customer facts	Each division's CRM	Each division's CRM	Party register for legal facts; CRMs for local facts
Cross-division links	Spreadsheets, informal	Link hub publishes verified links	Link hub, with correction service
Who may write links	Nobody formally	Party stewards	Party stewards and the correction service
Exit condition	Pilot approved	Pilot results meet the agreed thresholds	Spreadsheets retired, all consumers migrated

For each state, also name the allowed writers for every fact, the supported integrations, the access controls, the reconciliation rules and the operational owner. A temporary bridge needs a retirement condition and funded removal work; otherwise the temporary state quietly becomes the permanent architecture.

Retire systems on evidence rather than on a calendar date. Before switching something off, show that consumers have migrated, retained records remain accessible under policy, interfaces are no longer called and the contract can be ended. For example, a system can have no interactive users and still supply a critical monthly extract.

Respond to a 20% funding cut

Change card A reduces the envelope to €7.2 million, a cut of €1.8 million. Cutting every package by 20% would be easy arithmetic and poor architecture. Some packages contain work that cannot be divided, such as a security control or a dependency that other packages need. Cutting assurance while keeping every delivery promise increases risk without reducing scope.

One defensible scenario defers the €0.9 million analytics improvement and €0.9 million of legacy modernisation that has no fixed hosting deadline, provided that continued operation of those systems is acceptable. Legacy Orders, which does have a deadline, stays in scope.

The cut must also be applied to the annual limits, which still add up to €9.0 million. The deferred work was planned for years 2 and 3, so the revised schedule, in € millions, looks like this:

Work package	Year 1	Year 2	Year 3	Revised total	Initial total
Customer identity and data foundations	0.6	0.4	0.0	1.0	1.0
Workforce identity and access	0.4	0.1	0.0	0.5	0.5
Integration capability	0.5	0.5	0.2	1.2	1.2
Legacy modernisation	0.6	0.6	0.3	1.5	2.4
Analytics improvement	0.0	0.0	0.0	0.0	0.9
CRM and process pilot	0.3	0.6	0.3	1.2	1.2
Adoption and assurance	0.3	0.3	0.3	0.9	0.9
Contingency, held in reserve	0.3	0.3	0.3	0.9	0.9
Annual total	**3.0**	**2.8**	**1.4**	**7.2**	**9.0**

The architect should therefore ask the board to revise the annual limits to €3.0 million, €2.8 million and €1.4 million. A proportional cut of 20% to every year (€2.4 million, €2.8 million and €2.0 million) looks fairer but is infeasible for this plan: year 1 holds the identity foundations and the Legacy Orders deadline, and needs its full €3.0 million. If the board insists on proportional limits, €0.6 million of year-1 work must move later, which in practice means starting the CRM pilot a year late or running year 1 without contingency. Exercise 17.1 works through that choice.

The revision keeps the contingency, the pilot, the foundations and the assurance work. It also keeps the €2.5 million customer-handoff investment from Chapter 10 intact, so that investment's NPV is unchanged. The cost is real: group reporting stays inconsistent for longer, and some legacy systems keep running. If a deferred legacy system turns out to be unsafe to keep, the board needs another scope reduction or an explicit funding decision.

A reduced programme must make fewer promises or change how it keeps them. Recalculate benefits, contracts, annual cash needs and dependencies, and record the rejected alternatives with the reasons for choosing the revised sequence.

Exercises

11.1 Calculate the earliest completion time and C's slack for the customer identity pilot by hand. Explain how one shared implementation team changes the answer.

11.2 Specify the authority for each customer fact, the allowed writers and the retirement evidence for State 2, the pilot state.

11.3 Produce a different €7.2 million allocation from the one in the chapter, by package and by year, that stays within annual limits of €3.0 million, €2.8 million and €1.4 million. State one dependency and one risk that make a uniform 20% cut a poor choice.

11.4 Add task F, "train support staff", lasting 3 weeks after C, and make D depend on F as well as on B and C. Calculate the new completion time, the slack of each task and the critical path or paths. What does the result tell a programme manager?

11.5 A draft roadmap contains the single line "Replace CRM North with CRM Central in year two". Critique it and rewrite it as a set of roadmap entries that meet the standard set in this chapter.

Key points

- A roadmap is a sequence of operable intermediate states, each with named authorities, owners and exit conditions.
- The critical path sets the earliest completion; slack shows where delay is harmless. Shared scarce resources can remove slack and lengthen the schedule.
- A programme allocation must satisfy work-package totals and annual limits at the same time, with contingency inside the envelope.
- Funding cuts should remove scope deliberately, protect the dependencies that other work needs, and state openly which promises change.

CHAPTER 12

Organisational change, adoption and architecture leadership

Learning objectives

After this chapter you should be able to:

- Explain how responsibilities and incentives determine whether an architecture is adopted.
- Compare the Lewin, Kotter and ADKAR change models and critique the evidence behind them.
- Design an adoption plan with observable stages and measures.
- Facilitate a decision among stakeholders whose objectives differ for legitimate reasons.

Deployment is only one transition

A shared customer identifier is useful only if people and applications use it correctly. Local staff must know how to create, correct and dispute links. Support teams must know who resolves an ambiguous match. Reports must say which definition they use. Each of these changes in daily work needs an owner, practice and time.

Map the affected work before planning any training. For each role, record the current task, the proposed task, the decision authority, the information needed and the route for exceptions. If a new process adds three approval steps to a service agent's day, reluctance may reveal a poor process rather than a lack of commitment. Treat what people actually do as information about the design.

Helios's units also have different customer relationships. Industrial Maintenance works with asset owners and service sites, Retail Distribution with buyers and delivery addresses, and Digital Services with subscription administrators and end users. A single field labelled "customer" can hide several roles, and adoption planning must keep the distinctions that make local work valid.

Three models of change

Architects borrow three models of organisational change more than any others. Each is useful for a different purpose, and each has a weaker evidence base than its popularity suggests.

Lewin: unfreeze, change, refreeze

Kurt Lewin described planned change in groups as moving from one stable level of behaviour to another: unfreezing the present level, moving to a new level, and freezing group life at the new level [49]. The idea is that behaviour is held in place by social forces, so change needs those forces to be loosened first and re-established afterwards.

Cummings, Bridgman and Brown show that the familiar "three-step model" was assembled after Lewin's death by later writers and textbooks; comparing those accounts with what Lewin actually wrote, they conclude that he never developed such a model [50]. The model's lasting lesson is about the old way of working. If the shared spreadsheets that staff used for cross-division matching remain available, work drifts back to them. Retiring the old route is part of the change.

Kotter's eight steps

John Kotter's 1995 article analysed why transformation efforts fail and turned the common errors into eight steps [51]:

1. Establish a sense of urgency.
2. Form a powerful guiding coalition.
3. Create a vision.
4. Communicate the vision.
5. Empower others to act on the vision.
6. Plan for and create short-term wins.
7. Consolidate improvements and produce still more change.
8. Institutionalise new approaches.

A review by Appelbaum and colleagues found research support for most individual steps but no formal study of the model as a whole, and concluded that its popularity comes more from its direct, usable format

than from scientific consensus [52]. The model is also linear and top-down, whereas architecture change often proceeds in parallel streams and is reshaped by what local teams discover.

ADKAR

Jeffrey Hiatt's ADKAR model looks at change from the individual's point of view [53]. It holds that a person adopts a change by achieving five outcomes in order: **Awareness** of why the change is needed, **Desire** to take part, **Knowledge** of how to change, **Ability** to do it in practice, and **Reinforcement** that sustains it. The table applies the elements to a Helios service agent who is asked to use shared customer identifiers.

Element	What it means for the agent	Signal that it is missing	Typical response
Awareness	Knows why shared identifiers are being introduced	"Nobody told us why we have to do this"	Show the misrouting problem with the agent's own cases
Desire	Wants to take part	Keeps the old spreadsheet despite knowing the reason	Address what the agent loses, such as time or control
Knowledge	Knows how to search, verify and dispute a link	Errors in training exercises	Training, guides and worked examples
Ability	Can do it in real work, at normal speed, with the real screens	Correct in training, wrong under time pressure	Coaching, practice and design changes
Reinforcement	Keeps doing it after the project team leaves	Drift back to the old route	Measures, recognition and retiring the old route

The model is a diagnostic. Hiatt suggests assessing each element and working on the first one that is weak, which he calls the barrier point; effort spent on later elements is wasted while an earlier one is missing. The distinction architects most often need is between Knowledge and Ability. People can know exactly what to do and still be unable to do it, because the screen, the data or the time available does not allow it. The model comes from a change-management consultancy and rests mainly on practitioner experience, so use it to structure questions rather than as a tested theory.

How architects use the models

Model and level	Architectural use	Main limitation
Lewin: the group	Plan the removal of old routes so that work does not drift back	Implies a stable end state; the "model" is a later simplification
Kotter: the organisation	Sponsorship, a cross-unit coalition, pilot milestones as short-term wins	Linear and top-down; limited evidence for the whole sequence
ADKAR: the individual	Diagnose why a particular role has not adopted the change	Can treat structural problems as personal ones

For example, a Helios service agent may know the new matching procedure (Knowledge) but be unable to choose the right service site because the screen does not show sites at all (Ability). ADKAR locates the gap; the fix is a design change rather than more training. That is the general lesson: use the models to ask better questions, and expect many answers to be about the architecture.

Incentives can oppose the enterprise objective

Suppose the central platform team is measured on the number of applications migrated, while business units are measured on uninterrupted service and local delivery speed. The platform team pushes early migration; local managers delay it. Both responses are rational under their incentives. An enterprise decision needs a shared outcome and an agreed way to bear the cost of transition.

Make participation concrete. Allocate time for local specialists, include them in acceptance criteria, and fund the support responsibility after go-live. A programme that depends on unpaid local effort has hidden costs and an unreliable plan. Record disagreements about priorities as disagreements, rather than disguising them as technical incompatibility.

Plan adoption as an observable progression

Plan adoption in four stages, each with its own evidence:

1. **Readiness.** Support staff are trained and pilot data is reconciled.
2. **Supervised use.** Real work runs on the new route with the project team close by, revealing confusing tasks and exceptions.
3. **Independent use.** Normal work continues without the project team.
4. **Sustained operation.** Performance holds over an agreed period and recurring problems have owners.

Choose measures that follow the mechanism. Training attendance is an **input**. Correct use of the identifier in eligible workflows is a **behaviour**. Fewer misrouted service requests is an **outcome**. Staff can log in often and still keep a shadow spreadsheet, so observe the work and find out why parallel processes persist.

Compliance and understanding are different things. A mandatory field can force users to enter data while lowering its quality. Make the correct action easy and the exception visible. When staff cannot resolve a customer match, a governed “pending” state is more accurate than a forced false match.

Facilitate a consequential disagreement

Prepare a decision brief with the problem, mandatory conditions, options, supporting data and unresolved questions. During the meeting, separate facts from preferences, and authority from expertise. An experienced engineer may understand a failure mode; a business owner may be accountable for accepting its consequences. Both contributions matter, but they are different roles.

In a disagreement over group identifiers, for example, ask each stakeholder which consequence they are trying to prevent. Maintenance may fear incorrect entitlement, analytics may need consistent grouping, and the platform team may need a stable contract. A design that separates legal party, service site and reporting group may resolve all three concerns better than a debate about which team owns the word “customer”.

Record the decision, any material dissent, the conditions, the owner and the review date. A clear record lets someone who missed the meeting understand what is authorised, and it stops the same question being reopened without new information.

Research: adoption unfolds across levels

Dang and Pekkola studied enterprise architecture adoption in a public-sector organisation, using a qualitative case study and a process view of organisational change [54]. They found that the change was both sociotechnical and punctuated: it moved in bursts rather than smoothly, and it oscillated between different organisational levels as direction from the top met implementation below. A single case explains a process in its setting, which is useful for asking questions and weak for predicting a rollout sequence or EA’s benefit. For Helios it suggests a question: when local operating experience contradicts the authorised target, who can revise the target, and on what basis?

Worked adoption episode

The board approves a shared identifier, but the first Maintenance pilot reveals that staff choose the legal customer when the task needs a service site. The acceptance test checked that an identifier was present but never checked that the selected entity could legitimately receive the service. Staff start a spreadsheet to keep track of sites. The spreadsheet is a signal to investigate rather than proof of resistance.

Figure 15 shows two routes for the response. Local support can clarify labels and improve training within the current contract. A missing service-site concept, however, changes shared meaning and must go to the data owner and the consumer teams. The revised contract then returns through a controlled release and another operating trial. Training cannot repair a contract that cannot express the required fact, and a contract change cannot ensure that staff know which fact to enter.

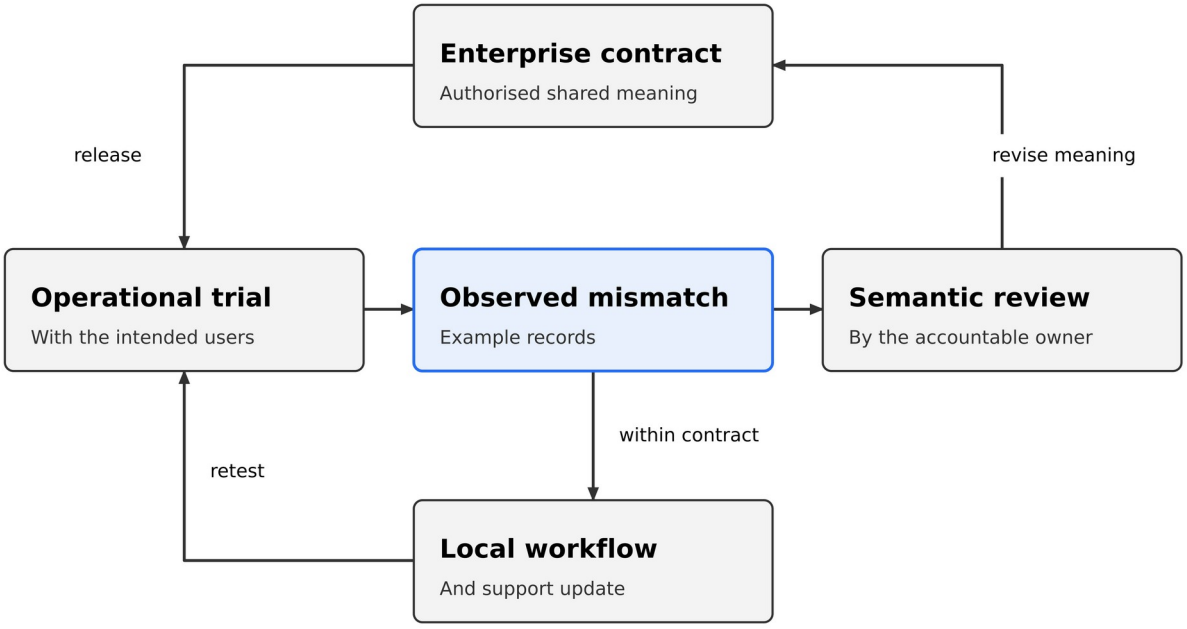


Figure 15 Operating experience can revise a local workflow, the enterprise contract, or both, before rollout expands.

Response	Rationale to test	What must be true before expansion
Enforce the current form	The contract already expresses sites; staff need better guidance	Staff resolve representative ambiguous cases without project help
Revise the shared contract	Site and party distinctions are missing or ambiguous	Agreed meanings, reconciled example records, consumer tests, authorised migration
Narrow the rollout	The current model fits Retail but not Maintenance	A clear eligibility boundary, a funded interim workflow, a dated decision for Maintenance

Helios chooses to revise the contract and narrow the rollout. Retail continues within its demonstrated scope, and Maintenance is not reported as migrated. A business data owner resolves the meanings, a product team implements the change, and an operating steward handles ambiguous records; the sponsor funds their time and accepts the revised schedule.

Collect observations that could overturn the choice. If representative records show that the contract already expresses site relationships correctly, the problem lies in the form or the training. If correct site selection still produces invalid entitlement, examine the entitlement service. If only unusually skilled pilot users succeed, repeat the trial with the intended operating team and normal support.

Leadership includes changing a recommendation

An architect should say so when new information weakens an earlier position. If the pilot shows that consolidated workflows slow urgent service work, revise the scope or the design, keep what was learned, and identify the next question that would distinguish between the options. Do not expand a programme simply because a pilot was popular; check whether its users, data, dependencies and support conditions represent the next rollout group.

Exercises

12.1 Build an adoption chain for group customer identifiers that separates an input, a behaviour and a business outcome, and name one measure for each.

12.2 Identify the conflict between a migration-count target and uninterrupted local service. Propose a shared acceptance condition that both teams could accept.

12.3 A pilot succeeds only because the project team resolves every exception by hand. Explain what must change before wider rollout.

12.4 Use ADKAR to diagnose why a Maintenance service agent keeps using the site spreadsheet. Identify the missing element under two different explanations and the action each implies.

12.5 Compare Lewin's and Kotter's models as guides for retiring Helios's shared spreadsheets. Critique the evidence behind each and say what an architect should add.

12.6 In the Maintenance pilot, compare enforcement, contract revision and narrowed rollout. Identify a finding that would reverse your preference, and explain why the public-sector case study cannot settle the choice for Helios.

Key points

- Architecture changes people's work. Adoption needs usable processes, aligned incentives, funded support and evidence that the new way works without the project team.
- Lewin, Kotter and ADKAR operate at group, organisation and individual levels. They are useful prompts with limited empirical support, and many of the gaps they reveal are design problems.
- Plan adoption in observable stages, and measure inputs, behaviours and outcomes separately.
- Facilitation separates facts from preferences and authority from expertise, and records decisions and dissent.
- Good leadership revises a recommendation when the operating experience contradicts it.

CHAPTER 13

Sourcing, procurement and supplier ecosystems

Learning objectives

After this chapter you should be able to:

- Compare build, buy and hybrid sourcing choices over the whole lifecycle of a responsibility.
- Apply mandatory conditions before weighted scoring, and test how sensitive a ranking is to its weights.
- Translate architectural assumptions into questions for a contract.
- Design an exit test that checks meaning and relationships as well as file transfer.

Source a responsibility rather than a feature list

Start with the business responsibility to be supported. A customer platform may need account relationships, customer identity resolution, staff sign-in, service history, data export, recovery and operational support. A feature checklist that leaves out data authority or exit can select a product that shines in a demonstration and then fits the enterprise poorly.

Build, buy and hybrid are families of choices:

- **Build** gives control over selected design decisions but needs lasting engineering and operating capacity.
- **Buy** transfers some implementation work to a supplier but keeps configuration, integration, governance and supplier management with the buyer.
- **Hybrid** buys a core service and builds the interfaces or domain behaviour that make Helios's work distinctive. For example, Helios could buy a case-management product and build its own customer-matching rules.

Research offers two useful lenses. Transaction cost economics asks which transactions an organisation should govern through markets and which it should govern internally, matching each kind of transaction to a suitable governance structure [55]. In IT terms, the more a service depends on knowledge specific to Helios, such as its customer-matching rules, the more costly and risky it is to buy it through a simple market contract. A review of IT outsourcing research by Lacity, Khan and Willcocks drew lessons from 191 studies, covering why organisations outsource, how they manage risk, and the client capabilities and relationship practices associated with success [56]. A consistent message is that outsourcing changes the buyer's work rather than removing it: the client still needs strong capabilities in contracting, integration and supplier management.

Compare the whole lifecycle

Compare total cost of ownership over the whole lifecycle (Chapter 10). Include discovery, licences, implementation, migration, integration, operation, support, training, upgrades, data retention and exit. For each cost, record who pays it and which assumptions it depends on: transaction volumes, contract renewal terms, exchange rates or organisational change. A subscription with a low first-year cost can still cost more than a build over the whole lifecycle.

Dependence is also a cost. **Vendor lock-in** means being unable to move away from a supplier without substantial cost. A survey of cloud adopters by Opara-Martins, Sahandi and Tian found that the problem grows as organisations move more of their computing to the cloud. The authors recommended attention to contracts, standard data formats and interfaces, and awareness of dependencies from the start [57]. Figure 16 shows the procurement sequence that keeps these concerns in view.

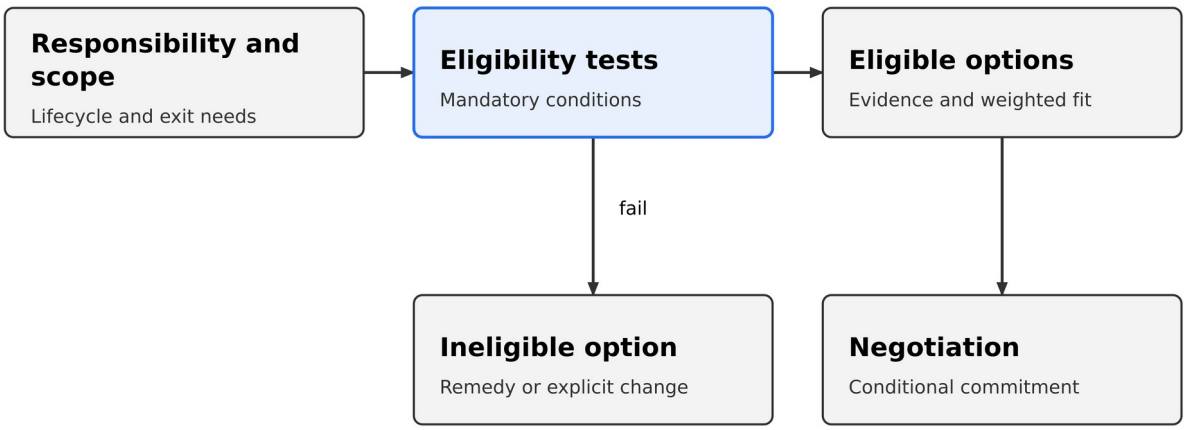


Figure 16 Procurement tests eligibility first, then compares value, then negotiates the conditions that keep the commitment safe.

Apply mandatory conditions before weights

Helios evaluates three suppliers for a customer workflow service. A tested export that keeps the required identifiers and relationships is **mandatory**. Supplier C cannot demonstrate it, so C is ineligible under the current requirement. However low its price, C cannot win by trading its price against the missing export, because a mandatory condition is not a weighted criterion.

For the eligible suppliers A and B, the panel scores three criteria from 1 to 5, where higher is better:

Criterion	Weight	A score	B score
Functional fit	0.40	4	3
Operability	0.35	3	5
Cost suitability	0.25	4	3

The weighted totals are 0.40 × 4 + 0.35 × 3 + 0.25 × 4 = 3.65 for A and 0.40 × 3 + 0.35 × 5 + 0.25 × 3 = 3.70 for B. A difference of 0.05 on a five-point scale of judgements is fragile. If the weights change to 0.50, 0.25 and 0.25, A scores 3.75 and B scores 3.50, and the ranking reverses.

Finding the tipping point

A's scores differ from B's by +1, –2 and +1 on the three criteria. Since the weights sum to 1, the difference in totals is (1 – w) – 2w = 1 – 3w, where w is the operability weight, however the remaining weight is split between the other two criteria. The ranking therefore tips at w = 1/3: below it A leads, above it B leads. The following program applies the mandatory condition and then checks three operability weights.

```
"""Supplier comparison: apply the mandatory condition first, then weighted scores."""

SUPPLIERS = {  # name: (passes the export test, scores for fit, operability, cost)
    'A': (True, (4, 3, 4)),
    'B': (True, (3, 5, 3)),
    'C': (False, (5, 4, 5)),
}

def ranking(weights: tuple[float, float, float]) -> list[tuple[str, float]]:
    assert abs(sum(weights) - 1) < 1e-9, 'weights must sum to 1'
    totals = {name: sum(w * s for w, s in zip(weights, scores))
              for name, (eligible, scores) in SUPPLIERS.items() if eligible}
    return sorted(totals.items(), key=lambda item: -item[1])

for weights in [(0.40, 0.35, 0.25), (0.50, 0.25, 0.25)]:
    print(weights, [(name, round(total, 2)) for name, total in ranking(weights)])

for w in [0.30, 1 / 3, 0.36]:          # vary the operability weight
    rest = 1 - w
    result = ranking((rest * 0.6, w, rest * 0.4))
    print(f'operability {w:.3f}:', [(name, round(total, 3)) for name, total in result])
```

Running it prints:

```
(0.4, 0.35, 0.25) [('B', 3.7), ('A', 3.65)]
(0.5, 0.25, 0.25) [('A', 3.75), ('B', 3.5)]
operability 0.300: [('A', 3.7), ('B', 3.6)]
operability 0.333: [('A', 3.667), ('B', 3.667)]
operability 0.360: [('B', 3.72), ('A', 3.64)]
```

Supplier C never appears, whatever the weights. The decision between A and B rests on one question: how much does operability matter, relative to everything else? That question deserves direct evidence. A realistic recovery and support exercise with both suppliers is worth more than adding twenty minor criteria to the matrix. Record the basis for each score, and keep unresolved assumptions as conditions on the award rather than rounding them into a confident total.

Contract and architecture must describe the same service

Turn each important architectural assumption into a question for procurement and legal specialists:

- Which data can be exported, in what structure, how quickly and at what cost?
- What happens to the data and the service after termination?
- Which interfaces are supported, and how are breaking changes announced?
- Who may approve subcontractors or changes to how data is processed?
- What evidence supports the supplier's recovery claims?

The architect does not draft jurisdiction-specific clauses. The architect's contribution is to state the technical and operational consequence the contract must address. A legal right to export is of little use if the export cannot be loaded and reconciled in the receiving system; a technical export is of little use if Helios lacks the right to use the data after exit.

Service credits deserve the same scrutiny. A credit may compensate for an outage under agreed terms without restoring the business service, so assess detection, response, restoration and data correction separately. The buyer's own responsibilities matter too. Configuration, identity, application design and backup choices often determine the service that users actually experience.

Test exit before dependence becomes deep

Define an exit test early, while dependence is still shallow. For example, Helios's test for the customer workflow service is:

1. Export a representative dataset: 5,000 customer organisations with their sites, contracts and two years of case history.
2. Load it into a separate test environment.
3. Reconcile record counts, keys, missing values and relationships against the source.
4. Rebuild one required report and compare it with the version produced by the current system.
5. Run three real business journeys, such as opening a case for an existing site.
6. Record the effort, the elapsed time, unsupported fields and any rights needed to keep using the data.

A small, real export reveals dependence that a general statement about portability hides. Repeat the test when the supplier releases major changes.

Look at concentration across suppliers as well. Several apparently separate products may depend on the same identity provider, cloud region or implementation partner. Operational independence has to be checked dependency by dependency, whatever the supplier list suggests. Deliberately shared services can also simplify control, provided the shared dependency is recognised and managed.

Exercises

13.1 Recalculate the two weighted comparisons by hand, and explain why neither establishes that one supplier is better in general.

13.2 Supplier C is the cheapest but fails the mandatory export condition. Write the evaluation decision and state the evidence that would allow C to be reconsidered.

13.3 Using the tipping-point formula, find the operability weight at which A and B would tie if B's operability score were 4 instead of 5. Explain the result.

13.4 Design an exit test for Helios's workforce identity platform that checks meaning and relationships rather than only the success of a file download.

13.5 A divisional manager proposes outsourcing customer matching to a data-services firm because “it is not our core business”. Evaluate the proposal using the transaction-cost and outsourcing-research perspectives in this chapter.

Key points

- Source a business responsibility over its whole lifecycle. Buying changes the buyer’s work rather than removing it.
- Apply mandatory conditions before weighted scoring. A low price cannot compensate for a failed mandatory condition.
- Weighted scores are judgements. Find the weight at which the ranking tips, and gather direct evidence about that criterion.
- The contract and the architecture must describe the same service, including export, exit, change and recovery.
- Test exit early with real data, and look for hidden concentration across suppliers.

CHAPTER 14

Measuring enterprise outcomes and architecture value

Learning objectives

After this chapter you should be able to:

- Design a measurement contract that links an architectural change to a business outcome.
- Recognise Simpson's paradox and regression to the mean in enterprise data, and calculate a standardised rate.
- Calculate a difference-in-differences estimate and state the assumptions needed to read it causally.
- Evaluate research on the value of enterprise architecture and apply it to a disputed benefit claim.

Measure the consequence the programme promised

Helios's board will ask what the programme has achieved, and this chapter shows how to answer with evidence that survives challenge. Architecture activity is easier to count than enterprise value. Diagrams drawn, standards published, review meetings held and applications migrated describe work done; better service, lower cost and reduced risk have to be measured separately. Keep activity measures for managing the work, and connect the programme to the outcomes stakeholders care about.

For Helios, a useful chain is: verified customer links enable accurate routing; accurate routing reduces reassignment; less reassignment shortens the time to resolve a service request. Each link can fail. More reliable links will change little if technician availability is the real constraint.

Write a **measurement contract** for each important measure:

- name, purpose and owner;
- unit of observation and population;
- numerator, denominator and exclusions;
- when and where the observation is taken, and over what time window;
- known limitations and the version of the definition.

If the denominator changes, a trend may reflect reclassification rather than improvement. Pair measures to guard against distortion: faster case handling with rework, lower cloud cost with demand and reliability, retirement counts with remaining interfaces. Campbell warned that the more a quantitative indicator is used for decisions, the more it is exposed to pressures that distort both the indicator and the process it is meant to monitor [58]. Each pair follows the mechanism by which a misleading success could appear.

Simpson's paradox

Aggregated rates can point the opposite way from every group within them. Simpson showed in 1951 that conclusions drawn from a combined table can differ from those drawn from its parts [59]; the effect is now called **Simpson's paradox**.

For example, before a change Unit A handles 100 requests with 10 failures and Unit B handles 900 with 45 failures. Their failure rates are 10% and 5%, and the pooled rate is 55 / 1,000 = 5.5%. After the change, A handles 900 requests with 72 failures and B handles 100 with 4. Both units improve: A falls to 8% and B to 4%. Yet the pooled rate rises to 76 / 1,000 = 7.6%, because the workload has shifted towards the unit with the higher failure rate.

A **standardised rate** answers a like-for-like question: what would the later unit rates imply under the earlier workload mix of 10% A and 90% B? The answer is 0.1 × 8% + 0.9 × 4% = 4.4%. The following program reproduces all of these figures, using exact fractions so that rounding cannot creep in, and also computes the difference-in-differences used later in the chapter.

```
"""Pooled, stratified and standardised error rates, and a difference-in-differences."""
from fractions import Fraction

BEFORE = {'A': (10, 100), 'B': (45, 900)}      # unit: (errors, requests)
AFTER = {'A': (72, 900), 'B': (4, 100)}

def pct(x: Fraction) -> str:
    return f'{float(x) * 100:.1f}%'

def rates(data: dict[str, tuple[int, int]]) -> dict[str, Fraction]:
    return {unit: Fraction(errors, n) for unit, (errors, n) in data.items()}

def pooled(data: dict[str, tuple[int, int]]) -> Fraction:
    return Fraction(sum(e for e, _ in data.values()), sum(n for _, n in data.values()))

for label, data in [('Before', BEFORE), ('After', AFTER)]:
    by_unit = ', '.join(f'{unit} {pct(r)}' for unit, r in rates(data).items())
    print(f'{label}: {by_unit}; pooled {pct(pooled(data))}')

total = sum(n for _, n in BEFORE.values())
mix = {unit: Fraction(n, total) for unit, (_, n) in BEFORE.items()}  # earlier mix
standardised = sum(mix[unit] * r for unit, r in rates(AFTER).items())
print('After, standardised to the earlier mix:', pct(standardised))

pilot, comparison = (10, 6), (8, 7)            # error rate (%) before and after
change_p, change_c = pilot[1] - pilot[0], comparison[1] - comparison[0]
print(f'Pilot {change_p} pp, comparison {change_c} pp, '
      f'difference-in-differences {change_p - change_c} pp')
```

Running it prints:

```
Before: A 10.0%, B 5.0%; pooled 5.5%
After: A 8.0%, B 4.0%; pooled 7.6%
After, standardised to the earlier mix: 4.4%
Pilot -4 pp, comparison -1 pp, difference-in-differences -3 pp
```

Report both figures. The pooled 7.6% is what the later population actually experienced. The standardised 4.4% isolates the change in unit performance from the change in mix. Neither explains why demand moved between units, or whether the architecture caused the unit improvements.

Regression to the mean

Regression to the mean occurs when a group is selected because of an extreme measurement: its next measurement tends to be closer to its long-run average, even if nothing changes [60]. It can make natural variation look like real change.

For example, Maintenance's reassignment rate has averaged 6.8% over the past eight quarters, ranging from 5.9% to 7.9%. Helios starts its pilot in Maintenance immediately after the worst of those quarters, the 7.9% quarter described in Chapter 1, because that is where the pain is most visible. The next quarter shows 6.6%. The apparent improvement is 1.3 percentage points, but some return towards the 6.8% average was expected anyway. If the 7.9% quarter were pure bad luck, full regression to the average would explain up to 1.1 of those points, so 1.1 is an upper bound. In practice regression is usually partial: part of a bad quarter reflects lasting causes, such as a hard case mix, that persist into the next quarter. The expected next value lies between the extreme and the average, closer to the average the less the quarterly rate is correlated from one quarter to the next. Choosing pilots because of a recent bad result is natural, so guard against this effect: use several quarters of baseline data, compare with units that were not selected, and be cautious about the first measurement after selection.

Attribution needs a credible comparison

A before-and-after improvement can coincide with staffing changes, seasonality, a different mix of customers or altered reporting. Record concurrent changes and preserve the baseline. Where possible, compare similar workflows or roll out in stages, and explain how the groups differ. Randomised rollout is often impractical; without it, causal confidence is lower, but the comparison still teaches something.

Difference-in-differences

In the participating division, reconciliation errors fall from 10% to 6% during the pilot. In a comparison division, they fall from 8% to 7% over the same period. The participating division changes by –4 percentage points and the comparison by –1, so the **difference-in-differences** is –3 percentage points. Subtracting the comparison's change removes influences that affected both divisions equally.

Figure 17 shows the logic. The dashed line is the path the participating division would have followed without the pilot, if it had moved in parallel with the comparison division.

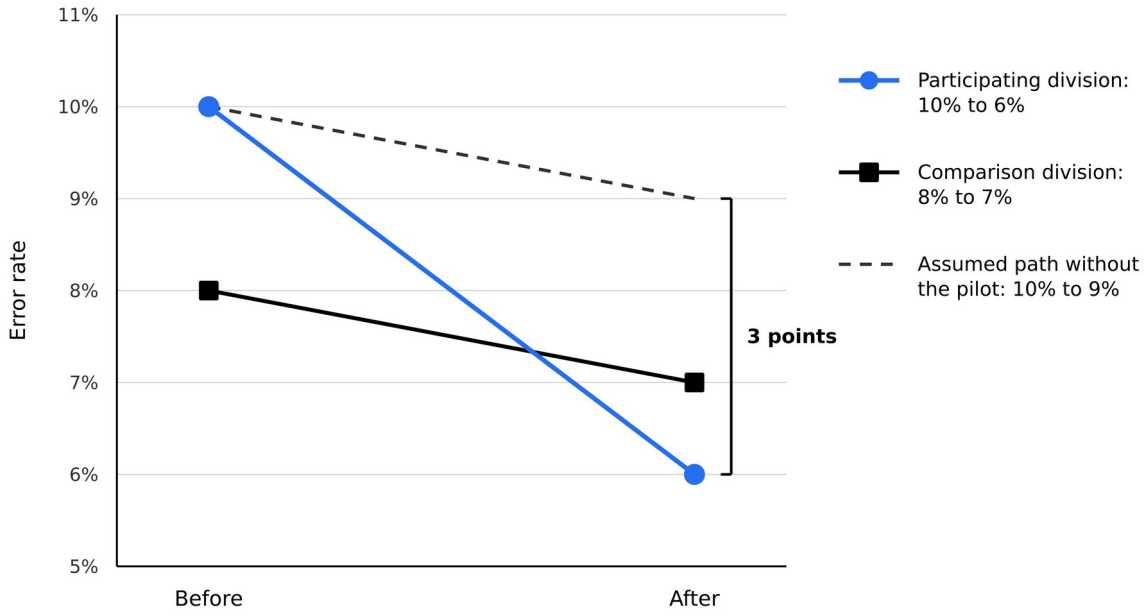


Figure 17 Difference-in-differences: the dashed line shows the participating division's assumed path without the pilot, parallel to the comparison division.

That assumption, called **parallel trends**, is the crux. The estimate is causal only if, without the pilot, the gap between the two divisions would have stayed constant [61]. Several things can break it:

- the participating division adds experienced staff at the same time;
- the comparison division also starts using the new customer-matching service;
- the pilot division was chosen after an unusually bad period, so regression to the mean inflates its improvement;
- the definition of an error changes in one division but not the other.

Check prior trends where data exists, but remember that similar past trends make the assumption more plausible without proving it.

The example also omits sample sizes. Ten errors in 100 cases and 1,000 errors in 10,000 cases are both 10%, but they carry very different amounts of information. Repeated records from the same customer or operator may not be independent. Before attaching an uncertainty estimate, define the unit of observation and how observations depend on each other.

If no credible comparison exists, report the descriptive change together with its competing explanations. That is still useful for a bounded, reversible decision.

What research says about the value of enterprise architecture

Three strands of research help Helios design its own evaluation.

- **A model of how value arises.** Tamm, Seddon, Shanks and Reynolds reviewed the literature and proposed that enterprise architecture creates benefits through four **benefit enablers**: organisational alignment, information availability, resource portfolio optimisation and resource complementarity [5]. Alignment can be improved by planning alone, but the two resource enablers depend on the plans actually being carried out. The model is conceptual, so it tells Helios which intermediate outcomes to measure rather than how large the benefits will be.
- **Survey evidence on success factors.** Lange, Mendling and Recker built a model of EA management success from the literature and interviews, and tested it with a survey of 133 practitioners [4]. Four

success factors were supported: the quality of EA products, the quality of the EA infrastructure, the quality of EA services and **organisational anchoring**, the extent to which EA is embedded in the organisation's decisions and processes. Anchoring was the focal concept: it mediated the effects of infrastructure quality and service quality on the two success measures, intention to use EA management and organisational and project benefits. Satisfaction with EA proved irrelevant for determining or measuring success. A related survey by van de Wetering and colleagues linked EA-driven dynamic capabilities to organisational benefits, partly through innovativeness [62]. Its 299 respondents were CIOs and IT managers, and all measures were self-reported at one point in time.

- **Studies of practice.** Kotusev's case studies, discussed in Chapter 2, show practitioners relying on specific artefacts used in planning and decision processes, rather than on framework procedures [17].

The strands agree on one practical point: value depends on whether architecture is used in real decisions. Most of the quantitative evidence, however, is cross-sectional and based on respondents' perceptions, so it shows association rather than effect. Helios therefore needs its own evidence: observed changes in decisions, operations and cash, measured with the methods in this chapter.

Worked example: a disputed benefit claim

Helios's sponsor claims that the programme has made the group "adaptable", because leaders report more confidence in changing systems. Confidence, the ability to change and the benefit realised from a change are three different things, and each needs its own observation:

1. **Confidence:** ask the relevant roles the same defined questions before and after the programme, recording role and response rates.
2. **Ability to change:** rehearse a real change, such as adding a service-site relationship, and record effort, dependencies and defects.
3. **Realised benefit:** observe the affected service population over a stated period, including exceptions and concurrent staffing changes.

Consider rival explanations for a positive survey. Better leadership may both sponsor the architecture work and improve service independently. Better-performing divisions may have more resources to invest in architecture, reversing the assumed direction of influence. Enthusiastic respondents may rate both capability and performance highly. Only different kinds of evidence can tell these accounts apart; more responses of the same kind cannot.

Decide in advance what each result will change. A positive survey with a failed rehearsal calls for work on capability before any announcement of savings. A successful rehearsal with no business improvement points to a constraint elsewhere. An invoice reduction after a licence is removed supports that particular cash saving, net of replacement costs, and nothing more general. A sound report might say: "The participating team completed the specified change with fewer cross-team handoffs; the available comparison does not isolate the programme's effect on service outcomes."

Technical measures keep their own scope

Software delivery measures help teams improve a particular service. The best known come from DORA, the DevOps Research and Assessment programme, which began as an independent research company and became part of Google Cloud in 2018. Its multi-year surveys, summarised by Forsgren, Humble and Kim, related software delivery performance to organisational performance [63]. DORA's current guide describes five measures in two groups [64]:

- **Throughput:** change lead time, deployment frequency and failed deployment recovery time.
- **Instability:** change fail rate and deployment rework rate.

The guide stresses context. It warns against comparing very different applications, such as a mobile app and a mainframe system, and against turning the measures into targets, which invites the distortion Campbell described. For Helios, a shorter change lead time for the customer lookup service supports the Digital Services objective when slow delivery was a real constraint, but it may change little if prioritisation, procurement or customer adoption dominates the elapsed time. Use such measures where they explain the relevant process rather than as a universal score for the architecture team.

The acronym has a second meaning in enterprise work. The EU's Digital Operational Resilience Act, Regulation (EU) 2022/2554, has applied to banks, insurers and other financial entities since January 2025, and sets requirements for ICT risk management, incident reporting, resilience testing and the management of ICT third-party providers [65]. It is unrelated to the delivery measures. An architect working for a financial group must treat it as a legal obligation, with the supplier and recovery analysis of Chapter 9 as part of the evidence.

For each measure, state who reviews it and what action it can trigger. A growing queue of unresolved customer links could lead to more stewardship capacity, better validation at source or a narrower rollout. A measure that stays red without ever changing a decision has become a ritual.

Exercises

14.1 Recalculate every rate in the Simpson's paradox example. Explain how both units improve while the pooled rate worsens, and say which figure you would show the board.

14.2 Write a measurement contract for "unresolved customer-identity links", and name one decision it can trigger.

14.3 Service resolution improves after the pilot while staffing also increases. State what can be concluded, and propose additional evidence to isolate the architectural contribution.

14.4 Calculate the difference-in-differences for a pilot division whose reconciliation error rate moves from 12% to 8% and a comparison division that moves from 9% to 8%. Give three conditions that would undermine a causal reading, including one based on regression to the mean.

14.5 A sponsor treats favourable leadership survey responses as proof of both adaptability and cash savings. Design one observation for each claim, give two rival explanations, and write a conclusion that the survey alone could support.

14.6 Using the research on EA value in this chapter, critique the claim: "Studies show that enterprise architecture improves organisational performance, so Helios's programme will pay for itself."

Key points

- Measure the outcome the programme promised, with a written contract for each measure and paired measures to resist distortion.
- Aggregated rates can reverse the pattern within groups (Simpson's paradox). Report actual and standardised figures.
- Units chosen because of an extreme result tend to improve anyway (regression to the mean). Use longer baselines and comparison groups.
- Difference-in-differences removes shared influences but relies on the parallel-trends assumption.
- Research suggests EA creates value when it is embedded in decisions, but most of the evidence is perceptual and cross-sectional. Each organisation must measure its own effects.

CHAPTER 15

Innovation, AI and sustainable technology decisions

Learning objectives

After this chapter you should be able to:

- Explain the difference between exploration and exploitation, and apply it to a portfolio of experiments.
- Design a bounded AI experiment with a baseline, consequential error categories and stop conditions.
- Calculate a carbon-intensity estimate for a stated functional unit, and explain how demand changes the total.
- Evaluate when local results should change an enterprise standard.

Exploration competes with current delivery

March distinguished **exploration**, the search for new possibilities, from **exploitation**, the refinement of what an organisation already knows [66]. His models show why the two compete: exploitation pays back sooner and more reliably, so organisations tend to drive out exploration, even though their long-run adaptation depends on it. The models describe a mechanism, and each organisation still has to set its own innovation budget.

At Helios, improving the reliability of an existing integration is exploitation, and testing a new way to help technicians diagnose faults is exploration. The categories describe the purpose of the learning rather than the novelty of the technology. Introducing a familiar product into an unfamiliar business process can still be substantial exploration.

Manage exploration as a portfolio of **bounded experiments**, as shown in Figure 18. Each experiment states:

- the decision it will inform;
- its cost limit and duration;
- the data it will collect and the baseline it will be compared with;
- its stop conditions;
- the possible next actions: stop, revise or scale.

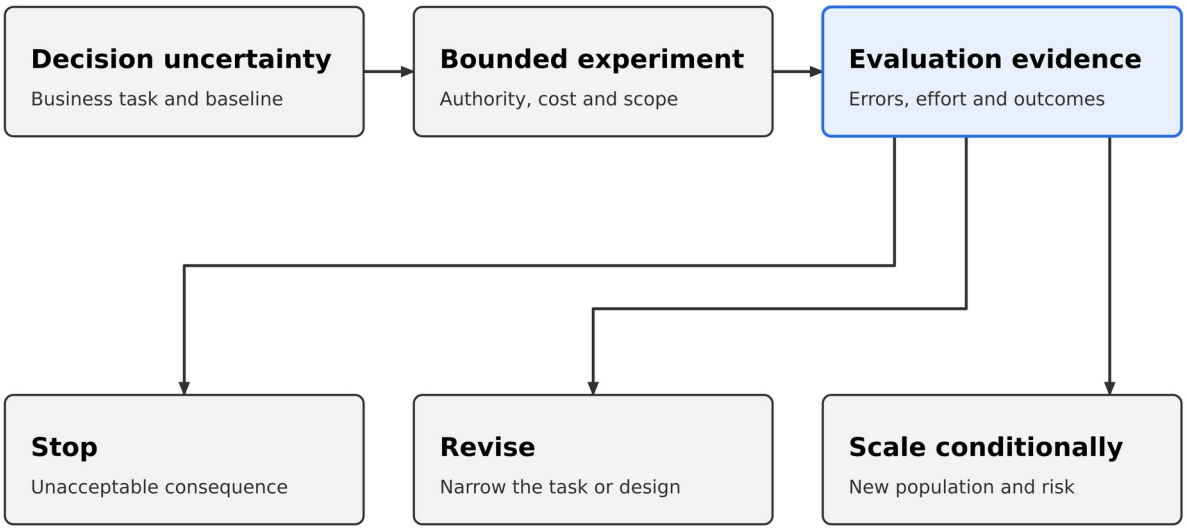


Figure 18 A bounded experiment connects its results to a decision to stop, revise or scale.

Protect time for learning, but make further investment depend on what is learned. A series of demonstrations with no decision record consumes money without reducing uncertainty.

An AI-assisted service case

Helios proposes an assistant that drafts a service summary from the maintenance records an agent is authorised to see. Start by fixing the task and the authority boundary. In the pilot, the assistant may suggest text. It may not change entitlement, dispatch technicians or send anything to a customer without a person's action.

Separate the stages of the work: retrieving records, generating text, validating it, human review and the final business action. Access checks must apply to the retrieved records and to the output. A fluent summary can still state something unsupported, omit a safety condition or mix records from two customers. Wrong retrievals and excessive permissions need fixes in the design; a disclaimer on the screen leaves them in place.

The NIST AI Risk Management Framework organises the work under four functions: Govern, Map, Measure and Manage [67]. For this case, **Govern** assigns accountability for the assistant; **Map** describes the context of use and who could be harmed; **Measure** evaluates the relevant errors; **Manage** decides how to respond, including how to withdraw the feature.

Evaluate against a baseline

Compare the assistant with the current workflow and, where useful, with a simpler template that fills in fields from the record. Use representative cases, including ambiguous and incomplete records, and keep the evaluation cases separate from any examples used to tune the system. Record the model version, prompt, retrieval settings and data version, so that the result can still be interpreted after something changes.

Define error categories by consequence:

- an unsupported claim;
- content about the wrong customer;
- an omitted safety-relevant condition;
- a misleading order of events;
- disclosure of information the agent may not see.

For example, in a 200-case evaluation, agents took a median of 9 minutes per summary with the assistant and 14 minutes without it, including the time to correct drafts. Three drafts mixed in content from another customer's records. That is 1.5% of cases, and it triggers the pre-agreed stop condition for wrong-customer content until retrieval is fixed.

Small evaluations also have a statistical limit. If a severe error never occurs in 200 cases, the approximate 95% upper bound on its true rate is 3 / 200 = 1.5%, so the evaluation cannot show that the rate is below 1 in 1,000. Rare, serious errors need controls that prevent them as well as tests that look for them. Review can fail too, when users over-trust fluent text or lack time to check it, so the operating design needs escalation, traceability and a way to withdraw the feature.

Sustainability needs a defined boundary

A claim that one design is greener than another needs a basis for comparison. The Green Software Foundation's **Software Carbon Intensity** (SCI) specification, published as ISO/IEC 21031, expresses emissions per **functional unit** R, such as one API call or one completed service request [68]. In its simplest form:

SCI = (E × I + M) per R

where E is the energy used, I is the carbon intensity of the electricity, and M is the share of the hardware's embodied emissions from manufacture.

For example, suppose one completed service request uses E = 0.02 kWh, the electricity intensity is I = 250 g CO2e per kWh, and the request's share of embodied hardware emissions is M = 1.5 g CO2e. The operational part is 0.02 × 250 = 5 g, so the SCI is 6.5 g CO2e per request. All three inputs are teaching assumptions; a real estimate would measure E and take I from the grid region.

Now suppose an optimisation halves the energy per request while demand triples. The answer depends on what happens to the hardware, so the assumption must be stated:

- **Hardware grows with demand.** If Helios adds servers in proportion to the extra requests, each request still carries 1.5 g of embodied emissions. The SCI falls to 0.01 × 250 + 1.5 = 4.0 g per request, a 38% improvement in intensity. Total emissions, however, rise from 6.5 g to 3 × 4.0 = 12.0 g for every request in the starting volume, an 85% increase.

- **Existing hardware absorbs the demand.** If the same servers handle three times the load, their embodied emissions are shared among three times as many requests, so M falls to 0.5 g. The SCI becomes 0.01 × 250 + 0.5 = 3.0 g, and the total rises to 3 × 3.0 = 9.0 g, a 38% increase.

In both cases operational energy rises by 0.5 × 3 = 1.5 times. Intensity and total impact answer different questions, so report both whenever demand can change, together with the hardware assumption. A lower emissions estimate also leaves the design's reliability and data-residency requirements in force.

Decide what to scale

Scale only what the evidence supports. A successful drafting pilot authorises drafting, and automatic diagnosis or dispatch would need evidence of its own. Wider use changes the data population, the consequence of an error, the permissions, the cost and the support needed. Treat each material change in scope as a new decision with proportionate evidence.

Learning versus standardisation

March's model of mutual learning describes individuals learning an organisation's code of practice while the code learns from individuals; if everyone converges on the code too quickly, the organisation stops learning [66]. The model is stylised, so its application to Helios is an inference to test.

Suppose the architecture group standardises one customer-matching method after a successful pilot. Reuse cuts repeated experimentation and simplifies support. Then an acquired business turns out to identify customers by service site and contract, unlike the pilot's legal-party view. Enforcing the old rule without investigation would create systematic mismatches. The organisation needs a way for contrary local results to change the standard, without turning every preference into an exception.

Compare two policies:

- A **strict reuse** policy permits only the approved method and measures compliance. It risks preserving a mistaken assumption.
- A **bounded learning** policy requires the approved method by default, but permits a time-limited experiment with a documented hypothesis, isolated data, explicit costs and a decision about whether to change the standard. It costs coordination effort and risks uncontrolled variation if the bounds are weak.

To tell useful variation from accidental divergence, measure false matches, missed matches and the downstream harm of each, on the same eligible cases for both methods. A single overall accuracy figure can hide a dangerous minority of false merges. If the alternative is better only for site-based contracts, the standard may need a conditional rule rather than replacement. Record the counterexample and its scope in the architecture record, so that the debate is not repeated after staff turnover.

Exercises

15.1 Design an experiment that could lead Helios to reject or narrow the service-summary assistant. Include a baseline, two consequential error categories and a stop condition.

15.2 Recalculate the SCI example. Then calculate the SCI and the change in total emissions if the energy per request halves, demand doubles and the electricity intensity falls to 200 g CO2e per kWh, under each of the two hardware assumptions. State what the boundary excludes.

15.3 Explain why a successful drafting pilot does not justify automatically dispatching technicians.

15.4 An evaluation of 300 cases finds no unauthorised disclosures. A manager concludes that the disclosure risk is "below 0.1%". Evaluate the conclusion and propose what else is needed.

15.5 Design a bounded experiment for the acquired business's customer matching. Include a stopping rule and the results that would change the enterprise standard.

Key points

- Exploration and exploitation compete for the same resources. Manage exploration as bounded experiments, each tied to a decision.
- AI pilots need a clear authority boundary, separated stages, consequence-based error categories and a baseline. Small samples cannot establish that rare, severe errors are rare.
- Sustainability claims need a functional unit and a boundary. Intensity can fall while total emissions rise.
- Standards should be able to learn: a bounded route for contrary local results keeps them from fossilising mistaken assumptions.

CHAPTER 16

Research methods and academic synthesis

Learning objectives

After this chapter you should be able to:

- Classify research designs and state what kind of claim each can support.
- Appraise a study's question, method, sample, results and limits.
- Synthesise several sources into a conditional recommendation that separates findings, inference and proposal.
- Write a critical synthesis with a source matrix and a counterargument.

Kinds of claim

Helios's sponsors, vendors and consultants all cite research. This chapter gives you a way to judge what each source can and cannot support before it shapes a decision.

Arguments about enterprise architecture mix several kinds of claim, and each needs different support:

- A **definition** says what a term includes. For example, "an operating model is the level of process integration and standardisation an organisation needs".
- A **mechanism** explains how one change could lead to another: verified customer links could reduce misrouted requests.
- An **association** reports that two things vary together: organisations reporting stronger EA capabilities also report more benefits.
- A **causal effect** states what would happen to an outcome if an intervention were made, compared with an alternative.
- A **design proposal** recommends what an organisation should do, under stated conditions.

Agreement on a definition does not establish a mechanism, and an association does not establish a causal effect. Much confusion in architecture debates comes from moving silently from one kind of claim to the next.

Research designs and what they support

The studies cited in this book use several designs. Each supports some kinds of claim well and others poorly.

Design	Example in this book	Good for	Weak for
Conceptual model from literature	Tamm et al., EA benefits (Chapter 14)	Proposing mechanisms to measure	Estimating effect sizes
Formal or simulation model	March, exploration and exploitation (Chapter 15)	Showing what follows from assumptions	Showing the assumptions hold
Comparative measurement	MacCormack et al., mirroring (Chapter 7)	Testing a relationship across matched cases	Generalising beyond the sample
Survey	Lange et al.; van de Wetering et al. (Chapter 14)	Associations among measured constructs	Causal direction; behaviour as opposed to perception
Qualitative case study	Jöhnk et al. (Chapter 8); Dang and Pekkola (Chapter 12); Kotusev (Chapter 2)	How and why processes unfold in context	Frequency and typicality
Quasi-experiment	Difference-in-differences (Chapter 14)	Estimating effects under stated assumptions	Settings where the assumptions fail
Standard or method	TOGAF, ArchiMate (Chapter 2)	Shared definitions and procedures	Showing that the procedure works

No design is best in general. A larger sample does not automatically defeat a smaller study if the two answer different questions.

Appraising a source

Read each source as an argument. A practical appraisal follows these steps:

1. **Question.** What exactly does the study ask?
2. **Constructs.** How are its key ideas defined and measured?
3. **Method.** Which design does it use, and why?
4. **Sample and selection.** Who or what was studied, and how were they chosen?
5. **Results.** What did it find, in its own terms?
6. **Limits.** What do the authors say limits the findings, and what else might?
7. **Transfer.** What would need to be true for the findings to apply to your case?

Worked appraisal

Apply the steps to the survey by van de Wetering and colleagues [62].

- **Question:** do EA-driven dynamic capabilities, innovativeness and organisational structure affect organisational benefits?
- **Constructs:** each was measured with questionnaire items, rated by respondents.
- **Method:** partial least squares structural equation modelling, plus fuzzy-set qualitative comparative analysis to find combinations of conditions associated with high benefits.
- **Sample and selection:** data from 299 CIOs and IT managers. Check in the paper how they were reached, because the recruitment route decides whom the results can describe.
- **Results:** EA-driven dynamic capabilities were associated with benefits, partly through innovativeness.
- **Limits:** the authors note self-reported measures and a single point in time. Nothing in the design makes the respondents a random sample of organisations.
- **Transfer:** for Helios, the study suggests that the capability to sense, mobilise and transform matters more than owning an architecture repository. It cannot say how much benefit Helios would gain, or whether the capabilities cause the benefits or the reverse.

The appraisal does not reject the study. It records exactly what the study can support, which is what a synthesis needs.

Independence of evidence

Count independent sources of evidence rather than citations. For example, a survey analysed in two ways still rests on the same respondents and measures. Two papers discussing the same case share their evidence. A practitioner guide that cites a study adds no new data to it. Record where each source's data came from, so that a long reference list does not overstate the support for a conclusion.

A synthesis structure

Separate three layers in every synthesis:

1. **Source findings:** what each inspected source argues or observes, within its own scope.
2. **Inference:** your argument about how those findings may apply to the case.
3. **Proposal and test:** the conditional commitment you recommend, and the observation that would show whether it holds.

Take the question: "Under what conditions could shared customer information improve Helios's service outcomes?" The operating-model perspective (Chapter 4) asks what the business must share. The EA benefits model (Chapter 14) proposes intermediate mechanisms. The mirroring study (Chapter 7) suggests examining technical and organisational boundaries together. March's model (Chapter 15) highlights the risk of standardising too early. These perspectives complement each other, but they are not four independent proofs that one investment will succeed.

A synthesis might conclude: Helios should first share customer identity while keeping local fulfilment, because process equivalence is unproven; it should fund stewardship, because shared meaning otherwise has no owner; and it should use a bounded pilot, because adoption uncertainty could change the expansion decision. Each clause rests on a different kind of support, and each can be challenged.

Rival explanations

Write at least one rival explanation for your preferred design. If you recommend federated coordination because divisions need different processes, test whether the differences reflect legal or customer obligations, temporary workarounds or habit. If you recommend a shared platform to reduce duplicated engineering, check whether its contract and funding actually let teams stop doing that work.

Where sources seem to disagree, first compare their units, constructs, outcomes and time periods. A shared platform could make routine deployments more consistent while slowing unusual local changes; both findings can be true. State the trade-off and the circumstances in which it matters.

> **Check your understanding.** Take one paragraph of your own writing that cites a source. Mark each sentence as source finding, inference or proposal. If a citation sits at the end of a sentence that is really your inference, rewrite the sentence.

The research assignment

Write a critical synthesis of 1,500 to 2,000 words on a question about the Helios programme. Use at least four relevant sources, including two research papers that you have read in full. Include:

- a source matrix with columns for question, design, sample, key finding, limits and relevance to Helios;
- a recommendation that separates source findings, inference and proposal;
- one counterargument to your recommendation, stated as strongly as you can;
- IEEE-style references numbered in order of first citation.

Use official documentation for claims about products and standards, and research for claims about effects. If a source you want is unavailable, choose an accessible primary source that addresses the same question. Assessment rewards traceable reasoning and honest limits, not the number of citations.

Exercises

16.1 For five studies cited in this book, name the research design and state one claim each can support and one it cannot.

16.2 Critique the claim: “A research paper associates architecture maturity with performance, so buying an architecture tool will improve Helios’s profit.” Identify the missing steps in the argument.

16.3 Write a 600-word synthesis comparing the operating-model, mirroring and organisational-learning perspectives for Helios’s shared customer service. Separate source findings, inference and the evidence you would need.

16.4 Choose two empirical studies discussed in this book. Compare their unit of analysis, case or sample selection, evidence and limits. Use them to support a conditional Helios decision, and name one observation that could overturn it.

16.5 Appraise the Lange, Mendling and Recker survey (Chapter 14) using the seven appraisal steps. What does it suggest Helios should measure?

Key points

- Distinguish definitions, mechanisms, associations, causal effects and design proposals; each needs different support.
- Every research design has strengths and blind spots. Match the design to the claim.
- Appraise sources systematically, including how the sample was selected and what would need to be true for the findings to transfer.
- Count independent sources of evidence rather than citations.
- A good synthesis separates source findings, inference and proposal, and states a rival explanation and a test.

CHAPTER 17

Capstone transformation programme and board defence

Learning objectives

After this chapter you should be able to:

- Design a three-year transformation programme that integrates enterprise and solution reasoning.
- Construct a dossier in which every board-level commitment can be traced to supporting analysis.
- Revise the programme coherently after a funding cut and an acquisition.
- Defend a conditional recommendation to a board and to a technical panel.

The assignment

Act as Helios's enterprise architect. Recommend a three-year transformation programme within the €9 million envelope, and explain its relationship to the separate €12 million annual operating baseline. The programme must improve selected capabilities, preserve valid local differences and keep the business running through every transition.

The task is to make a defensible set of commitments rather than to choose one product for every capability. For each commitment, say what will change and why, who owns the consequence, what it costs, what must happen first and how success or failure will be recognised. Where a business promise depends on a specific integration, data or recovery mechanism, show that mechanism in enough technical detail to be checked.

Required dossier

Keep the board brief short, and put supporting analysis in appendices that can be traced from it. Page counts are guides; a clear model can replace several pages of prose.

Deliverable	Required content
Board decision brief	Two pages: problem, recommendation, commitment, cost, major risks and decisions requested
Enterprise baseline	Capability map and heat map, value stream, six representative applications, dependencies, known gaps
Operating model and principles	Operating-model choice by capability, architecture principles, tailored method
Target and transitions	Data authority, shared and local responsibilities, platform policy, at least two operable intermediate states
Governance and assurance	Decision rights, exception process, risk ownership, assurance plan
Investment and roadmap	Cash-flow model, benefits register, dependencies, annual constraints, retirement conditions
Adoption and sourcing	Changed work, service ownership, supplier eligibility, tested exit scenario
Measurement and research	Outcome measures, baseline, comparison and its limits, critical synthesis of sources
Technical appendix	One detailed integration or migration design with invariants, failure behaviour and operational evidence

The six representative applications are a sample of forty-three and leave most of the portfolio unexamined. State what further inventory work is needed before any group-wide retirement commitment. Do not invent interfaces to fill a diagram; label hypotheses and assign discovery work.

Review gates

The programme passes three gates:

1. **Gate 1, discovery and pilot funding.** Is the problem defined and the scope clear enough to fund discovery and the pilot?
2. **Gate 2, transition approval.** Do the pilot results support the chosen transition?
3. **Gate 3, rollout.** Do service ownership, recovery, data correction and adoption results support wider rollout?

At each gate, name what is approved and what remains conditional. A gate is useful only if it can change the decision, so define in advance the results that would stop, narrow or delay the work. For example, Gate 2 might require identity-related misrouting in the pilot divisions to fall by a stated amount against a comparison division. Keep the cost of delay and the cost of continuing the baseline in view, so that caution is weighed fairly.

Change card A: funding reduction

After your first complete dossier, reduce the envelope by 20% to €7.2 million. Keep a dated copy of the first version. Produce a revised allocation that sums correctly and update the deferred outcomes, dependencies, benefits, annual cash needs and risks.

Chapter 11's deferral scenario is one possible answer. Challenge its assumption that the deferred legacy systems can keep running safely, and propose another scope choice if that assumption fails. Chapter 10 shows how a cut in the wrong place can destroy most of an investment's value while saving little; check your revised plan for the same effect. Do not silently remove contingency, assurance or support while claiming that the earlier promises still hold.

Change card B: acquisition

Helios acquires a business that records customer relationships by service site and contract, whereas the pilot links records mainly by legal party. Some sites share a parent company but have different permissions, entitlements and billing arrangements; for example, two depots owned by the same logistics group may hold separate maintenance contracts. The quality of the acquired data has not been measured.

Revise the data model and the integration scope. A match on the parent company must not automatically grant access to every site. Introduce explicit relationships, named authorities and review of uncertain links, then reassess the benefits and the migration schedule. The acquisition may strengthen the case for shared identity meanings while weakening the case for early process consolidation.

Neither the initial €9 million nor the revised €7.2 million automatically includes acquisition integration. State whether you absorb a defined scope by deferring other work, or request a separate investment decision. Either can be defensible if the costs, dependencies and consequences are explicit.

Assessment guide

The weights below are a course rubric for reviewing your submission. A different design can earn full credit when its reasoning and commitments are coherent.

Dimension and weight	What strong work shows
Business problem and capability reasoning, 15%	Clear outcomes, legitimate variation and scope boundaries
Architecture and transition coherence, 20%	Consistent authority, interfaces, intermediate states and retirement conditions
Economics and roadmap feasibility, 20%	Correct arithmetic, explicit assumptions, dependencies and honest scope changes
Governance, risk and adoption, 15%	Accountable decisions, operational ownership and proportionate assurance
Research and measurement, 15%	Relevant sources, critical synthesis, stable measures and limits on attribution
Communication and revision, 15%	Traceable board brief, clear defence and a coherent response to both change cards

Some defects need correction whatever the total score: unauthorised data access, unsupported claims of safe retirement, hidden double counting, or a roadmap that exceeds its funding. Use the rubric as feedback for revision, not as a way to average away a serious flaw.

Prepare the defence

Give a ten-minute board presentation, followed by questions. Lead with the decision required and its consequence. Be ready to explain one rejected option, one assumption that could reverse the recommendation, one business risk during transition and one concrete result that would support rollout.

Then defend the technical appendix to a panel of architects. Trace one important operation through customer identity resolution, workforce access, data authority, failure, recovery and measurement. Explain where a contractual or organisational condition is needed for the technical design to work. This is where enterprise architecture meets the systems that carry out its promises.

Exercises

17.1 Using the package-by-year tables in Chapter 11, check that the €7.2 million revision satisfies both its package totals and the revised annual limits. Then suppose the board insists on cutting every year's limit by 20%, to €2.4 million, €2.8 million and €2.0 million. Show where the revised plan breaks these limits, propose a schedule that meets them, and state what is delayed. Finally, identify the scheduling information still missing once the arithmetic agrees.

17.2 Respond to the acquisition without merging legal party, service site, contract and access entitlement into one identifier.

17.3 Write a board recommendation of no more than 250 words that names one conditional approval and the results required to release the next investment stage.

17.4 Design the decision criteria for Gate 2. For each criterion, state the measure, the threshold and the action if it is not met.

Key points

- A strong recommendation connects strategy to systems through explicit responsibilities, operable transitions, supporting analysis and revisable investment commitments.
- Every board commitment should be traceable to the analysis behind it.
- Review gates are useful only when their results can change the decision.
- Change cards test whether the architecture can absorb new constraints honestly, without hiding lost promises.

Exercise answers and assessment guidance

Attempt each exercise before reading its answer. Numerical answers follow the assumptions stated in the chapter and show their working. For open questions, the answer gives a model response and the criteria a marker should look for; other answers are equally valid when their authority, reasoning and consequences are explicit.

Chapter 1

1.1 Retail deploys a reliable customer database with its own identifiers and no shared correction process. Retail's service works well, but group reporting and cross-division handoffs now need manual reconciliation, because nobody can match Retail's customers to the other units'. The enterprise cost comes from incompatible identifiers and unclear authority rather than from poor local engineering, so a shared identifier contract may fix it without replacing the database. Credit answers that name the mechanism as well as the symptom.

1.2 Three examples: annual licence and support spend on duplicate applications, measured from invoices after verified retirement (owner: CIO); hours per month spent reconciling duplicate customer records, from time recording (owner: service operations manager); the percentage of cross-unit handoffs that fail because identifiers do not match, from the integration log (owner: group data owner). Each needs a baseline period and population. Application count alone is a weak measure, because consolidation can leave interfaces, support work and contract costs in place.

1.3 Chain: verified links, then correct routing, then fewer reassignments, then faster resolution. Challenge the first link with audited false matches; the second with routes that ignore the verified identifier; the third with an unchanged reassignment rate; the fourth with data showing that technician availability dominates elapsed time. Credit answers in which each challenge is something that could actually be observed.

1.4 Overall reassignment is 330 / 5,000 = 6.6%. Identity-related reassignments are about 0.40 × 330 = 132, or 132 / 5,000 = 2.6% of eligible requests, against 3.6% last quarter. The comparison may mislead because each share comes from a review of only 100 cases, so the 40% and 46% are uncertain by several points; and because seasonal demand, case mix, or a change in how reviewers classify causes could move the figures without any real change. Regression to the mean is also likely, because last quarter was the worst of the past eight (Chapter 14).

1.5 A model brief asks three questions: how the service processes differ and which differences are contractual; who should be authoritative for each shared customer fact; and what consolidation and a staged alternative would cost. Its outputs are a process-variation assessment, a fact-authority proposal and two costed options. The group service director owns it, supported by the architect, with a six-week deadline. Decision rule: consolidate if the process differences are accidental and one product meets all contractual obligations at an acceptable transition cost; otherwise fund shared identity and contracts first and revisit in a year. Credit a rule that the board could apply mechanically to the outputs.

1.6 The claim jumps from an association (mature EA and profit occur together) to a causal effect of buying a tool. The missing steps are a mechanism (how would the tool change decisions, data quality or operations?), the direction of influence (profitable firms may simply invest more in architecture), the definition of "mature EA" (a practice rather than a tool) and transfer to Helios. A sound response asks which decisions the tool would improve and how that improvement would be observed.

1.7 Reporting to the CIO keeps the team close to technology decisions, platforms and delivery, but the board may see it as an IT specialism and business units may bypass it. Reporting to the transformation programme director puts it next to investment and sequencing decisions, but it may lose influence over run-the-business technology and be disbanded when the programme ends. One good solution keeps a reporting line to the CIO, gives the head of architecture a standing seat on the transformation board and embeds solution architects in the units. Rhythm: fortnightly design reviews against the review triggers, a monthly portfolio and exception update, a quarterly roadmap review tied to funding gates and an annual review of principles. Measures: the share of significant investment decisions that used current architecture information; median time from design submission to decision; exceptions past their expiry date. Credit measures of effect on decisions rather than counts of documents.

Chapter 2

2.1 A taxonomy classifies what could be described: Zachman's grid used to check whether Helios has documented the "why" of its CRM decision. A method orders the work: a tailored ADM cycle for the customer-identity decision. A modelling language expresses the model precisely: ArchiMate elements and

relationships for the service view. A practice is what architects actually do: Helios's architects attending investment reviews with a one-page options paper. Reward answers that show the four are complementary.

2.2 Board three-year goals: why (motivation), scope contexts or business concepts depending on detail. Technician on-call rota: who (responsibility) and when (timing), technology physics or tool components. Retail warehouse network diagram: where (distribution), technology physics. Entitlement rules for maintenance contracts: why (motivation) and how (process), business concepts. Nightly warehouse load schedule: when (timing), technology physics. List of data owners: who (responsibility), business concepts. The most important remaining gap for the customer-identity decision is a business-concepts definition of what is shared (what, inventory) together with the rules that justify sharing it (why, motivation). Accept other gaps with a reason tied to the decision, and accept defensible row placements when the student explains the level of detail.

2.3 A model answer. Preliminary (light, half a week): confirm that existing principles and decision rights apply to the acquisition. Phase A (full, 1.5 weeks): scope, sponsor, stakeholders from both businesses, and the decisions required. Phases B and C (full, 4 weeks): profile the acquired data, model party, site and contract relationships, assess access rules and affected applications. Phase D (light, half a week): confirm hosting and integration options. Phases E and F (full, 1.5 weeks): integration options, transition states, costs and a funding decision. Phases G and H: set up governance for the chosen option. Give credit where answers concentrate effort on data meaning and access, where the uncertainty is.

2.4 Example: name "Grant access by verified relationship"; statement "Access to customer data across divisions is granted only through verified relationships, never through a probabilistic match"; rationale "a false match can disclose one customer's data to another"; implications "matching services must record match status; access policies read only verified links; unresolved matches need a steward process". It would rule out a proposal to give service agents access to every record with a match score above 0.85. Look for principles that can be broken and that change a real decision.

2.5 Business domain: service coordinator (role, active structure; a common-domain element used here for a business role), handle service request (process, behaviour), field maintenance (service, behaviour), customer (business actor, active structure). Application domain: CRM North and customer link hub (application components, active structure), case management and customer lookup (services, behaviour). Technology domain: shared cloud platform (node, active structure), container hosting (service, behaviour). To add CRM Central, add an application component realising a case-management service for Retail's process, served by container hosting, and served by customer lookup if Retail uses the link hub. The view then shows that both CRMs and the link hub depend on one hosting service and that the link hub serves both divisions: two shared dependencies. In ArchiMate 3.2 the behaviour elements would have been a business process, a business service, two application services and a technology service, and the coordinator a business role.

2.6 The claim treats framework adoption as the goal. Kotusev's case study of a TOGAF-based practice concluded that none of the TOGAF-specific recommendations proved useful there, and his analysis of 27 organisations found 24 artefacts that proved useful in practice, used for specific purposes. Full adoption would consume effort on unused deliverables. A better measure of progress is the share of significant investment, design and governance decisions that used current architecture information, together with whether those decisions changed as a result. Look for answers that acknowledge the studies' limited number of cases.

Chapter 3

3.1 Capability: entitlement checking. Process: look up the contract, check the asset, record the decision. Application: CRM North. Organisational unit: the Maintenance service desk. The capability is the most stable because it describes what Helios must be able to do, which remains true when the process, the software or the reporting line changes.

3.2 Active time is 0.8 days in both cases. With the entitlement improvement alone, elapsed time is 4.7 days and flow efficiency is 0.8 / 4.7 = 17.0%. Reducing the billing wait by a further 1.2 days gives elapsed time 4.7 – 1.2 = 3.5 days and flow efficiency 0.8 / 3.5 = 22.9%. Nobody works faster; all the gain comes from shorter queues.

3.3 Importance: medium, because it supports cost control and audit findings but no board objective depends directly on it. Effectiveness: 2, because one invoice in eight needs manual matching and audit has commented on late payments. Confidence: high for the matching rate, which comes from the accounts-payable log for a full quarter; the effectiveness rating as a whole is medium, because the cause of the manual matching is not yet known. The capability is not a priority candidate, because its importance

is not high. The rule points to a quick fix (for example, checking whether a few suppliers cause most of the mismatches) and continued measurement. Marking: any defensible ratings earn credit if each states its source; extra credit for separating confidence in the data from confidence in the rating.

3.4 Example: “Finance can report monthly revenue per customer group, where groups are formed from legal entities using the governed group identifier. The mapping used is versioned and no more than one day old at month end. Revenue from customers with unverified links is reported separately and totals reconcile to ERP Core.” Credit requirements with a named user, scope, authority, freshness and explicit treatment of uncertain mappings.

3.5 A single red-amber-green colour mixes importance, effectiveness and confidence, so readers cannot tell why a capability is red or what action follows. Better: rate importance and effectiveness separately on defined scales, show confidence (for example by border style), record the basis for each rating, and apply an explicit rule that links ratings to actions. Designs earn credit if they remain readable in greyscale.

Chapter 4

4.1 Customer identity resolution: units must recognise shared customers for cross-unit service and reporting (high integration), but maintenance, retail and subscription processes can remain different (low standardisation). The answer should say which facts must agree and which process differences remain legitimate.

4.2 “Closed” could mean that a technician completed the work, that a retail order was delivered or that a customer confirmed the fix. One shared code without a mapping produces false comparisons of closure times. Either agree a common business definition and change processes to match, or keep distinct events and map each to an explicit reporting concept, preserving the distinctions where no valid equivalence exists.

4.3 Payroll: low integration, high standardisation (replication-like): the same process and rules in every unit, with little need to share payroll data across units. Customer billing across the three units: high integration if group customers are to receive consolidated invoices, with low standardisation where each unit bills different products (coordination); if units bill separately, low integration. Group cash management: high integration, high standardisation (unification), because the treasury needs a single view and common rules. Retail’s warehouse picking does not belong in the grid, because only Retail performs it: the grid describes how units relate to each other, so a single-unit process is a local design decision. Credit justified placements that consider the scope of sharing.

4.4 Central consolidation is hardest to unwind: Retail’s data, process and operating responsibility must be extracted from a common application. Federated coordination needs separable identity mappings, permissions, contracts and platform dependencies, which can remain contractual after the sale through a transition services agreement. Local autonomy simplifies separation but leaves group reporting and shared supplier contracts to untangle. Data rights and transition agreements must be explicit; not every shared service needs physical duplication. Credit a consequence-specific boundary rather than a blanket preference.

4.5 The claim describes only the strategy-execution perspective. The strategic alignment model also includes perspectives in which IT strategy enables new business strategy (competitive potential) or shapes IT services to the organisation (service level). Helios example: a shared customer identity could enable Digital Services to sell combined maintenance and monitoring contracts, a strategy the group cannot pursue today. Alignment also requires strategic fit between external choices and internal infrastructure, which “doing what the strategy says” ignores.

4.6 ERP Core, the nightly billing run and the party register belong to the operational backbone: they run core transactions and hold authoritative core data, and they change deliberately. The customer lookup service and the stream of verified-link events belong to the digital platform: they are reusable components that teams can combine quickly into offerings. The link hub itself sits between the two, as a backbone data service exposed through platform interfaces. Building the monitoring contract directly on CRM North’s database would couple a fast-changing offering to a backbone system’s internal schema: a CRM upgrade could break the offering, the offering could write data that bypasses CRM North’s rules and the customer-identity controls, and the backbone’s change process would slow the offering down. Full marks go to answers that name the published contract as the boundary.

Chapter 5

5.1 €420,000 + €360,000 + €900,000 + €600,000 + €300,000 + €240,000 = €2,820,000. These costs are already inside the €12 million baseline, so adding them would count the same money twice. The remaining €9.18 million cannot be attributed to particular applications without further data.

5.2 It may support a legally required low-volume process; it may feed an undiscovered batch interface or report; and exit may carry contract penalties or data-retention costs. A replacement could also cost more than continued operation. Investigate dependencies, obligations and lifecycle costs before treating low usage as a reason to retire.

5.3 Keeping both preserves local fit and avoids migration now, but requires governed contracts and continuing dual support. Consolidation may reduce duplication but introduces process change, migration and concentration risk. The information most likely to change the choice: whether the process differences are real or accidental, the true run costs, the interface dependencies, contract terms and the users' actual work. A pilot should target the most consequential of these uncertainties.

5.4 Business value = (3 + 2 + 2 + 3) / 4 = 2.5; technical fitness = (4 + 2 + 3 + 2) / 4 = 2.75. Both are below the 3.0 threshold (a score of exactly 3.0 would count as high), so the system falls in Eliminate. The quadrant alone should not decide, because the system may be Retail's only route for picking orders, and fitness at 2.75 is close to the boundary: one criterion moving by a point would place it in Tolerate. Facts to check first: which processes and interfaces depend on it (for example, whether order allocation and delivery scheduling read its data), whether a replacement exists, the contract and exit terms, and whether the low value scores reflect a few users rather than low importance. Good answers treat Eliminate as a prompt for investigation and a retirement plan, never as an instruction.

5.5 The original deadline is 14 months; the extension moves it to 20 months for €150,000. Without the extension the margins are: rehost 10 months, replatform 6, repurchase 2, and refactor –4 (infeasible). With it they are: rehost 16, replatform 12, repurchase 8 and refactor 2. Only refactoring needs the extension; for repurchase it buys comfort, and replatforming does not need it.

Compare refactor and replatform over the same five years from the decision, at 8%, with one-off costs at the start and running costs at year ends, as in Chapter 10's TCO table. Replatform: €0.60m one-off; year 1 €0.55m (8 months of Legacy Orders at €0.6m a year, €0.40m, plus 4 months of the new platform, €0.15m); years 2 to 5 €0.45m each; present value €2.49m. Refactor: €1.80m plus the €0.15m extension, €1.95m one-off; Legacy Orders keeps running for 18 months, €0.60m in year 1 and €0.30m in year 2, which is €0.50m more legacy cost than replatforming; the refactored system runs for only the last 3.5 years at an unknown cost R a year. Its present value is $1.95 + 0.60 / 1.08 + 0.30 / 1.08^2 + R \times (0.5 / 1.08^2 + 1 / 1.08^3 + 1 / 1.08^4 + 1 / 1.08^5) = €2.76m + 2.64 \times R$. Even with R = 0, refactoring costs €0.27m more in present value than replatforming over five years, and with R = €0.45m it costs €3.95m. Refactoring could only pay off over a longer horizon, through benefits other than running cost, or both, and it would still have only two months' margin. Replatform remains the recommendation, with repurchase plus the extension as the alternative if process change is wanted.

Chapter 6

6.1 Example rows: service site (authority: party register for site identity, maintained by party stewards; the CRM may hold local site notes); customer contact person (authority: the CRM of the unit that owns the relationship; local service staff may update it); contract entitlement (authority: ERP Core or the contract system; changed only by contract administration). Reward choices that separate facts which different teams legitimately maintain, and that justify one choice by its consequence.

6.2 A probabilistic match with a known error rate can be acceptable for estimating, for example, the number of shared customers. Used for authorisation, a single false match can disclose one customer's data to another. The consequence of an error, and therefore the acceptable threshold, is different. Use verified, governed relationships for access decisions and send ambiguous matches to review.

6.3 The sample rate is 3 / 400 = 7.5 per 1,000, above the contract's limit of 5, so the contract is breached. Applied to 20,000 links, about 20,000 × 0.0075 = 150 links could be false. With only 3 errors in the sample, the estimate is very uncertain: an exact 95% interval for the rate runs from about 1.5 to 21.8 per 1,000, or roughly 30 to 440 false links. The audit should be enlarged, or the next month's sample combined, before drawing firm conclusions.

6.4 Platform operations pause propagation; the evidence is that the identifier has stopped flowing. The data steward lists affected copies and reports from the lineage records. The group data owner decides the split and signs a decision naming the correct source records. The link-hub team publishes the correction event, and each consumer acknowledges it. Division stewards reconcile the CRMs and the analytics

owner rebuilds or annotates reports, recording reconciliation counts. The privacy officer assesses any disclosure. The group data owner closes the case once every consumer confirms completion. Strong workflows preserve history and reach every copy.

6.5 One master record for every fact would force every system to wait for, and agree with, a single model, even where meanings differ legitimately; it would also concentrate risk and slow local change. A better design assigns authority fact by fact, uses a governed identifier with match status to link records, publishes contracts for shared facts and leaves local facts local.

6.6 The registry style keeps each CRM authoritative and needs little change in the divisions, which suits federated coordination and a short timetable; its weakness is that every correction still has to be made in the source system. A centralised hub would give one place to maintain customer facts and stronger control, but it forces every division, including the acquired business, to adopt common meanings for those facts and to change its processes and integrations to write to the hub. For Helios the change is justified only if the divisions agree common definitions for most customer facts, the acquired business's site-and-contract model can be expressed in the hub without losing distinctions, and the cost and transition risk fit the funding. A coexistence step, with the hub and the sources synchronised, is a defensible intermediate. Strong answers tie the style to authority and process change rather than to technology preference.

Chapter 7

7.1 Example: scope: all production applications. Requirement: logs use the group structure with timestamp, service, correlation identifier and severity, and are sent to the central platform within five minutes. Rationale: incident analysis across services currently takes days because formats differ. Owner: observability platform owner. Release evidence: an automated test shows a request traced across two services. Exceptions: time-limited, approved by the owner, with a dated plan. A strong standard states why it exists and how compliance is shown.

7.2 Code may run elsewhere but still rely on provider-specific database features. Data portability needs an export with types, keys and relationships intact. Operational portability needs deployment, credentials, backup, monitoring and recovery working in the destination. Commercial portability depends on licences, egress charges and exit terms. Demonstrate a business journey after transfer; a built container or a downloaded file proves only a narrower property.

7.3 A faulty identity configuration change can block sign-in to every application on the platform. Recovery requirement: a tested rollback of identity configuration within a stated time, protected break-glass administrative access, and a documented degraded procedure for critical workflows. The numerical target should come from the business service's needs and measured recovery capability.

7.4 Across all ten changes the mean wait is 67 / 10 = 6.7 working days; across the eight that needed approval it is 67 / 8 = 8.4 days. Six of the eight approvals, 75%, concerned single-team data. A first action is to delegate approval of single-team schema changes to the owning team under published rules, keeping central review for shared tables. Success after three months: median wait for single-team changes near zero, no increase in incidents caused by schema changes, and central review time focused on genuinely shared data.

7.5 Conway's law states that organisations design systems whose structure copies their communication structure. The mirroring study found that, across matched software products, loosely coupled organisations produced more modular designs. Together they justify investigating whether Helios's approval structure is shaping its system coupling. Neither proves that reorganising will improve delivery: the evidence concerns software products in a small sample, organisational and technical changes interact, and Helios's own data must show where delay arises. Reward responses that propose a bounded change with measures.

Chapter 8

8.1 Architects and security specialists advise on consequences and controls. The authorised business risk owner decides whether the residual risk is accepted, within delegated limits. A review board may hold delegated rights, but only if that is explicit. At Helios, the platform owner advises and the division's risk owner accepts.

8.2 The record should name the application, the affected identity functions, why integration cannot happen yet, the observed gaps, the controls actually operating (for example, monthly manual account reconciliation), the owner, the expiry date and the integration milestone. Planned controls, such as automated removal, are listed as conditions that must be shown working before approval. Include triggers for revocation or escalation.

8.3 Either the standard does not fit a legitimate, common need, or a shared implementation or support problem prevents teams from complying with a sound standard. Compare the use cases, consequences, effort and control results across the three requests. Change the standard only once the pattern is understood; otherwise fix the platform or guidance and keep the requirement.

8.4 IT principles: anarchy (nobody states them). IT architecture: anarchy for customer data, since nobody owns the identifier. IT infrastructure: IT monarchy (the CIO). Business application needs: feudal (each unit buys its own). Investment: feudal or business monarchy, depending on how budgets are set. The most serious gap is the absence of any decision right over shared customer data, which is exactly where the board's objectives need coordination. A federal arrangement for that domain is the natural fix.

8.5 A model answer escalates the change of meaning to the group data owner with both business roles represented, and keeps delegated approval for compatible additions. Structural: named owner and escalation route. Procedural: example records, consumer tests, migration window, versioned decision. Relational: service staff who resolve disputed cases join the design discussion. The main paradox is collaboration (integration versus autonomy), with distinction (comparability versus differentiation) also present. Credit work that links each mechanism to a specific failure it prevents, and that treat the two-case study as a source of questions rather than proof.

8.6 Charging per event discourages teams from publishing events that reconciliation depends on, which undermines correctness. Better units are per connected application with a fixed tier, or a central allocation for core events with charges only for unusually high volumes. Whatever the unit, check that it does not penalise the behaviour the architecture requires.

8.7 Evaluate: the board assesses the options paper and each funding-gate submission, including the business case and its sensitivity analysis. Direct: it sets the architecture principles, approves the envelope and the gate conditions, and names accountable owners. Monitor: it reviews gate results, the benefits register, overdue exceptions and audit findings each quarter. Engage stakeholders: it hears business-unit leaders, and where relevant key customers, before each gate. A suitable design factor is the I&T-related issues factor, because inconsistent customer data across acquired systems is a known current problem; the risk profile (acquired applications and personal data) and compliance requirements are also defensible. The best answers link the factor to a Helios fact.

Chapter 9

9.1 Risk: because account removal does not reach acquired applications, a former employee may sign in after leaving, causing unauthorised disclosure of customer data. Control 1: lifecycle events reach every application, with coverage monitoring; evidence: a reconciled list of applications and recorded removal events. Control 2: automated denied-access tests for removed accounts; evidence: test results naming the identity, resource, configuration and date. A policy document alone is not evidence that either control works.

9.2 They can share an identity provider, a DNS configuration, an administrator account, a software release pipeline or a supplier's control plane. A failure in any of these can affect all three at once, so separate hosting does not guarantee independence.

9.3 Availability is $0.998^4 \times 0.999 = 0.991$, so the service is unavailable about 0.9% of the year, or roughly 79 hours. If failures are correlated, for example if two applications share an administrator account or release, the simple product understates the risk of simultaneous outages and overstates confidence in the figure; the shared elements need their own analysis and recovery tests.

9.4 Which service and failure scenarios does the objective cover? When does the clock start and stop? What does "recovered" mean, and what data-loss objective applies? What evidence supports the figure? Which steps remain Helios's own: detection, the decision to resume, access restoration, integration catch-up and reconciliation? What configuration duties and exclusions apply to Helios as the customer?

9.5 A certificate shows that the supplier's management system met a standard within a stated scope at audit time. It does not cover Helios's configuration, identity integration, data flows or application code. Request the certificate's scope, the supplier's relevant control reports, and Helios's own results: configuration review, access tests, a restore test and a review of data flows for the new service.

9.6 Today each acquired application trusts anyone who holds a valid local account, so a leaver whose local account was not removed can still sign in. In a zero trust architecture the application grants no implicit trust: every access request is evaluated per session against current policy, including the person's status in the workforce identity service and the state of their device. The policy engine decides, consulting the workforce identity service and device information; the policy administrator opens or refuses the session; the policy enforcement point, for example an access gateway or proxy in front of the acquired

CRM, carries out the decision. A leaver is refused at the enforcement point even if the CRM's own account list is stale. Look for responses that recognise zero trust as a target direction that needs integration work, with interim controls meanwhile.

Chapter 10

10.1 Investment €2.5m at time zero; benefits at the end of years 1 to 3. At 8%: –2,500,000 + 1,200,000 / 1.08 + 1,200,000 / 1.08^2 + 1,200,000 / 1.08^3 = –2,500,000 + 1,111,111 + 1,028,807 + 952,599 = €592,516. With €960,000 a year: –2,500,000 + 888,889 + 823,045 + 762,079 = –€25,987. The investment is unchanged while the discounted benefits fall by 20%, from €3.09m to €2.47m, which is just below the €2.5m outlay, so the sign changes. The break-even benefit is €970,084 a year, 19.2% below the estimate.

10.2 Record the €200,000 reduction in contractor invoices once, as a cash benefit, if the reduction is achievable. The released hours are the mechanism behind that saving and cannot be counted as a second €200,000. If some hours are redeployed rather than removed from invoices, split the population: the removed hours are cash, and the redeployed hours are capacity, with any further value supported separately.

10.3 The real rate is 1.08 / 1.03 – 1 = 4.85%. Discounting €1.2m a year in today's prices at 4.85% gives present-value benefits of €3,276,836 and an NPV of about €776,836. The same answer follows from inflating the benefits by 3% a year and discounting at 8%. Discounting the constant-price benefits at 8% would understate the NPV by about €184,000.

10.4 The gross value of the pilot policy is 0.34. With a cost of 0.16 its net value is 0.18, below the 0.20 of expanding now, so the pilot is not worth running; the break-even cost is 0.34 – 0.20 = 0.14. If the result cannot change the decision because expansion is already committed, the information is worth nothing: the policy is worth 0.20 minus the pilot's cost, unless the pilot creates separately specified learning or implementation value.

10.5 Fund a migration rehearsal on representative data and the riskiest interface, with a fixed budget and a four-week limit. Collect reconciliation defects, throughput, specialist effort and rollback feasibility, then re-estimate the migration cost. Decision rule: proceed if the re-estimated cost keeps the NPV positive at the agreed discount rate and all continuity conditions are met; narrow the scope if only part of the migration meets the rule; stop if the rehearsal reveals data losses that cannot be fixed within the budget. Credit rules set before the rehearsal.

10.6 The IRR of 20.71% is well above the 8% hurdle, but the argument ignores how the value depends on assumptions. The break-even benefit is €0.97m a year, 19.2% below the estimate, so a 20% shortfall turns the NPV to –€25,987. IRR also says nothing about scale or about what happens if the budget changes. If €0.6m of integration work is deferred, the NPV falls to €407,886; if it is removed and the service-credit benefit is lost, the NPV falls to €161,678 and the case becomes fragile. Approval should rest on confidence in the size and timing of the benefits, with the integration funding protected or a staged commitment tied to evidence of benefits, rather than on the IRR alone.

10.7 The listing books each year's €1.2m benefit at the end of the year, so cumulative benefits pass €2.5m only when year 3's cash arrives: payback is 3 years. The analyst assumed benefits accrue evenly within each year and interpolated: 2 + (2.5 – 2.4) / 1.2 = 2.08 years. Both are arithmetically right under their conventions. The board should see 3 years, the figure consistent with the NPV model, or both figures clearly labelled; mixing an interpolated payback with a year-end NPV makes the case look faster than the model behind it.

10.8 With an investment of €3.0m and the same benefits, the NPV is –3,000,000 + 3,092,516 = €92,516, and the IRR falls to 9.70%. The case then looks marginal. Charging the workforce identity work to it mixes two different mechanisms: the €0.5m workforce package delivers a risk benefit (former staff lose access) that has nothing to do with handoffs. Keeping them apart lets the board judge the handoff case on its own cash flows and the access case on its risk reduction, and stops either being approved or rejected for the other's reasons.

Chapter 11

11.1 Forward pass: A finishes at week 4, B at 10, C at 7, D at 14 and E at 16. Backward pass: C may finish as late as week 10, so it has 3 weeks of slack; A, B, D and E have none, giving the critical path A–B –D–E. If one indivisible team must do B and then C, C runs from week 10 to 13, D from 13 to 17 and E from 17 to 19, so completion moves to week 19.

11.2 In State 2, each division's CRM remains authoritative for its own customer records; the customer link hub is authoritative for verified cross-system links; only party stewards may create or reject links;

consumers read verified links only. Retirement evidence for the spreadsheets: every consumer migrated to the link hub, records reconciled, retained history accessible, no remaining dependent reports and a named operational owner for the link service.

11.3 One alternative keeps legacy modernisation at €2.4m (because several legacy systems carry security risk) and instead removes the €0.9m analytics package, reduces the CRM pilot to €0.6m and reduces contingency to €0.6m, saving 0.9 + 0.6 + 0.3 = €1.8m. By year, in € millions: customer identity 0.6, 0.4, 0; workforce identity 0.4, 0.1, 0; integration 0.5, 0.5, 0.2; legacy modernisation 0.6, 0.9, 0.9; CRM pilot 0.3, 0.3, 0; adoption and assurance 0.3, 0.3, 0.3; contingency 0.3, 0.3, 0. The years total 3.0, 2.8 and 1.4, within the limits. Dependency: the customer identity foundations must stay whole because integration, the pilot and analytics all rely on them. Risk: year 3 has no contingency just when the largest legacy migrations happen. A uniform 20% cut would weaken every package, including indivisible security and foundation work, without removing any promise.

11.4 A finishes at 4; B and C both start at 4; C finishes at 7; F runs from 7 to 10; B finishes at 10; D starts at 10 and finishes at 14; E finishes at 16. Completion stays at 16 weeks, but every task now has zero slack: there are two critical paths, A–B–D–E and A–C–F–D–E. The programme manager has lost all scheduling flexibility, so a delay in training now delays the pilot as surely as a delay in the identifiers.

11.5 The line names no outcome, no intermediate states and no conditions. A better set of entries runs as follows. First, move from State 1 to State 2: the link hub goes live for two divisions, with exit when the agreed pilot thresholds are met. Second, complete the process analysis and take the consolidation decision. Third, if consolidation is approved, migrate CRM North's open cases and history and publish the reconciliation results. Fourth, run both systems in parallel for a defined period with named owners. Finally, retire CRM North only when consumers have migrated, records reconcile, retained history is accessible and the contract can be ended. Each entry needs an owner, a cost and its dependencies.

Chapter 12

12.1 Input: support staff trained and stewards appointed (measure: completed training and filled roles). Behaviour: verified identifiers used correctly in eligible workflows (measure: audited sample of requests). Outcome: fewer requests misrouted because of identity errors (measure: identity-related reassignment rate). Audit for false links and shadow spreadsheets, because training can be complete while the workflow still encourages a wrong selection.

12.2 A migration-count target rewards moving applications even when local service is not ready. A shared acceptance condition: a migration counts only when data is reconciled, recovery has been tested, support staff are trained, service performance meets the agreed level for four weeks and exceptions have an owner. Include a funded transition window so that the condition can be met.

12.3 Measure the volume, causes and resolution time of the exceptions and the skills they need. Then improve the design so that fewer exceptions arise, or fund and train a permanent operating team, and repeat the pilot without project-team rescue. A controlled exception process can be part of the architecture, but its capacity and cost must be explicit. The pilot did not show independent operation.

12.4 Explanation 1: the screen does not display service sites, so the agent cannot select one. The missing element is Ability, and the action is a design change. Explanation 2: the agent does not believe that the new identifier is reliable after seeing wrong matches. The missing element is Desire, and the action is to fix the matching errors and show the agent the improved results. Give credit to answers that recognise that ADKAR locates the gap but that the remedy may be structural.

12.5 Lewin's idea focuses attention on removing the old route and stabilising the new one, which suits retiring the spreadsheets. Kotter's steps add sponsorship, a cross-unit coalition and visible short-term wins. Both are popular more for their usability than their evidence: the three-step model was assembled after Lewin's death, and Kotter's whole sequence has not been tested. An architect should add a design check (can the new route express everything the spreadsheet held?), measures of actual use and a dated retirement decision.

12.6 Contract revision with narrowed rollout is defensible when the current contract cannot express the site relationship. Enforcement is preferable only if the meaning already exists and better guidance would fix the errors. Finding that representative records already express sites correctly would reverse the preference; finding invalid entitlement despite correct sites would redirect attention to the entitlement service. The public-sector case describes a process in a different setting, so it suggests questions rather than the answer for Helios. Credit a funded operating path and an explicit limit on inference.

Chapter 13

13.1 Base weights: A = 0.40 × 4 + 0.35 × 3 + 0.25 × 4 = 1.60 + 1.05 + 1.00 = 3.65; B = 0.40 × 3 + 0.35 × 5 + 0.25 × 3 = 1.20 + 1.75 + 0.75 = 3.70. Revised weights: A = 2.00 + 0.75 + 1.00 = 3.75; B = 1.50 + 1.25 + 0.75 = 3.50. The ranking reverses because the importance given to operability changes. Scores are judgements on an ordinal scale, so neither total shows that one supplier is better in general.

13.2 Decision: Supplier C is ineligible because it did not demonstrate a tested export that preserves identifiers and relationships; its scores are not considered. Reconsideration requires a successful representative export that reconciles in a usable destination under acceptable contract terms, or an explicit decision by the requirement's owner to change the requirement after assessing the consequences. Price alone cannot waive it.

13.3 With B's operability at 4, the differences between A and B are +1, −1 and +1, so the difference in totals is (1 − w) − w = 1 − 2w, which is zero at w = 0.5. B now overtakes A only if operability carries more than half the total weight. Lowering B's operability score makes the result much less sensitive to the weights, which shows that the base ranking hinged on B's single strong criterion.

13.4 Export a representative set of identities with their groups, roles, entitlements and history; import it into a separate identity environment; reconcile counts, unique identifiers, group memberships and role assignments; sign in as sample users and check that each can reach exactly the resources they should; rebuild an access review report; record effort, duration, unsupported attributes and any licensing needed to keep the data. Reward tests that check behaviour and relationships, not just file transfer.

13.5 Customer matching depends on knowledge specific to Helios (its customers, sites and contracts), changes often, and is tied to access decisions, so transaction-cost reasoning suggests that a simple market contract would be costly to specify and risky to depend on. Outsourcing research shows that buying a service shifts work into contracting and supplier management rather than removing it. A reasonable answer keeps matching rules and stewardship in-house and may buy commodity components, such as address validation. Credit balanced answers that name what could make outsourcing sensible.

Chapter 14

14.1 Before: A 10 / 100 = 10%, B 45 / 900 = 5%, pooled 55 / 1,000 = 5.5%. After: A 72 / 900 = 8%, B 4 / 100 = 4%, pooled 76 / 1,000 = 7.6%. Both units improve, but the higher-rate unit's share of demand rises from 10% to 90%, so the pooled rate rises. Standardised to the earlier mix, the later rate is 4.4%. Show the board both: 7.6% is what customers experienced, and 4.4% isolates the change in unit performance.

14.2 Name: unresolved customer-identity links. Definition: proposed cross-system links awaiting a steward's decision, counted daily at 06:00, with their age in working days. Population: links in the customer link hub for participating divisions. Exclusions: links withdrawn by the source system. Owner: group data owner. Decision: if more than 200 links are older than five working days for two consecutive weeks, add stewardship capacity or pause rollout to new divisions; set the threshold from business consequence and observed demand.

14.3 The improvement is real if the measure was stable, but the architecture's contribution is not isolated. Compare similar workflows with different rollout timing, stratify by staffing level and case mix, examine reassignment causes and review incidents and interviews. State the remaining confounding and avoid crediting the whole improvement to either staffing or architecture.

14.4 The pilot changes by 8 − 12 = −4 percentage points and the comparison by 8 − 9 = −1, so the difference-in-differences is −3 percentage points. Three threats: the pilot was chosen after an unusually bad quarter at 12%, so some improvement is regression to the mean; staffing or process changed in the pilot division at the same time; the comparison division was partly exposed to the new service. Sample sizes and dependence are also needed before stating any uncertainty.

14.5 Confidence: a defined before-and-after survey of relevant roles. Adaptability: a rehearsed change, such as adding a service-site relationship, with effort, dependencies and defects recorded. Cash savings: realised incremental cash flows net of replacement and transition costs. Rival explanations: better leadership that improves both; reverse influence from better-performing divisions; shared optimism among respondents. Supportable conclusion: "Respondents report greater confidence in changing systems than before the programme." It cannot establish adaptability or savings.

14.6 The research does not show that EA improves performance in general. Tamm and colleagues offer a conceptual model; Lange and colleagues and van de Wetering and colleagues report associations in cross-sectional, perception-based surveys; Kotusev's case studies show practitioners relying on artefacts used in planning and decisions. Together they suggest that value depends on embedding architecture in

real decisions, which tells Helios what to measure rather than what it will gain. Whether the programme pays for itself depends on Helios's own cash flows, measured as in Chapter 10.

Chapter 15

15.1 Compare three arms on the same held-out cases: current workflow, a template baseline and the assistant. Measure time to a finished summary including correction, and two consequential errors: wrong-customer content and omitted safety conditions. Stop condition: any unauthorised disclosure, or wrong-customer content above an agreed rate, suspends the pilot. Narrowing rule: if errors concentrate in one record type, exclude it. Keep business actions outside the assistant's authority and record configuration changes.

15.2 Before: 0.02 × 250 + 1.5 = 6.5 g per request. If hardware grows with demand, M stays at 1.5 g: the new SCI is 0.01 × 200 + 1.5 = 3.5 g per request, a 46% fall in intensity, and with demand doubled total emissions are 2 × 3.5 = 7.0 g for every request in the starting volume, 7.7% more than before. If the existing hardware absorbs the extra demand, M halves to 0.75 g: the SCI is 0.01 × 200 + 0.75 = 2.75 g, and the total is 2 × 2.75 = 5.5 g, 15.4% less than before. The boundary excludes user devices, networks outside the service and, in the first case, any change in the type of hardware bought.

15.3 Drafting and dispatch have different authority and consequences. Dispatch changes real work and can affect safety, entitlement, cost and service commitments. The drafting pilot tested none of those decisions, integrations or failure modes. A new proposal needs its own bounded action model, controls, evaluation and accountable owner.

15.4 Zero disclosures in 300 cases gives an approximate 95% upper bound of 3 / 300 = 1% on the true rate, so the data cannot show that the rate is below 0.1%; that would need roughly 3,000 cases with no disclosures. More useful than a larger test are preventive controls (access checks at retrieval and on output), monitoring in operation and a fast way to withdraw the feature.

15.5 Eligible cases: site-and-contract records from the acquired business, with independently adjudicated true matches. Compare the standard method and the alternative on the same cases, measuring false merges and missed matches and their downstream harm. Budget and time limit fixed in advance. Stop if false merges exceed an agreed level or the budget is spent. Change the standard only if the alternative reduces harmful errors on its eligible cases; a result confined to one contract type supports a conditional rule. The thresholds come from business consequences, not from March's model.

Chapter 16

16.1 Tamm et al. built a conceptual model: it supports a proposed mechanism but cannot estimate an effect size. March used a formal model: it shows the consequences of assumptions without showing that they hold. MacCormack et al. used comparative measurement: it supports a relationship across matched products rather than a general law. Lange et al. ran a survey: it supports associations among perceived constructs, not causal direction. Jöhnk et al. studied qualitative cases: they explain how mechanisms work in context, not how common they are.

16.2 The argument moves from association to causation, does not say which part of maturity matters, assumes a tool changes practice, ignores reverse influence (profitable firms invest more in architecture) and ignores differences between the studied organisations and Helios. The missing chain links the tool to changed decisions, those decisions to operational changes, and those changes to measured financial outcomes.

16.3 A strong synthesis identifies three different questions (what to share, how organisational and technical boundaries interact, and how standards affect learning), bounds each source accurately, applies them to one coherent Helios option and gives a credible counterargument. It does not treat a stylised model, a small comparative sample and a practitioner framework as equivalent proof. Mark for traceability and handling of assumptions, not citation count.

16.4 For example, compare the van de Wetering survey (299 CIOs and IT managers reporting on their organisations, perceptions at one time) with the Dang and Pekkola case (one public-sector organisation, process over time). Neither isolates the effect of Helios's intervention. A conditional decision: fund a bounded stewardship pilot while collecting workflow and outcome data. Overturning observation: no change in identity-related reassignment, or an unsustainable exception workload. Good answers distinguish the studies' observations from the learner's proposal.

16.5 Question: which factors and measures characterise EA management success. Constructs: EA product, infrastructure and service quality, organisational anchoring, satisfaction, intention to use and benefits. Method: a model built from literature and interviews, tested with a cross-sectional survey.

Sample: 133 practitioners. Results: four success factors were supported; organisational anchoring mediated the effects of infrastructure and service quality on the success measures; satisfaction was irrelevant. Limits: perceptions, one point in time, practitioner respondents. Transfer: Helios should measure whether architecture is anchored in its decision processes, such as investment reviews and design approvals, and whether people intend to use it, rather than how satisfied they are with it.

Chapter 17

17.1 Revised package totals: 1.0 + 0.5 + 1.2 + 1.5 + 0 + 1.2 + 0.9 + 0.9 = €7.2m. Revised years: 3.0, 2.8 and 1.4, which match the revised limits and also sum to €7.2m. Under proportional limits of €2.4m, €2.8m and €2.0m, year 1 is €0.6m over. The Legacy Orders work and the identity foundations cannot move, because of the hosting deadline and the dependencies. One schedule that fits: move the year-1 CRM pilot spend (€0.3m) to year 2 and release no contingency in year 1, holding that €0.3m in year 3; then move €0.3m of CRM pilot spend from year 2 to year 3 to keep year 2 at €2.8m. The years become 2.4, 2.8 and 2.0. Consequences: the CRM pilot starts a year late, so Gate 2 and any consolidation decision slip by about a year, and a year-1 overrun would force a scope cut because no contingency is available. Still missing once the arithmetic agrees: resource availability by quarter, dependencies between packages, overlap of old and new systems during transitions, and contractual payment timing.

17.2 Model legal parties, sites, contracts and access entitlements as distinct concepts with explicit relationships and named authorities. A site can belong to a party without sharing every contract or permission. Profile the acquired data, review uncertain matches and reconcile pilot links before extending access. Stage the integration and state whether its cost displaces other work or needs a separate decision.

17.3 Model answer: “Approve the identity and customer-handoff pilot within the agreed allocation, conditional on a named business owner and verified access controls. Keep local customer processes while testing governed cross-system links. Release rollout funding only when representative records reconcile, ambiguous matches have an operating process, recovery has been tested and service users complete their work without project-team help. The pilot will report its actual cost and the change in identity-related misrouting, with the remaining uncertainty. Revisit consolidation if shared identifiers cannot support the required meanings or transition costs exceed the approved range.” A different first stage earns full credit when its rationale and conditions are explicit.

17.4 Example criteria, each with the action if it is missed. First, identity-related reassignment in the pilot divisions falls by at least 1.5 percentage points against the comparison division, measured by difference-in-differences over two quarters; if not, investigate the mechanism before expanding. Second, false links stay below 5 per 1,000 in the audit sample; if not, add stewardship and repeat the audit. Third, the operating team resolves exceptions without project help for four consecutive weeks; if not, fund and train the team. Fourth, pilot cost stays within 10% of budget; if not, re-estimate rollout costs before approval. Credit criteria that are measurable, set in advance and linked to a specific action.

Glossary

This glossary collects the technical terms used in the book. Each entry gives the meaning used here; the chapters show the terms at work in the Helios case.

ADKAR. Hiatt's model of individual change: awareness, desire, knowledge, ability and reinforcement, achieved in order. The first weak element is where to act.

Alignment perspective. In the strategic alignment model, a path that starts from one strategy domain, works through a second domain and changes a third, such as strategy execution or competitive potential.

Application portfolio. The set of applications an organisation runs, managed as a group of investments with recorded purpose, owner, cost, dependencies and lifecycle status.

ArchiMate. The Open Group's modelling language for enterprise architecture. Version 4 organises elements into domains and aspects and uses one set of behaviour elements across the business, application and technology domains.

Architecture. The fundamental concepts or properties of a system or enterprise in its environment, together with the principles that govern its realisation and evolution.

Architecture board. A cross-unit body that reviews significant designs and approves standards and exceptions within delegated authority, sending investment and risk decisions to their accountable owners.

Architecture description. A work product, such as a set of views and models, that expresses an architecture for its stakeholders.

Architecture Development Method (ADM). TOGAF's cycle of phases, from the preliminary phase and architecture vision to architecture change management, around requirements management.

Architecture principle. A rule that guides architecture decisions, written with a name, statement, rationale and implications. A useful principle is one that a real proposal could break.

Aspect. In ArchiMate, the classification of an element as active structure (performs behaviour), behaviour (the work done) or passive structure (acted upon).

Assurance. Justified confidence that controls work as intended, based on current evidence with a stated scope and date.

Authoritative source. The system whose value wins when copies of a specific business fact disagree.

Benefit chain. A sequence of observable steps from an intervention to a benefit, each with an owner and a measure.

Benefits register. A record of each expected benefit with its baseline, mechanism, measure, owner, timing, type and evidence of realisation.

Bimodal IT. An organisation of IT into a stability-oriented mode and a speed-oriented mode, each with its own practices.

Business capability. Something an organisation is able to do to achieve a purpose, described independently of process, application and organisational unit.

Capability heat map. A capability map with separate ratings for importance, effectiveness and confidence, each with a recorded basis.

Capability map. A hierarchy of business capabilities, usually two levels deep, used to compare investments across reorganisations and system changes.

Capability-based planning. Planning and funding change by capability increments, with projects as the means of delivering them.

COBIT 2019. ISACA's framework for the governance and management of enterprise information and technology, with five governance objectives, 35 management objectives and design factors for tailoring.

Common-mode failure. A single failure, such as a shared identity configuration or control plane, that affects many components at once.

Conway's law. The observation that organisations design systems whose structure copies their communication structure.

Critical path. The longest chain of dependent activities in a schedule, which sets the earliest completion date.

Customer identity resolution. Deciding which records in different systems describe the same customer organisation, site or contact, and recording the links with their match status.

Data contract. A published agreement about a dataset's meaning, grain, identifiers, freshness, quality, access, change policy and support.

Data owner. The person accountable for the meaning, approved uses and quality decisions of a business fact.

Data steward. The person who maintains definitions, resolves day-to-day data issues such as proposed matches, and coordinates corrections.

Decision rights. The right to make a decision, as distinct from input rights, the right to be consulted.

Design factor. In COBIT 2019, a characteristic of the enterprise, such as its risk profile or sourcing model, that changes the priority of governance and management objectives.

Difference-in-differences. An estimate of an intervention's effect: the change in the treated group minus the change in a comparison group over the same period.

Digital platform. A set of reusable business, data and technology components from which teams can quickly assemble and change digital offerings.

Domain. In ArchiMate 4, a grouping of elements: common, business, application, technology, strategy, motivation, and implementation and migration.

DORA. Either the DevOps Research and Assessment programme and its software delivery measures, or the EU's Digital Operational Resilience Act for financial entities. The context decides which.

Enterprise architecture. The description of how capabilities, processes, information, applications, technology and decision rights fit together, used to choose and sequence change.

Enterprise architecture function. The roles, bodies and processes through which an organisation creates, approves, enforces and observes architecture decisions.

Exception record. A governed, time-limited approval to depart from a standard, with reason, scope, owner, risk, operating controls, expiry and path to compliance.

Expected value of information. The increase in expected value from making a decision after, rather than before, obtaining information such as a pilot result.

Exploration and exploitation. March's distinction between searching for new possibilities and refining what an organisation already knows.

Federated governance. An arrangement in which a central team sets shared identifiers, rules and minimum controls while domain teams own the meaning and quality of their data.

Flow efficiency. Active time divided by elapsed time for a request moving through a value stream.

Functional unit. In the Software Carbon Intensity specification, the unit of work that emissions are expressed per, such as one completed service request.

Grain. What a single row of a dataset represents, such as one customer organisation or one pair of linked records.

Group customer identifier. Helios's governed identifier for a customer organisation, linking the identifiers used in each source system.

Internal rate of return (IRR). The discount rate at which an investment's net present value is zero.

ISO/IEC 38500. The international standard for governance of IT. The governing body evaluates, directs and monitors, and the 2024 edition adds engaging stakeholders and adopts the eleven principles of ISO 37000.

IT governance. The specification of decision rights and accountability that encourages desirable behaviour in the use of IT.

Master data. Core data about the main business entities, such as customers, sites and products, shared by many processes and systems.

Master data management style. Where master values live and are changed: registry, consolidation, coexistence or centralised.

Match status. The state of a proposed link between records: proposed, verified or rejected.

Measurement contract. A written definition of a measure: purpose, owner, population, numerator, denominator, exclusions, timing and known limits.

Mechanism. An explanation of how one change could lead to another, through observable intermediate steps.

Mirroring hypothesis. The proposition that a product's architecture tends to mirror the structure of the organisation that develops it.

Multiplicity. In ArchiMate 4, a constraint on how many element instances may sit at each end of a relationship.

Net present value (NPV). The sum of an investment's future cash flows discounted to a single date, minus the investment.

Nominal and real values. Nominal amounts are in the money of the year they occur; real amounts are in constant prices. Each must be discounted at the matching rate.

Operating model. The level of business process integration and standardisation an organisation needs to deliver its goods and services.

Operational backbone. The integrated core systems, data and processes that run the business reliably and efficiently.

Organisational anchoring. The extent to which enterprise architecture is embedded in an organisation's decisions and processes.

Parallel trends. The assumption, needed for a causal reading of difference-in-differences, that the groups would have moved in parallel without the intervention.

Payback. The time until cumulative benefits, discounted or not, repay an investment. The result depends on the timing convention.

Plateau. In ArchiMate, a relatively stable state of the architecture, such as a baseline, transition state or target.

Priority candidate. In this book's heat-map rule, a capability with high importance, effectiveness of 2 or below and at least medium confidence.

Real option. The right, without an obligation, to take a later action on a real investment, such as expanding, delaying or abandoning it.

Regression to the mean. The tendency of a group selected for an extreme measurement to move towards its average next time, even without any change.

Review trigger. A condition, such as cross-unit impact or a new use of sensitive data, that sends a change for wider review.

Risk. In ISO 31000, the effect of uncertainty on objectives. A risk statement names a condition, an event and a consequence.

Roadmap. A sequence of operable intermediate states, with the work, dependencies and conditions needed to move between them.

Seven Rs. AWS's vocabulary of cloud migration strategies: retire, retain, rehost, relocate, repurchase, replatform and refactor.

Simpson's paradox. A reversal in which a pattern within every group disappears or reverses when the groups are combined.

Slack. The time an activity can be delayed without delaying the whole schedule.

Software Carbon Intensity (SCI). The Green Software Foundation's measure of emissions per functional unit: energy times grid intensity, plus embodied emissions, per unit.

Solution architect. An architect who designs a specific change within enterprise principles and domain standards. Enterprise and domain architects work across the organisation and within one area respectively.

Standardised rate. A rate recalculated with a fixed reference mix of groups, to separate changes in group performance from changes in mix.

Strategic alignment model. Henderson and Venkatraman's model linking business strategy, IT strategy, organisational infrastructure and IT infrastructure through strategic fit and functional integration.

Technical custodian. The person or team that operates the storage, access mechanisms and backups for data.

Technical debt. The future cost of change created by past design shortcuts.

TIME model. A practitioner heuristic, generally attributed to Gartner, that classifies applications by business value and technical fitness as tolerate, invest, migrate or eliminate.

TOGAF Standard. The Open Group's framework for developing and governing enterprise architecture, built around the Architecture Development Method.

Total cost of ownership (TCO). All the costs of an option over the same period: implementation, migration, operation, support, training, upgrades and exit, including the old system's cost during transition.

Transaction cost economics. Williamson’s theory of which transactions to govern through markets and which internally, depending on factors such as asset specificity.

Transition architecture. In TOGAF, an intermediate state of the architecture between baseline and target.

Value stream. The sequence of stages through which value reaches a stakeholder, from trigger to result.

Vendor lock-in. Being unable to move away from a supplier without substantial cost.

View and viewpoint. A view addresses particular stakeholder concerns; the viewpoint sets the conventions for constructing and reading it.

Workforce identity and access. The management of how staff sign in to applications and lose access when they change role or leave.

Zachman Framework. Zachman’s ontology of enterprise descriptions: six interrogatives as columns and six audience perspectives as rows, 36 cells in version 3.0.

Zero trust. A security approach that grants no implicit trust based on network location or ownership, and evaluates each access request against current identity, device state and policy.

Templates and further study

This chapter collects the templates used throughout the book, a self-assessment for the end of the course, further reading for each chapter and suggestions for further study. Copy the templates into any document or spreadsheet. Leave a field blank, marked “unknown”, when the information does not yet exist; an honest gap is more useful than an invented value.

Dossier templates

Decision brief

Field	What to record
Decision, date and owner	The decision requested, when it is needed and who is accountable
Problem and scope	What is wrong today, for whom, and what is out of scope
Recommendation	The proposed option and at least one credible alternative
Mandatory conditions	Legal, security or continuity conditions every option must meet
Cost and consequences	Investment, operating cost changes and transition effects
Uncertainty	What is not yet known and how it will be resolved
Condition for the next stage	The results that release, narrow or stop further funding

Capability assessment

Field	What to record
Capability	Identifier, name, definition and level-1 domain
Importance	High, medium or low, and the strategic objective it supports
Effectiveness	Score from 1 to 5 and the data behind it
Confidence	High, medium or low, and the source of the rating
Supporting elements	Processes, applications and the accountable owner
Proposed action	Measure, quick fix, investment or no change, with the reason

Application inventory record

Field	What to record
Application	Identifier, business purpose and accountable owner
Users and consumers	Interactive users and non-interactive consumers such as batch jobs
Authority and interfaces	Facts it is authoritative for, interfaces and dependencies
Cost	Annual operating cost by type: licences, infrastructure, support, shared services
Lifecycle and contract	Support status, contract end date and exit terms
Assessment	TIME scores and quadrant, and the proposed disposition
Gaps	Missing facts and who will find them

Fact-authority register

Field	What to record
Fact	The business fact or relationship, with its definition
Authority	The authoritative source and the accountable owner
Writers and consumers	Who may change it and who relies on it, with freshness needs
Correction	How disputes and errors are resolved, and by whom
Policy	Access rules and the source of the retention policy

Exception record

Field	What to record
Standard	The standard affected and its owner
Reason and scope	Why compliance is not yet possible and what is covered
Controls	Compensating controls that are operating now, and conditions still to be met
Risk owner	Who accepts the residual risk, within which delegation
Expiry	End date, review trigger and the funded plan to comply

Benefits register

Field	What to record
Benefit	Identifier, affected population and baseline
Mechanism	How the change produces the benefit, and its measure and owner
Type	Cash, capacity or risk reduction
Timing and cost	When it arrives and what it costs to realise
Checks	Possible double counting and alternative causes
Realisation	The data that will show it happened, and the action if it does not

Transition state

Field	What to record
State	Identifier, entry condition and exit condition
Authority	The authoritative source and allowed writers for each fact
Operation	Interfaces, recovery, reconciliation and the support owner
Temporary elements	Bridges and workarounds, with their retirement condition and budget
Work	Work packages, predecessors, durations and resource assumptions
Money and risk	Annual cash need, main risks and the effect of each change card

Source log

Field	What to record
Claim	The claim the source is used for, and its scope
Source	Full reference and the kind of source
Location	Where in the source the support appears
Design	Method, sample and main limits
Inference	How it applies to Helios, and the strongest counterargument
Test	The observation that would confirm or overturn the inference

Self-assessment

Before submitting the capstone, check that you can do each of the following without notes. If one is weak, return to the chapter shown.

I can…	Chapter
Explain why a local solution can raise enterprise cost, with a mechanism	1
Tailor an architecture method to a decision and write principles in a standard format	2
Build a capability heat map that separates importance, effectiveness and confidence	3
Place capabilities in the operating-model grid and justify the placement	4
Score applications with TIME and say what the score leaves out	5
Assign authority for individual facts and design a correction workflow	6
Diagnose a delivery bottleneck using Conway’s law and local data	7
Describe who holds decision rights for each IT decision domain	8
Calculate the availability of a service with shared dependencies	9
Calculate NPV, IRR and payback, and use real and nominal rates consistently	10
Find the critical path and explain how shared resources change it	11
Diagnose an adoption problem and choose between design and training fixes	12
Apply mandatory conditions and find the weight at which a ranking tips	13
Recognise Simpson’s paradox and regression to the mean, and calculate a difference-in-differences	14
Design an AI experiment with consequence-based errors and stop conditions	15
Appraise a study and separate findings, inference and proposal	16

Further reading

Two or three sources for each chapter, chosen to take the topic further. Most are cited in the chapter itself; the reference list gives full details.

- **Chapter 1, The enterprise architecture problem.** Ross, Weill and Robertson set out the organising-logic view of enterprise architecture that this book builds on [2]. ISO/IEC/IEEE 42010 defines the vocabulary of architecture descriptions [1]. Van der Raadt and van Vliet show how to design an architecture function [3].
- **Chapter 2, Frameworks, methods and modelling.** Read the ArchiMate 4 Specification [12] with Lankhorst’s summary of what changed [13]. The TOGAF Standard [9] is the primary source for the ADM, and Kotusev’s case study shows how one organisation actually used it [16].

- **Chapter 3, Capabilities, value streams and requirements.** The TOGAF series guides on business capabilities [19], value streams [20] and capability-based planning [15] give definitions and worked examples.
- **Chapter 4, Operating models and alignment.** Ross, Beath and Mocker develop the operational backbone and digital platform [23]. Henderson and Venkatraman's paper remains the clearest statement of the strategic alignment model [22], and Ross's briefing argues for focusing IT on the operating model [21].
- **Chapter 5, Application portfolios.** The AWS guidance explains each migration strategy [24]. Ellram's analysis of total cost of ownership [45] and Opara-Martins and colleagues on lock-in [57] cover the economics of leaving a system or supplier.
- **Chapter 6, Information and data governance.** DAMA-DMBOK covers every knowledge area of data management [25]. The General Data Protection Regulation itself, especially its principles and its requirement for data protection by design, is short enough to read directly [26].
- **Chapter 7, Technology strategy and platforms.** Read Conway's original paper [29], the mirroring study [30] and Skelton and Pais on designing team boundaries [31].
- **Chapter 8, Governance and decision rights.** Weill and Ross's book is the source of the domains and archetypes [32]. COBIT 2019 [36] and ISO/IEC 38500 [34] give the standard frameworks for governance of IT.
- **Chapter 9, Risk, security and assurance.** The NIST Cybersecurity Framework 2.0 [38] and NIST SP 800-207 on zero trust [40] are free and readable; ISO 31000 frames risk around objectives [39].
- **Chapter 10, Investment economics.** The OpenStax finance chapters on NPV [41] and IRR [42] cover the methods with exercises. Benaroch and Kauffman develop real options for IT [47], and Peppard, Ward and Daniel explain benefits management [46].
- **Chapter 11, Roadmaps and transitions.** Kelley and Walker's paper introduced the critical path method [48]; TOGAF's phases E and F describe transition architectures and migration planning [10].
- **Chapter 12, Change and adoption.** Cummings, Bridgman and Brown re-examine Lewin [50], Appelbaum and colleagues review the evidence for Kotter's model [52], and Dang and Pekkola follow an architecture adoption over time [54].
- **Chapter 13, Sourcing and suppliers.** Lacity, Khan and Willcocks summarise outsourcing research [56]; Williamson's paper explains transaction cost reasoning [55].
- **Chapter 14, Measuring outcomes.** Huntington-Klein's chapter on difference-in-differences is free online [61]. Tamm and colleagues model how EA creates value [5], and Forsgren, Humble and Kim present the research behind the DORA measures [63].
- **Chapter 15, Innovation, AI and sustainability.** March's paper on exploration and exploitation [66], the NIST AI Risk Management Framework [67] and the Software Carbon Intensity specification [68] are the primary sources.
- **Chapter 16, Research methods.** Recker's introduction covers research design, methods and writing in information systems [69]. Kotusev's study of architecture artefacts is a good example to appraise [17].
- **Chapter 17, Capstone.** Weill and Ross's one-page governance summary [33] and Lankhorst and colleagues on enterprise modelling and analysis [70] help with the dossier and its defence.

Further study

To transfer the course to your own work, choose one real problem whose details you are permitted to use. Define the decision, gather authorised data and reuse the reasoning structures from the book, replacing the Helios assumptions with your organisation's facts and verifying current technical, legal and contractual conditions. For deeper research work, compare two primary studies that address the same question, analysing their constructs, methods, contexts and limits before drawing a recommendation. If your organisation's models are still in ArchiMate 3.2, plan their conversion to version 4 as a small project of its own, using the differences listed in Chapter 2.

References

References are numbered in order of first citation. Online sources were accessed in October 2026. Product documentation changes; check it again before implementation.

[1] ISO/IEC/IEEE, *Software, systems and enterprise — Architecture description*, ISO/IEC/IEEE 42010:2022, 2nd ed., Nov. 2022. Accessed: Oct. 11, 2026. [Online]. Available: https://www.iso.org/standard/74393.html

[2] J. W. Ross, P. Weill, and D. C. Robertson, *Enterprise architecture as strategy: Creating a foundation for business execution*, Boston, MA, USA: Harvard Business School Press, 2006, ISBN 978-1-59139-839-4. Accessed: Oct. 11, 2026. [Online]. Available: https://store.hbr.org/product/enterprise-architecture-as-strategy-creating-a-foundation-for-business-execution/8398

[3] B. van der Raadt and H. van Vliet, “Designing the enterprise architecture function,” in *Quality of Software Architectures: Models and Architectures (QoSA 2008)*, S. Becker, F. Plasil, and R. Reussner, Eds., LNCS 5281. Berlin, Germany: Springer, 2008, pp. 103-118. doi: 10.1007/978-3-540-87879-7_7

[4] M. Lange, J. Mendling, and J. Recker, “An empirical analysis of the factors and measures of enterprise architecture management success,” *European Journal of Information Systems*, vol. 25, no. 5, pp. 411-431, Sep. 2016. doi: 10.1057/ejis.2014.39

[5] T. Tamm, P. B. Seddon, G. Shanks, and P. Reynolds, “How does enterprise architecture add value to organisations?,” *Communications of the Association for Information Systems*, vol. 28, art. 10, pp. 141-168, Mar. 2011, doi: 10.17705/1CAIS.02810. Accessed: Oct. 11, 2026. [Online]. Available: https://aisel.aisnet.org/cais/vol28/iss1/10/

[6] J. A. Zachman, “A framework for information systems architecture,” *IBM Systems Journal*, vol. 26, no. 3, pp. 276-292, 1987. doi: 10.1147/sj.263.0276

[7] J. F. Sowa and J. A. Zachman, “Extending and formalizing the framework for information systems architecture,” *IBM Systems Journal*, vol. 31, no. 3, pp. 590-616, 1992. doi: 10.1147/sj.313.0590

[8] J. P. Zachman, “The Zachman Framework evolution (part 2),” *Business Rules Journal*, vol. 16, no. 6, Jun. 2015. Accessed: Oct. 11, 2026. [Online]. Available: https://www.brcommunity.com/articles.php?id=b817

[9] The Open Group, “The TOGAF Standard, 10th edition: Introduction and core concepts,” Reading, UK: The Open Group, document C220, Apr. 2022. Accessed: Oct. 11, 2026. [Online]. Available: https://www.opengroup.org/library/c220

[10] The Open Group, “The TOGAF Standard, 10th edition: Architecture development method,” Reading, UK: The Open Group, document C220, Apr. 2022. Accessed: Oct. 11, 2026. [Online]. Available: https://pubs.opengroup.org/togaf-standard/adm/chap01.html

[11] The Open Group, “Architecture principles,” in *The TOGAF Standard, 10th Edition: ADM Techniques*, Reading, UK: The Open Group, document C220, Apr. 2022, ch. 2. Accessed: Oct. 11, 2026. [Online]. Available: https://pubs.opengroup.org/togaf-standard/adm-techniques/chap02.html

[12] The Open Group, “ArchiMate 4 specification,” Reading, UK: The Open Group, document C260, Apr. 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://publications.opengroup.org/standards/archimate/c260

[13] M. Lankhorst, “What really changes in the ArchiMate 4 specification and what does not,” Bizzdesign blog, Aug. 20, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://bizzdesign.com/blog/archimate-4-changes

[14] The Open Group, “ArchiMate 3.2 specification,” Reading, UK: The Open Group, document C226, Oct. 2022. Accessed: Oct. 11, 2026. [Online]. Available: https://pubs.opengroup.org/architecture/archimate32-doc/

[15] The Open Group, “TOGAF series guide: Business capability planning,” Reading, UK: The Open Group, document G233, Apr. 2023. Accessed: Oct. 11, 2026. [Online]. Available: https://pubs.opengroup.org/togaf-standard/business-architecture/business-capability-planning.html

[16] S. Kotusev, “TOGAF-based enterprise architecture practice: An exploratory case study,” *Communications of the Association for Information Systems*, vol. 43, paper 20, pp. 321-359, Sep. 2018, doi: 10.17705/1CAIS.04320. Accessed: Oct. 11, 2026. [Online]. Available: https://aisel.aisnet.org/cais/vol43/iss1/20/

[17] S. Kotusev, “Enterprise architecture and enterprise architecture artifacts: Questioning the old concept in light of new findings,” *Journal of Information Technology*, vol. 34, no. 2, pp. 102-128, Jun. 2019. doi: 10.1177/0268396218816273

[18] S. Brown, “The C4 model for visualising software architecture,” Accessed: Oct. 11, 2026. [Online]. Available: https://c4model.com/

[19] The Open Group, “TOGAF series guide: Business capabilities, version 2,” Reading, UK: The Open Group, document G211, Apr. 2022. Accessed: Oct. 11, 2026. [Online]. Available: https://pubs.opengroup.org/togaf-standard/business-architecture/business-capabilities.html

[20] The Open Group, “TOGAF series guide: Value streams,” Reading, UK: The Open Group, document G178, Apr. 2022. Accessed: Oct. 11, 2026. [Online]. Available: https://pubs.opengroup.org/togaf-standard/business-architecture/value-streams.html

[21] J. W. Ross, “Forget strategy: Focus IT on your operating model,” *Research Briefing*, vol. V, no. 3C, MIT Sloan Center for Information Systems Research, Dec. 2005. Accessed: Oct. 11, 2026. [Online]. Available: https://cisr.mit.edu/publication/2005_12_3C_OperatingModels

[22] J. C. Henderson and N. Venkatraman, “Strategic alignment: Leveraging information technology for transforming organizations,” *IBM Systems Journal*, vol. 38, no. 2.3, pp. 472-484, 1999; reprint of vol. 32, no. 1, pp. 4-16, 1993. doi: 10.1147/sj.382.0472

[23] J. W. Ross, C. M. Beath, and M. Mocker, *Designed for digital: How to architect your business for sustained success*, Cambridge, MA, USA: MIT Press, 2019, ISBN 978-0-262-04288-8. Accessed: Oct. 11, 2026. [Online]. Available: https://mitpress.mit.edu/9780262042888/designed-for-digital/

[24] Amazon Web Services, “About the migration strategies,” in *Guide for AWS Large Migrations*, AWS Prescriptive Guidance. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/prescriptive-guidance/latest/large-migration-guide/migration-strategies.html

[25] DAMA International, *DAMA-DMBOK: Data management body of knowledge*, 2nd ed., revised. Technics Publications, 2024. Accessed: Oct. 11, 2026. [Online]. Available: https://technicspub.com/dmbok2/

[26] European Parliament and Council of the European Union, “Regulation (EU) 2016/679 (General Data Protection Regulation),” *Official Journal of the European Union*, L 119, pp. 1-88, May 4, 2016. Accessed: Oct. 11, 2026. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2016/679

[27] Microsoft, “Microsoft Azure Well-Architected Framework,” Microsoft Learn. Accessed: Oct. 11, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/well-architected/pillars

[28] Amazon Web Services, “The pillars of the framework,” in *AWS Well-Architected Framework*. Accessed: Oct. 11, 2026. [Online]. Available: https://docs.aws.amazon.com/wellarchitected/latest/framework/the-pillars-of-the-framework.html

[29] M. E. Conway, “How do committees invent?,” *Datamation*, vol. 14, no. 4, pp. 28-31, Apr. 1968. Accessed: Oct. 11, 2026. [Online]. Available: https://www.melconway.com/Home/Committees_Paper.html

[30] A. MacCormack, C. Baldwin, and J. Rusnak, “Exploring the duality between product and organizational architectures: A test of the mirroring hypothesis,” *Research Policy*, vol. 41, no. 8, pp. 1309-1324, Oct. 2012. doi: 10.1016/j.respol.2012.04.011

[31] M. Skelton and M. Pais, *Team topologies: Organizing business and technology teams for fast flow*, Portland, OR, USA: IT Revolution Press, 2019, ISBN 978-1-942788-81-2. Accessed: Oct. 11, 2026. [Online]. Available: https://openlibrary.org/books/OL28745421M/Team_Topologies

[32] P. Weill and J. W. Ross, *IT governance: How top performers manage IT decision rights for superior results*, Boston, MA, USA: Harvard Business School Press, 2004, ISBN 978-1-59139-253-8. Accessed: Oct. 11, 2026. [Online]. Available: https://store.hbr.org/product/it-governance-how-top-performers-manage-it-decision-rights-for-superior-results/2535

[33] P. Weill and J. W. Ross, “IT governance on one page,” MIT Sloan Center for Information Systems Research, working paper no. 349, Nov. 2004. Accessed: Oct. 11, 2026. [Online]. Available: https://cisr.mit.edu/publication/MIT_CISRwp349_ITGovOnOnePage

[34] ISO/IEC, *Information technology — Governance of IT for the organization*, ISO/IEC 38500:2024, 3rd ed., Feb. 2024. Accessed: Oct. 11, 2026. [Online]. Available: https://www.iso.org/standard/81684.html

[35] M. Shanahan, *White paper on the differences between ISO/IEC 38500:2015 and ISO/IEC 38500:2024*, ISO/IEC JTC 1/SC 40 Working Group 1, Apr. 13, 2024. Accessed: Oct. 11, 2026. [Online]. Available: https://committee.iso.org/sites/jtc1sc40/home/projects/wg-1/published-wg1/content-left-area/isoiec-sc-40-wg-1-develops-the-s/white-paper-on-the-differences-b.html

[36] ISACA, “COBIT 2019 framework: Introduction and methodology,” Schaumburg, IL, USA: ISACA, 2018. Accessed: Oct. 11, 2026. [Online]. Available: https://www.isaca.org/resources/cobit

[37] J. Jöhnk, S. Oesterle, T. J. Winkler, J. Nørbjerg, and N. Urbach, “Juggling the paradoxes: Governance mechanisms in bimodal IT organizations,” in *Proceedings of the 27th European Conference on Information Systems*, Stockholm and Uppsala, Sweden, Jun. 2019, research paper 93. Accessed: Oct. 11, 2026. [Online]. Available: https://aisel.aisnet.org/ecis2019_rp/93

[38] National Institute of Standards and Technology, “The NIST Cybersecurity Framework (CSF) 2.0,” NIST CSWP 29, Feb. 2024. doi: 10.6028/NIST.CSWP.29

[39] International Organization for Standardization, *Risk management — Guidelines*, ISO 31000:2018, 2nd ed., Feb. 2018. Accessed: Oct. 11, 2026. [Online]. Available: https://www.iso.org/standard/65694.html

[40] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” NIST SP 800-207, Aug. 2020. doi: 10.6028/NIST.SP.800-207

[41] J. Dahlquist and R. Knight, “Net present value (NPV) method,” in *Principles of Finance*, Houston, TX, USA: OpenStax, 2022, sec. 16.2. Accessed: Oct. 11, 2026. [Online]. Available: https://openstax.org/books/principles-finance/pages/16-2-net-present-value-npv-method

[42] J. Dahlquist and R. Knight, “Internal rate of return (IRR) method,” in *Principles of Finance*, Houston, TX, USA: OpenStax, 2022, sec. 16.3. Accessed: Oct. 11, 2026. [Online]. Available: https://openstax.org/books/principles-finance/pages/16-3-internal-rate-of-return-irr-method

[43] J. Dahlquist and R. Knight, “Payback period method,” in *Principles of Finance*, Houston, TX, USA: OpenStax, 2022, sec. 16.1. Accessed: Oct. 11, 2026. [Online]. Available: https://openstax.org/books/principles-finance/pages/16-1-payback-period-method

[44] J. Dahlquist and R. Knight, “Applications of TVM in finance,” in *Principles of Finance*, Houston, TX, USA: OpenStax, 2022, sec. 7.4. Accessed: Oct. 11, 2026. [Online]. Available: https://openstax.org/books/principles-finance/pages/7-4-applications-of-tvm-in-finance

[45] L. M. Ellram, “Total cost of ownership: An analysis approach for purchasing,” *International Journal of Physical Distribution & Logistics Management*, vol. 25, no. 8, pp. 4-23, 1995. doi: 10.1108/09600039510099928

[46] J. Peppard, J. Ward, and E. Daniel, “Managing the realization of business benefits from IT investments,” *MIS Quarterly Executive*, vol. 6, no. 1, art. 3, pp. 1-11, 2007. Accessed: Oct. 11, 2026. [Online]. Available: https://aisel.aisnet.org/misqe/vol6/iss1/3/

[47] M. Benaroch and R. J. Kauffman, “A case for using real options pricing analysis to evaluate information technology project investments,” *Information Systems Research*, vol. 10, no. 1, pp. 70-86, Mar. 1999. doi: 10.1287/isre.10.1.70

[48] J. E. Kelley and M. R. Walker, “Critical-path planning and scheduling,” in *Papers Presented at the Eastern Joint IRE-AIEE-ACM Computer Conference*, Dec. 1959, pp. 160-173. doi: 10.1145/1460299.1460318

[49] K. Lewin, “Frontiers in group dynamics: Concept, method and reality in social science; social equilibria and social change,” *Human Relations*, vol. 1, no. 1, pp. 5-41, Jun. 1947. doi: 10.1177/001872674700100103

[50] S. Cummings, T. Bridgman, and K. G. Brown, “Unfreezing change as three steps: Rethinking Kurt Lewin’s legacy for change management,” *Human Relations*, vol. 69, no. 1, pp. 33-60, Jan. 2016. doi: 10.1177/0018726715577707

[51] J. P. Kotter, “Leading change: Why transformation efforts fail,” *Harvard Business Review*, vol. 73, no. 2, pp. 59-67, Mar.-Apr. 1995. Accessed: Oct. 11, 2026. [Online]. Available: https://hbr.org/1995/05/leading-change-why-transformation-efforts-fail-2

[52] S. H. Appelbaum, S. Habashy, J.-L. Malo, and H. Shafiq, “Back to the future: Revisiting Kotter’s 1996 change model,” *Journal of Management Development*, vol. 31, no. 8, pp. 764-782, 2012. doi: 10.1108/02621711211253231

[53] J. M. Hiatt, *ADKAR: A model for change in business, government and our community*, Prosci, 2006, ISBN 978-1-930885-50-9. Accessed: Oct. 11, 2026. [Online]. Available: https://openlibrary.org/isbn/9781930885509

[54] D. Dang and S. Pekkola, “Organizational change and enterprise architecture adoption: A case study in the public sector,” in *Information Systems Research in Vietnam: A Shared Vision and New Frontiers*, N. H. Thuan, D. Dang-Pham, H.-S. Le, and T. Q. Phan, Eds. Singapore: Springer, 2023, pp. 49-64. doi: 10.1007/978-981-19-3804-7_4

[55] O. E. Williamson, “The economics of organization: The transaction cost approach,” *American Journal of Sociology*, vol. 87, no. 3, pp. 548-577, Nov. 1981. doi: 10.1086/227496

[56] M. C. Lacity, S. A. Khan, and L. P. Willcocks, “A review of the IT outsourcing literature: Insights for practice,” *Journal of Strategic Information Systems*, vol. 18, no. 3, pp. 130-146, Sep. 2009. doi: 10.1016/j.jsis.2009.06.002

[57] J. Opara-Martins, R. Sahandi, and F. Tian, “Critical analysis of vendor lock-in and its impact on cloud computing migration: A business perspective,” *Journal of Cloud Computing*, vol. 5, art. 4, Apr. 2016. doi: 10.1186/s13677-016-0054-z

[58] D. T. Campbell, “Assessing the impact of planned social change,” *Evaluation and Program Planning*, vol. 2, no. 1, pp. 67-90, 1979. doi: 10.1016/0149-7189(79)90048-X

[59] E. H. Simpson, “The interpretation of interaction in contingency tables,” *Journal of the Royal Statistical Society, Series B*, vol. 13, no. 2, pp. 238-241, 1951. doi: 10.1111/j.2517-6161.1951.tb00088.x

[60] A. G. Barnett, J. C. van der Pols, and A. J. Dobson, “Regression to the mean: What it is and how to deal with it,” *International Journal of Epidemiology*, vol. 34, no. 1, pp. 215-220, Feb. 2005. doi: 10.1093/ije/dyh299

[61] N. Huntington-Klein, “Difference-in-differences,” in *The Effect: An Introduction to Research Design and Causality*, Boca Raton, FL, USA: Chapman & Hall/CRC, 2021, ch. 18. Accessed: Oct. 11, 2026. [Online]. Available: https://theeffectbook.net/ch-DifferenceinDifference.html

[62] R. van de Wetering, T. Hendrickx, S. Brinkkemper, and S. Kurnia, “The impact of EA-driven dynamic capabilities, innovativeness, and structure on organizational benefits: A variance and fsQCA perspective,” *Sustainability*, vol. 13, no. 10, art. 5414, May 2021. doi: 10.3390/su13105414

[63] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The science of lean software and DevOps*, Portland, OR, USA: IT Revolution Press, 2018, ISBN 978-1-942788-33-1. Accessed: Oct. 11, 2026. [Online]. Available: https://itrevolution.com/product/accelerate/

[64] N. Harvey, “DORA’s software delivery performance metrics,” DORA guides, Google Cloud, updated Jan. 5, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://dora.dev/guides/dora-metrics/

[65] European Parliament and Council of the European Union, “Regulation (EU) 2022/2554 on digital operational resilience for the financial sector,” *Official Journal of the European Union*, L 333, pp. 1-79, Dec. 27, 2022. Accessed: Oct. 11, 2026. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2022/2554/oj

[66] J. G. March, “Exploration and exploitation in organizational learning,” *Organization Science*, vol. 2, no. 1, pp. 71-87, Feb. 1991. doi: 10.1287/orsc.2.1.71

[67] National Institute of Standards and Technology, “Artificial intelligence risk management framework (AI RMF 1.0),” NIST AI 100-1, Jan. 2023. doi: 10.6028/NIST.AI.100-1

[68] Green Software Foundation, “Software carbon intensity (SCI) specification,” version 1.1.0; also published as ISO/IEC 21031:2024. Accessed: Oct. 11, 2026. [Online]. Available: https://sci.greensoftware.foundation/

[69] J. Recker, *Scientific research in information systems: A beginner’s guide*, 2nd ed. Cham, Switzerland: Springer, 2021. doi: 10.1007/978-3-030-85436-2

[70] M. Lankhorst et al., *Enterprise architecture at work: Modelling, communication and analysis*, 4th ed. Berlin, Germany: Springer, 2017. doi: 10.1007/978-3-662-53933-0

Index

Page numbers point to definitions and main discussions in the chapters.

A

active structure 15
ADKAR 62–63, 66
alignment perspective 24, 26
annual total 58, 60
application portfolio 28
ArchiMate 11, 14–17, 79
architecture 5–7, 11–13
architecture board 7, 9, 46
architecture description 5, 11
Architecture Development Method (ADM) 11–13
architecture principle 11, 14, 17, 43, 82
aspect 11, 14–17
assurance 47–49, 60, 82–83
authoritative source 14, 32, 35

B

backward pass 59
benefit chain 8–9
benefit enablers 73
benefits register 43, 51, 54, 56, 82
Bimodal IT 42, 45–46
bounded experiments 76, 78
bounded learning 78
business capability 16, 18
business units 19, 24–25, 43–44, 63

C

capability heat map 18, 20
capability map 16, 18–21, 82
capability-based planning 16
capacity benefits 54
causal claim 8–9
causal effect 79, 81
central allocation 45
COBIT 2019 42–43, 46
common domain 15–16
common-mode failure 38, 40
consumption charges 45
Conway's law 37, 39–41
critical path 57–59, 61
critical path method 58
customer identity resolution 19–20, 51, 58, 67, 84

D

data contract 9, 31–32, 34, 36
data governance 29, 32, 34
data owner 17, 32–33, 46, 64–65
data steward 11, 32–33, 44
decision domains 42, 46
decision rights 9, 14, 42–43, 46, 82
design factor 43, 46
difference-in-differences 71–73, 75, 79
digital platform 22, 24–27
domain 6, 15–16, 24, 42–43
domain architects 6–7
DORA 74

E

engage stakeholders 43
enterprise architects 6
enterprise architecture 5–6, 9, 11, 16, 73
enterprise architecture function 5–6
exception record 44
expected value of the information 55
exploration and exploitation 76, 78–79

F

federated governance 34
flow efficiency 19, 21
forward pass 59
foundation for execution 22
functional integration 24, 27
functional unit 76–78
fundamental content 12

G

governance objectives 43, 46
grain 34–36
group customer identifier 33–34, 42, 65

I

implementation and migration 15–16
input rights 43, 46
internal rate of return (IRR) 51–52
ISO/IEC 38500 42–44, 46
IT governance 24, 42–43, 46

K

Kotter 62–63, 66

L

Lewin 62–63, 66

M

management objectives 43–44, 46
master data 32–35
master data management style 32–33
match status 33, 35
measurement contract 71, 75
mechanism 7–8, 45–46, 79, 82
mirroring hypothesis 37, 39, 41
modelling language 11, 14, 17
multiplicity 15–16

N

net present value (NPV) 51

O

operating model 22–23, 26, 33, 79, 82
operational backbone 22, 24, 26–27
organisational anchoring 74

P

parallel trends 73
passive structure 15
payback 51–53, 56
plateau 15–16, 57
platform team 25, 37, 39, 45, 63–64
policy administrator 48
policy enforcement point 48
policy engine 48
priority candidate 20
process integration 22–23, 26, 79
process standardisation 22–23, 26
project decision 37, 40

R

real option 51, 54, 56
reference design 6, 37, 40
regression to the mean 71–73, 75

requirements management 13, 17
review trigger 7, 14, 42, 44
risk 43–44, 47–48, 54, 78
risk benefits 51, 54, 56
risk statement 47–50
roadmap 51, 57, 60–61, 82–83

S

series guides 12
Seven Rs 30–31
shared platform teams 25
Simpson's paradox 71, 75
slack 57–61
Software Carbon Intensity 77
Software Carbon Intensity (SCI) 77
solution architect 7
standardised rate 71–72
strategic alignment model 22, 24, 26
strategic fit 24, 27
strict reuse 78

T

target architecture 44, 48, 57
technical custodian 32–33
technical debt 39
TIME model 28, 31
TOGAF Standard 12
total cost of ownership (TCO) 51, 54, 67
transaction cost economics 67
transition architecture 13, 57

V

value stream 16, 18–21, 82
vendor lock-in 67

W

workforce identity and access 28, 37, 51, 57–58, 60

Z

Zachman Framework 11, 17
zero trust 47–48, 50